



MENOUFIA UNIVERSITY
FACULTY OF ARTIFICIAL INTELLIGENCE



Project Titel

MooNet

“Livestock in the Era of AI and IoT”

A Graduation Project Submitted for the Degree of
Bachelor in Artificial Intelligence

Project Team:

Name	Department
Heba Atef Mohammed Tolan	IS
Mennat-Allah Yousri Rabeh	MI
Mennat-Allah Medhat Mostafa	MI
Sohila Ayman Mohammed Lashien	MI
Abdelrahman El-Sayed Mohammed	MI

Supervised by:

Assoc.Prof. Mina Ibrahim Samaan Ibrahim

Machine intelligence Department
Faculty of Artificial Intelligence
Menoufia University

Spring 2025

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **God**, whose guidance and blessings made this achievement possible.

Our heartfelt thanks go to our **families** for their unwavering support, patience, and belief in us throughout this journey. Their encouragement has been a constant source of strength.

We are deeply thankful to **Dr. Mina Ibrahim**, our project supervisor, for his continuous support, insightful guidance, and patience. His mentorship was instrumental to our progress.

We also thank the **veterinarians** who generously offered their time and expertise, providing valuable insights that greatly enhanced the quality and relevance of our work.

To our **friends, teachers, colleagues, and mentors and everyone who supported us** who have supported us throughout our academic journey or encouraged us.

This project is the result of collaboration, dedication, and the unwavering support of those who believed in us.

Thank you.

Team Members

ABSTRACT

In an era marked by rapid technological advancement, the exchange of data has become fundamental across various sectors, including education, banking, e-commerce, and digital government services. This digital transformation calls for smart, efficient, and secure systems that can support and optimize real-world applications.

In this context, the MooNet project introduces a smart and integrated livestock management system that leverages Artificial Intelligence (AI) and the Internet of Things (IoT) to enhance animal health monitoring, environmental control, and feeding efficiency. MooNet empowers farmers with real-time data and intelligent tools to make informed decisions on the farm.

The system utilizes YOLOv8 for real-time detection of cows, while disease diagnosis including conditions such as Mastitis and Lumpy Skin Disease is handled by advanced deep learning classification models trained on both visual and sensor data.

Additionally, the system features a multi-agent feed optimization module, an LLM-powered chatbot assistant for interactive support, and a mobile application for real-time monitoring and control.

By combining computer vision, data analytics, and natural language processing, MooNet offers a unified, scalable, and cost-effective platform that transforms traditional cattle farming into a smart and sustainable operation. It aligns with national goals for digital agriculture and food security, particularly in regions such as Egypt and the Middle East.

Keywords:

Artificial Intelligence (AI), Internet of Things (IoT), Smart Farming, Livestock Monitoring, Disease Detection, Cow Health, Feed Optimization, YOLOv8, Lumpy Skin Disease, Mastitis, Multi-Agent Systems, Mobile Application, Real-Time Monitoring, UWB Localization, 3D Camera, Environmental Control, Agricultural Technology, Precision Livestock Farming, Chatbot Assistant, Deep Learning.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	iii
ABSTRACT	4
TABLE OF CONTENTS	6
LIST OF FIGURES	13
LIST OF TABLES	16
LIST OF ABBREVIATIONS.....	17
PROJECT SUMMARY	18
INTRODUCTION	20
1.1 Overview	20
1.2 Objectives.....	20
1.3 Methodology	21
1.4 Document Organization	22
PROJECT CONTEXT.....	25
2.1 Introduction	25
2.1 System Overview.....	26
2.2 Impact.....	27
2.3 Novelty & Features	29
2.3.1 Novelty	29
2.3.2 Our Features	30
2.3.3 Related Product in The Market	33
2.4 Business Model	34
2.4.1 Customer Segments.....	34
2.4.2 Value Propositions.....	34
2.4.3 Channels.....	35
2.4.4 Customer Relationships	36
2.4.5 Revenue Streams.....	36

2.4.6 Key Resources.....	36
2.4.7 Key Activities.....	36
2.4.8 Key Partnerships	36
2.4.9 Cost Structure.....	37
2.4.10 Our Business Model	37
MARKET ANALYSIS.....	38
3.1 Introduction	38
3.2 Industry Overview.....	38
3.2.1 Global Livestock Monitoring Market.....	38
3.2.2 Local Livestock Market (Egypt & MEA Region).....	41
3.2.3 Market Trends Driving Growth.....	43
3.3 Target Audience.....	43
3.4 Competitive Analysis	44
3.5 SWOT Analysis.....	45
3.5.1 Strengths	45
3.5.2 Weaknesses	46
3.5.3 Opportunities	47
3.5.4 Threats	48
3.5.5 Solutions for Weaknesses	49
3.5.6 Solutions for threats.....	50
3.6 Our SWOT	51
RELATED WORK	52
4.1 Introduction	52
4.2 Existing Competitors.....	52
4.3 Competitive Analysis	56
SYSTEM DESIGN	57

5.1 Introduction	57
5.2 Data Flow Diagram	57
5.2.1 System Scenario	57
5.2.2 Context Diagram	58
5.2.3 Level 0 Diagram	59
5.3 AI System Design.....	60
5.3.1 AI Workflow for Disease Detection and Report Generation.....	60
5.3.2 LLM-Based Multi-Agent Framework for Smart Query Answering	62
5.3.3 Feed Optimization System Using CrewAI.....	63
5.4 Conclusion.....	64
HARDWARE IMPLEMENTATION.....	65
6.1 Introduction	65
6.2 Data Flow Description	66
6.3 Sensor Modules.....	67
6.3.1 DHT11 (Temperature sensor).....	67
6.3.2 LDR (Light Dependent Resistor)	68
6.3.3 MQ-137 (Ammonia (NH ₃) Gas Sensor).....	69
6.3.4 Water Level Sensor.....	70
6.3.5 pH sensor	71
6.3.6 VOC Sensor (Acetone Detection)	72
6.4 Processing Unit	74
6.5 Communication Module.....	75
6.6 Arduino code	76
6.7 ESP8266 NodeMCU code.....	80
6.8 Detecting and tracking	82
6.8.1 Objectives	82

6.8.2 Scenario Overview	82
6.8.3 System Architecture.....	84
6.8.4 DWM1000 Arduino Connection	85
DISEASE DETECTION SYSTEM IMPLEMENTATION	86
7.1 Introduction to AI System Integration and Implementation	86
7.1.1 Overview of the Implemented AI Solutions.....	86
7.1.2 Data Ingestion and Preprocessing	88
7.2 Model Architectures and Training Pipelines.....	93
7.2.1 Mastitis Detection Model	93
7.2.2 Lumpy Skin Disease Detection Model.....	97
7.2.3 Cow Detection Using YOLOv8	101
7.3 Real-time Inference and LLM Integration	103
7.3.1 Individual Model Inference	103
7.4 Centralized Inference and LLM Report Generation	107
7.4.1 Overview	107
7.4.2 System Flow Overview	108
7.4.3 Code Modules and Explanations.....	108
7.5 Performance Evaluation	111
7.5.1 Mastitis Detection Results and Analysis	111
7.5.2 Lumpy Skin Detection Results and Analysis	116
7.5.3 YOLOv8 Cow Detection Results and Analysis.....	120
FEED OPTIMIZATION SYSTEM	123
8.1 System Overview and Motivation.....	123
8.2 Agent-Based Architecture with CrewAI	123
8.2.1 Overview of CrewAI Framework.....	123
8.2.2 Agents	124

8.2.3 Agent Workflow & Interaction	125
8.3 LLM Integration and Implementation Details	125
8.3.1 LLM for Report Synthesis.....	125
8.3.2 Code Snippets and Tools Used	125
8.4 Output Testing and Validation.....	128
8.4.1 Sample Cow Profile Input	129
8.4.2 Ideal Feed Formulation Output	129
8.4.3 Nutrition Balance Evaluation Output	130
8.4.4 Best Feed Recommendation Endpoint	131
CHATBOT SYSTEM	132
9.1 Introduction	132
9.2 System Architecture and Agent Design	132
9.3 Question Classification and Routing Logic	133
9.4 RAG-Based Document Reasoning.....	134
9.4.1 Objective and Use Case.....	134
9.4.2 Knowledge Base Construction	134
9.4.3 Semantic Answering Pipeline.....	136
9.5 Structured Data Analysis with Pandas Agent.....	137
9.5.1 Purpose	137
9.5.2 Workflow and Execution	138
9.6 Real-Time Knowledge via Web Search Agent.....	141
9.6.1 Objective.....	141
9.6.2 Process Flow.....	141
9.7 Centralized Smart Answer Routing.....	143
9.8 Output Formatting and User Interaction	144
9.8.1 Example 1 – Structured Data Query (Pandas Agent)	144

9.8.2 Example 2 – Internal Knowledge Query (RAG Agent)	145
9.8.3 Example 3 – External Knowledge Query (Web Search Agent)	146
9.9 Conclusion.....	147
MOBILE APPLICATION	148
10.1 Introduction	148
10.2 User Scenario and Interface Design	149
10.2.1 Personas	149
10.2.2 User Journey Maps	151
10.2.3 Initial Sketches	152
10.2.4 Final UI Design	156
10.2.5 User Scenarios	164
10.2.6 UI / UX Prototype	171
10.3 Backend Implementation	172
10.3.1 Introduction	172
10.3.2 Database Models (SQLAlchemy ORM)	173
10.3.3 API Schemas (Pydantic Models).....	174
10.3.4 Real-Time Camera Integration and Health Analysis.....	177
10.3.5 Cow Management and Notification Endpoints	181
10.3.6 User Registration and Authentication.....	187
10.3.7 Chatbot and Smart Answer APIs	191
10.3.8 Feed Recommendation and Optimization Endpoints.....	193
10.3.9 Conclusion.....	197
10.4 Flutter Implementation.....	198
10.4.1 Account Creation and Login Pages	200
10.4.2 Home Page.....	201
10.4.3 Feed Mangement	204

10.4.4 Chat Bot.....	206
10.4.5 Cow Health Monitoring.....	208
10.4.6 Flutter and Dart Code Snippets	210
10.4.7 Backend Integration and API Functionality	215
CONCLUSION AND FUTURE WORK.....	222
11.1 Introduction	222
11.2 Conclusions.....	222
11.3 Concluding Remarks.....	223
11.4 Future Work	224
BIBLIOGRAPHY	225
ملخص المشروع.....	228
الموجز	229

LIST OF FIGURES

Figure 2.1: Business Model	37
Figure 3.1: Livestock Monitoring Market Size.....	39
Figure 3.2: Livestock Monitoring Market Size US.....	39
Figure 3.3: Livestock Monitoring Market Share.....	40
Figure 3.4: Global Livestock Monitoring Market	40
Figure 3.5: Livestock & Meet Market	41
Figure 3.6: Middle East and Africa Livestock Monitoring Market.....	42
Figure 3.7: Egypt’s Livestock	42
Figure 3.8: SWOT Analysis	51
Figure 4.1: Competitive analysis	56
Figure 5.1: Context Diagram	58
Figure 5.2: Level 0 diagram.....	60
Figure 5.3: AI Workflow for Disease Detection and Report Generation	61
Figure 5.4: LLM-Based Multi-Agent Framework for Smart Query Answering .	63
Figure 5.5: CrewAI-Based Multi-Agent Feed Recommendation Workflow	64
Figure 6.1: DHT11 sensor	67
Figure 6.2: DHT11 with Arduino	68
Figure 6.3: LDR sensor	68
Figure 6.4: LDR with Arduino	69
Figure 6.5: MQ-137 sensor	69
Figure 6.6: MQ-137 with Arduino.....	70
Figure 6.7: Water Level sensor	70
Figure 6.8: Water Level with Arduino	71
Figure 6.9: PH sensor	71
Figure 6.10: PH with Arduino	72
Figure 6.11: VOC sensor	72
Figure 6.12: VOC with Arduino	73
Figure 6.13: Arduino Uno	74
Figure 6.14: ESP8266 NodeMCU.....	75

Figure 6.15: Cow Localization via UWB.....	82
Figure 6.16: Match UWB Tags to 3D Detections	83
Figure 6.17: Health Analysis via CNN	83
Figure 6.18: DWM1000	85
Figure 7.1: Classification Report – Mastitis.....	114
Figure 7.2: Confusion Matrix-Mastitis.....	114
Figure 7.3: Visual Predictions Grid – Mastitis.....	116
Figure 7.4: Classification Report – Lumpy Skin	119
Figure 7.5: Confusion Matrix – Lumpy Skin.....	119
Figure 7.6: Visual Predictions Grid – Lumpy Skin	120
Figure 7.7: Precision-Recall Curve – YOLOv8	122
Figure 7.8: Training mAP per Epoch Chart – YOLOv8.....	122
Figure 7.9: Sample Detection Results – YOLOv8	122
Figure 10.1: Farm Manager Persona.....	150
Figure 10.3: Veterinarian Persona	150
Figure 10.4: Farm Manager Journey Map	151
Figure 10.5: Veterinarian Journey Map.....	151
Figure 10.6: Account Registration and Authentication	152
Figure 10.7: Home Page & some features	153
Figure 10.8: Chat & Feed & Repots	154
Figure 10.9: Cow Health & Vet Home Page.....	155
Figure 10.10: Manager Registration	156
Figure 10.11: Manager Home Page & Cam & More.....	157
Figure 10.12: Env & Alerts	157
Figure 10.13: Manager Cow Health1	158
Figure 10.14: Manager Cow Health1	158
Figure 10.15: Feed	159
Figure 10.16: Manager Chat Bot.....	160
Figure 10.17: Vet Registration	161
Figure 10.18: Vet Home Page.....	161
Figure 10.19: Vet Cow Health1	162

Figure 10.20: Vet Cow Health2.....	162
Figure 10.21: Vet Chat Bot.....	163
Figure 10.22: Retrieve Latest Camera Frame Endpoint	182
Figure 10.23: Get Sick Cow Notifications Endpoint	183
Figure 10.24: Cow Status Endpoint	184
Figure 10.25: Report Browsing Per Cow Endpoint.....	185
Figure 10.26: Retrieve Full Report Content Endpoint	186
Figure 10.27: Homepage Overview Stats Endpoint	187
Figure 10.28: Sign up error: Password too short	188
Figure 10.29: Sign up error: Passwords do not match.....	189
Figure 10.30: User Table	190
Figure 10.31: Log in Endpoint.....	190
Figure 10.32: Farmer Chat Endpoint	192
Figure 10.33: Veterinarian Chat Endpoint	193
Figure 10.34: Feed Recommendation Endpoint	194
Figure 10.35: Feed Balance Check Endpoint.....	195
Figure 10.36: Best Feed Product Recommendation	197
Figure 10.37: Splash Screen and Icon.....	199
Figure 10.38: Splash Screen and Icon-2	199
Figure 10.39: Login and Registration Pages	200
Figure 10.40: Login and Registration Pages-2.....	201
Figure 10.41: Manger Home Page.....	202
Figure 10.42: Veterinarian Home Page.....	203
Figure 10.43: Feed Management Page.....	205
Figure 10.44: Feed Management Page-2	206
Figure 10.45: Feed Management Page.....	206
Figure 10.46: Chatbot Page	207
Figure 10.47: Cow List and History Pages	208
Figure 10.48: Cow List and History Pages-2.....	209
Figure 10.49: Cow List and History Pages-3.....	209

LIST OF TABLES

Table 7.1: Aggregate Metrics Summary – Mastitis Detection	113
Table 7.2: Aggregate Metrics Summary – Lumpy Skin Detection.....	118
Table 7.3: YOLOv8 Test Set Performance Metrics	121

LIST OF ABBREVIATIONS

Abbreviation	Referenced Terms
AI	Artificial Intelligence
IoT	Internet of Things
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
API	Application Programming Interface
UI/UX	User Interface / User Experience
UWB	Ultra-Wideband
DFD	Data Flow Diagram
CSV	Comma-Separated Values
BLE	Bluetooth Low Energy
SPI	Serial Peripheral Interface
YOLOv8	You Only Look Once, version 8 (Object Detection Model)
DHT11	Digital Humidity and Temperature Sensor
LDR	Light Dependent Resistor
MQ-137	Ammonia Gas Sensor
VOC	Volatile Organic Compounds
PH	Potential of Hydrogen (acidity/alkalinity measure)
ESP8266	Wi-Fi Module Microcontroller
DWM1001/1000	UWB Modules for Positioning (Qorvo)
ORM	Object-Relational Mapping
SQL	Structured Query Language
API	Application Programming Interface
RGB-D	Red Green Blue + Depth (Camera type)
KPI	Key Performance Indicator
JSON	JavaScript Object Notation
FAQ	Frequently Asked Questions
MEA	Middle East and Africa
PWA	Progressive Web Application
ISP	Internet Service Provider

PROJECT SUMMARY

This project consists of eleven chapters. It explains how to develop a smart livestock management system using modern technologies such as Artificial Intelligence (AI) and the Internet of Things (IoT). Below is a brief overview of what each chapter contains:

Chapter 1: This chapter introduces the **MooNet** project, its main goals, and outlines the motivation and structure of the report.

Chapter 2: This chapter presents the context of the project, including system overview, key features, and the problems it aims to solve.

Chapter 3: This chapter analyzes the livestock market and identifies MooNet's target users, competitors, and business potential.

Chapter 4: This chapter reviews related systems and explains how **MooNet** provides a unique AI-powered solution.

Chapter 5: This chapter explains the system design, including architecture, data flow, and AI-based decision-making.

Chapter 6: This chapter describes the hardware components used in **MooNet**, such as sensors and communication devices.

Chapter 7: This chapter discusses the implementation of the disease detection system using AI models like YOLOv8 and LLMs.

Chapter 8: This chapter presents the feed optimization system, powered by multi-agent technology for better nutrition plans.

Chapter 9: This chapter introduces the chatbot assistant that answers users' questions using LLM and external data.

Chapter 10: This chapter presents the mobile application, its interface, and how it connects users with the **MooNet** system.

Chapter 11: This chapter provides the final conclusions and suggests possible future improvements for **MooNet**.

Chapter 1

INTRODUCTION

1.1 Overview

The rapid advancement of Artificial Intelligence (AI) and the Internet of Things (IoT) has opened new horizons for transforming traditional livestock management. In Egypt, livestock farming still relies heavily on manual practices, resulting in challenges such as delayed disease detection, inefficient resource usage, and high operational costs.

MooNet introduces a smart, integrated system that leverages AI and IoT to modernize livestock care. By using real-time sensors, predictive models, and mobile technologies, **MooNet** enables early disease detection, automates water and feed management, and provides farmers with instant insights to make smarter decisions.

The system aims to improve animal health, reduce operational costs, and enhance productivity, offering a scalable and sustainable solution for modern farms. Through this project, we aim to contribute to Egypt's agricultural development by promoting the adoption of intelligent livestock technologies.

1.2 Objectives

This project aims to design and implement a smart livestock management system that leverages AI and IoT technologies to improve efficiency, sustainability, and profitability in Egyptian farms. The system is designed to address critical issues including disease detection delays, water and feed waste, and poor environmental control.

By integrating real-time sensor data, AI-driven analysis, and mobile-based user interfaces, MooNet empowers farmers with timely information and smart decision-making tools to enhance animal welfare and optimize operations.

Key Objectives:

- Implement AI models to predict and detect diseases at early stages.
- Integrate IoT sensors for monitoring temperature, gas levels, water quality, and lighting.
- Automate water and feed management to minimize waste and reduce cost.
- Deliver real-time alerts and actionable recommendations via a mobile application.
- Improve livestock welfare by optimizing barn environmental conditions.
- Support scalability for small to large farms across Egypt.
- Promote sustainable agriculture using smart, data-driven technologies.

1.3 Methodology

To achieve the objectives of MooNet, the development followed a structured methodology that combines technical design, field validation, and iterative feedback.

- **Problem Analysis and Requirement Gathering**

Initial research included field visits and interviews with veterinarians to understand farmers' real-life challenges. From these sessions, system requirements were derived, including needed sensor types (e.g., temperature, gas, water level, animal behavior).

- **System Design**

A modular architecture was developed that combines IoT hardware, AI models, and a mobile application. System diagrams like DFDs were created to map the data flow and ensure modularity and scalability.

- **Hardware Implementation**

Sensors like DHT11 (temperature), MQ-137 (ammonia), water level, and pH were integrated. Arduino microcontrollers were used for local data collection and transmission.

- **Software Development**

Custom AI models were trained to detect diseases such as Lumpy Skin Disease

and Mastitis. The mobile app was developed to deliver insights, notifications, and visualization of environmental and health data to farmers.

1.4 Document Organization

This document is structured into eleven chapters, each of which presents a different aspect of the **MooNet** system, from the conceptual foundation to technical implementation and evaluation. Below is a detailed outline of the content covered in each chapter:

Chapter 2: Project Context

This chapter establishes the background and justification for developing **MooNet**. It presents the challenges faced in traditional livestock farming and outlines how **MooNet** addresses these issues through AI and IoT technologies. The chapter includes a clear project description, system overview, impact on users and society, key innovative features, the business model canvas, market analysis (local and global), and a SWOT analysis that evaluates the project's feasibility and potential risks.

Chapter 3: Market Analysis

This chapter investigates the livestock monitoring industry at both global and local levels. It provides market size projections, growth trends, and key drivers such as technological adoption and animal welfare concerns. It identifies **MooNet's** target audience, competitive environment, and emerging trends. The chapter concludes by positioning **MooNet** as a competitive solution with strong market potential.

Chapter 4: Related Work

This chapter reviews existing livestock management systems, including tools such as Farm4Trade, Ranch Manager, Moovement, MERCK Animal Health, and AgriERP. It highlights their strengths and limitations, especially the lack of full integration between disease detection, feed management, and environmental monitoring. The chapter then shows how **MooNet** fills this gap with a unified, intelligent platform.

Chapter 5: System Design

This chapter introduces the technical architecture of MooNet. It includes diagrams such as Data Flow Diagrams (DFDs), context diagrams, and system schemas to explain how data moves through the system. It describes user interactions, how sensors and AI models are connected, and the logic behind disease prediction, environmental monitoring, and feed planning.

Chapter 6: Hardware Implementation

This chapter focuses on the physical components used in MooNet. It describes each hardware module (e.g., DHT11, MQ137, MQ138, LDR, pH sensor, and water level sensors) and how they are integrated using Arduino and ESP8266 boards. It also discusses how sensor data is collected, processed, and transmitted to the mobile app, and introduces advanced features like cow localization using UWB and image-based diagnosis with 3D cameras.

Chapter 7: Disease Detection System Implementation

Here, the implementation of computer vision and deep learning models is detailed. The chapter explains how YOLOv8 is used for cow detection and CNN models are applied to classify diseases like Lumpy Skin Disease and Mastitis. It covers dataset preparation, model training, deployment, evaluation metrics, and integration into the mobile application with real-time report generation.

Chapter 8: Feed Optimization System

This chapter discusses the use of CrewAI's multi-agent architecture and Large Language Models (LLMs) to develop smart, personalized feeding plans. It explains how cow data (weight, breed, health status) is used to generate three types of reports: a comprehensive nutrition guide, diet adjustment feedback, and commercial product recommendations. APIs are used to connect these insights to the app interface.

Chapter 9: Chatbot System

This chapter explains the architecture of **MooNet's** smart chatbot. Built on a multi-agent system, it supports RAG (Retrieval-Augmented Generation), Web search agents, and structured data querying using Pandas. The system intelligently routes user questions to the appropriate agent and provides real-time, context-aware responses. It supports both farm managers and veterinarians in navigating the app and interpreting data.

Chapter 10: Mobile Application

This chapter presents the design and implementation of the **MooNet** mobile app. It includes UI/UX sketches, journey maps, and personas representing the two main user types (manager and veterinarian). It explains how the app integrates all system features including cow monitoring, environmental control, feed planning, and chatbot access. The backend was developed using FastAPI, and the frontend using Flutter.

Chapter 11: Conclusion and Future Work

The final chapter summarizes the contributions and impact of **MooNet**. It reflects on how the system addresses the real-world challenges of livestock management. It also presents future enhancements such as automated waste management, biometric cow identification, offline app access, and expanding the system for broader agricultural use.

Chapter 2

PROJECT CONTEXT

2.1 Introduction

Modern agriculture is rapidly evolving to meet growing demands for food security, efficiency, and sustainability. Within this transformation, livestock management presents unique challenges related to animal health, environmental conditions, and operational costs. In response to these needs, **MooNet** was developed as a smart, integrated solution that leverages **AI** and the **IoT** to support livestock farmers in optimizing their operations.

The **MooNet** system offers a unified platform that addresses key issues in animal farming such as delayed disease detection, inefficient resource usage, environmental instability, and high feed costs. It combines real-time sensor data with intelligent decision-making tools and features (smart chatbot) that provides instant, contextual support to guide farmers through daily decisions, clarify sensor outputs, and recommend actions based on the farm's condition.

This chapter provides a comprehensive overview of the project's background and direction. It discusses the specific problems addressed by **MooNet**, its impact across user and market levels, the innovations that distinguish it from existing solutions, and the strategic business model supporting its deployment. A market analysis and SWOT evaluation are also included to demonstrate feasibility and growth potential.

2.1 System Overview

MooNet is an intelligent livestock management system that leverages AI and IoT technologies to improve traditional farming. It enables farmers to monitor animal health, control environmental conditions, manage feed and water, and receive real-time alerts and smart recommendations.

The system operates through physical sensors and local data processing; all managed via a mobile application. Its interface is simple and accessible, allowing farmers to make informed decisions quickly and improve overall farm productivity.

The problems that the system deals with:

MooNet was developed in response to several critical challenges observed in traditional livestock farming. These include:

- **Delayed Disease Detection**

Health conditions often go unnoticed until symptoms worsen, increasing the risk of spread, economic loss, and animal suffering.

- **Water Mismanagement**

Manual monitoring leads to water waste or shortage, affecting animal hydration and farm sustainability.

- **Unstable Environmental Conditions**

Poor air quality, gas accumulation, and temperature fluctuations negatively impact animal health and productivity.

- **Lack of Real-Time Insights**

Farmers often operate without up-to-date data, which limits their ability to make timely and effective decisions.

- **High Feed Costs and Nutritional Imbalance**

Improper feed planning results in excessive costs or nutritional deficiencies that hinder animal growth or milk production.

- **Limited Access to Veterinary Knowledge**

Farmers struggle to find and interpret accurate health and treatment information in real time.

- **Confusing Sensor Readings**

Even when equipped with IoT devices, many farmers are unable to understand raw sensor outputs without expert guidance.

- **Time-Consuming Manual Research**

Searching online for advice is often inefficient and unreliable, wasting time and leading to inconsistent results.

2.2 Impact

The **MooNet** system is designed to generate meaningful impact across multiple levels empowering individual farmers, improving market efficiency, supporting societal development, and promoting environmental sustainability.

User Impact

- **Early Detection of Health Issues**

The system identifies signs of diseases like ketosis and mastitis before they escalate, allowing for timely intervention and improved recovery outcomes.

- **Operational Cost Reduction**

By optimizing water usage, feed plans, and veterinary interventions, **MooNet** reduces unnecessary spending and improves overall farm efficiency.

- **Informed Decision-Making**

Real-time sensor data, intelligent alerts, and smart recommendations help farmers act quickly and with confidence.

- **Simplified Access to Expert Knowledge**

The integrated chatbot allows farmers to ask questions in natural language and receive clear, concise answers without relying on manuals or internet searches.

- **Increased Productivity**

Improved animal health and stable environmental conditions lead to better milk yields and meat quality.

- **User-Friendly Interface**

Designed to be accessible for non-technical users, enabling broader adoption among small and medium-scale farmers.

Market Impact

- **Improved Farm Efficiency**

MooNet promotes better resource utilization and automated management, enhancing productivity and competitiveness.

- **Support for Local Economies**

Feed recommendations include sourcing suggestions from local suppliers, strengthening rural supply chains.

- **Alignment with Digital Agriculture Trends**

Supports the transition toward smart farming and precision livestock management.

- **Potential for Investment and Scale**

The modular design and real-world impact of **MooNet** create opportunities for partnerships, funding, and wider adoption across regions.

Societal Impact

- **Higher Food Quality and Safety**

Healthier animals produce safer and more nutritious food products, contributing to public health.

- **Awareness and Skill Development**

Introduces modern farming practices, especially in underserved rural areas, encouraging youth participation in Agri-tech.

- **New Employment Opportunities**

Creates demand for roles in veterinary tech, sensor installation, data support, and AI application in agriculture.

Environmental Impact

- **Water Conservation**

Automatic water level monitoring helps avoid overuse and ensures efficient hydration for livestock.

- **Air Quality Management**

Sensors detect harmful levels of ammonia, allowing corrective action.

- **Sustainable Feeding Practices**

Data-driven feed planning minimizes waste & reducing environmental load.

- **Waste Management (in future features)**

Waste management systems will reduce pollution in the environment.

2.3 Novelty & Features

2.3.1 Novelty

Traditional systems specialize in one thing and do not provide a comprehensive solution, but our system provides an integrated solution that combines early disease detection, water management, environmental monitoring, and preparation, and improving nutrition.

All this in one system to save time and effort.

2.3.2 Our Features

Core Features

Early Detection of Animal Diseases

- Utilizes AI algorithms to monitor animal behavior and detect early signs of health deterioration.
- Generates instant alerts when symptoms of illness are detected, enabling farmers to act quickly and reduce losses.
- The system currently monitors conditions such as:
 - **Lumpy Skin Disease**
 - **Mastitis**
 - **Ketosis**

Automated Water Management

- Employ water-level sensors to monitor tanks and troughs continuously.
- Provides an automated refilling mechanism to ensure a steady water supply and prevent both drought and overflow.

Environmental Monitoring and Climate Control

- Uses IoT sensors to track barn conditions including air quality, humidity, temperature, gas levels, and lighting.
- AI processes this data and recommends or triggers environmental adjustments to improve animal comfort and productivity.

Real-Time Livestock Condition Monitoring

- Allows continuous observation of animal well-being and a barn environment.
- Smart sensors collect and analyze data on temperature, humidity, and animal movement.
- AI predicts potential health risks before they escalate, enabling preventive management.

Automated Data Collection and Analysis

- Sensor data is collected automatically, without manual input.
- Intelligent analytics provide farmers with clear insights into trends and patterns in livestock health and farm performance.
- Supports better planning for breeding, nutrition, and disease prevention.

Centralized Smart Application

- A mobile application serves as the main interface, connecting the user to all farm systems.
- Sends instant notifications about health or environmental anomalies.
- Designed to streamline farm management and boost efficiency through one unified dashboard.

Feed Analysis and Cost Optimization

- Farmers input current feed details through the app.
- The system analyzes nutritional composition and checks for deficiencies or excesses.
- Suggests adjustments and affordable feed alternatives to reduce costs without compromising health.
- For instance, it may recommend replacing expensive ingredients like soybeans with safe, cost-effective byproducts.

AI-Powered Smart Assistant (Chatbot)

- Provides real-time answers to farmer queries via the mobile app.
- Covers topics such as animal diseases, symptoms, treatment, environmental control, and general farm management.
- Delivers clear and simple explanations, improving decision speed and confidence.

Future Features

The following enhancements are planned for future versions of **MooNet**:

Waste Management

- Smart modules will help automate animal waste processing, reducing pollution and improving farm hygiene.

Livestock Identification via Camera

- Using image recognition, each animal will be uniquely identified through the mobile camera, allowing better tracking of health and performance.

Historical Health Records

- Each cow will have a centralized digital history, summarizing its medical and productivity records for long-term health analysis.

2.3.3 Related Product in The Market

Farm4Trade SUITE:

It provides health monitoring and feed improvement, but it does not use AI. Does not provide automatic data collection, monitor environment, or water management.

Ranch Manager:

It provides health monitoring and nutrition improvement, but it does not use AI. Does not monitor the environment or water management.

Movement:

It uses IoT to collect data and provide animal tracking, but it does not provide environmental monitoring or water management.

MERCK:

Specialized in animal health monitoring and genetic efficiency analysis. Does not provide water management or automated tasks.

AgriERP:

It provides health monitoring and feed improvement, but it does not provide automated tasks or predictions.

Our system is different we have a comprehensive solution that Provides all solutions in one place, from disease monitoring to water, environment, and feed management.

2.4 Business Model

MooNet follows a scalable and flexible business model built on the **Business Model Canvas** framework. It defines key components related to customer interaction, revenue generation, resource management, and value creation.

2.4.1 Customer Segments

- **Livestock Farmers** – Small, medium, and large-scale farmers aiming to improve farm operations and reduce losses.
- **Agribusiness Corporations** – Large farms or firms seeking data-driven and efficient livestock management tools.
- **Veterinary Services** – Clinics and veterinarians looking for smart systems to monitor animal health.
- **Government & NGOs** – Organizations focused on promoting sustainable agriculture and supporting rural development.

2.4.2 Value Propositions

1. Healthier Livestock & Higher Profits

- AI-powered health monitoring for early disease detection.
- Immediate alerts help reduce animal mortality and economic loss.
- Prevents the spread of major diseases like Lumpy Skin Disease, Mastitis, and Ketosis.

2. Lower Costs & Higher Efficiency

- Sensors monitor water levels and automate refill systems.
- Reduces wastage and ensures hydration, optimizing farm operations.

3. Better Livestock Productivity

- IoT-based environmental monitoring improves barn conditions.
- AI makes automatic adjustments to temperature, air quality, and lighting for optimal livestock performance.

4. Smarter Decisions with Minimal Effort

- Automated data collection and analysis offer actionable insights.
- Help optimize breeding, nutrition, and disease prevention.

5. Manage the Farm Remotely

- Mobile application connects to IoT system with real-time control.
- Sends health/environment alerts and improves decision-making.

6. Cost Reduction Without Compromising Health

- Feed ingredients are analyzed against nutritional requirements.
- Detect deficiencies or risks and recommend cheaper alternatives.

7. Future-Proof Farming

- Planned features include waste management automation, animal identification via camera, and historical health tracking for each cow.

2.4.3 Channels

- Direct sales through the mobile application
- Partnerships with agricultural equipment providers and veterinary services
- Presence at exhibitions and agricultural conferences
- Collaborations with government agencies and NGOs
- Online marketing through social media and digital platforms

2.4.4 Customer Relationships

- Dedicated customer support (chat, phone, email)
- Free trial periods and flexible subscription models
- Educational resources including guides, webinars, and tutorials
- Farmer communities and discussion forums
- Tailored solutions for various farm sizes and needs

2.4.5 Revenue Streams

- Monthly or annual subscription plans
- Premium features (e.g., advanced AI analytics, automated disease alerts)
- Funding partnerships with governments or NGOs for rural deployments

2.4.6 Key Resources

- AI and IoT technologies for real-time analytics and monitoring
- Development team (AI engineers, app developers)
- Partnerships with agribusinesses, vet clinics, and IoT suppliers
- Data infrastructure for storing farm reports and analytics

2.4.7 Key Activities

- Development and maintenance of the smart farm management platform
- Distribution of IoT devices for livestock tracking
- Farmer onboarding, education, and support
- Ongoing enhancement of AI models for predictive analytics
- Marketing, engagement, and customer success initiatives

2.4.8 Key Partnerships

- IoT hardware manufacturers (sensors, trackers)
- Agricultural cooperatives for farmer access and adoption
- Veterinary organizations for data integration and validation
- Cloud and tech providers (e.g., AWS, Google Cloud)
- Government agencies and NGOs supporting digital agriculture

2.4.9 Cost Structure

- Production and R&D costs for hardware (IoT devices)
- Software development and infrastructure maintenance
- Investment in AI model training and improvement
- Marketing, outreach, and farmer onboarding efforts
- Customer support and training resources

2.4.10 Our Business Model

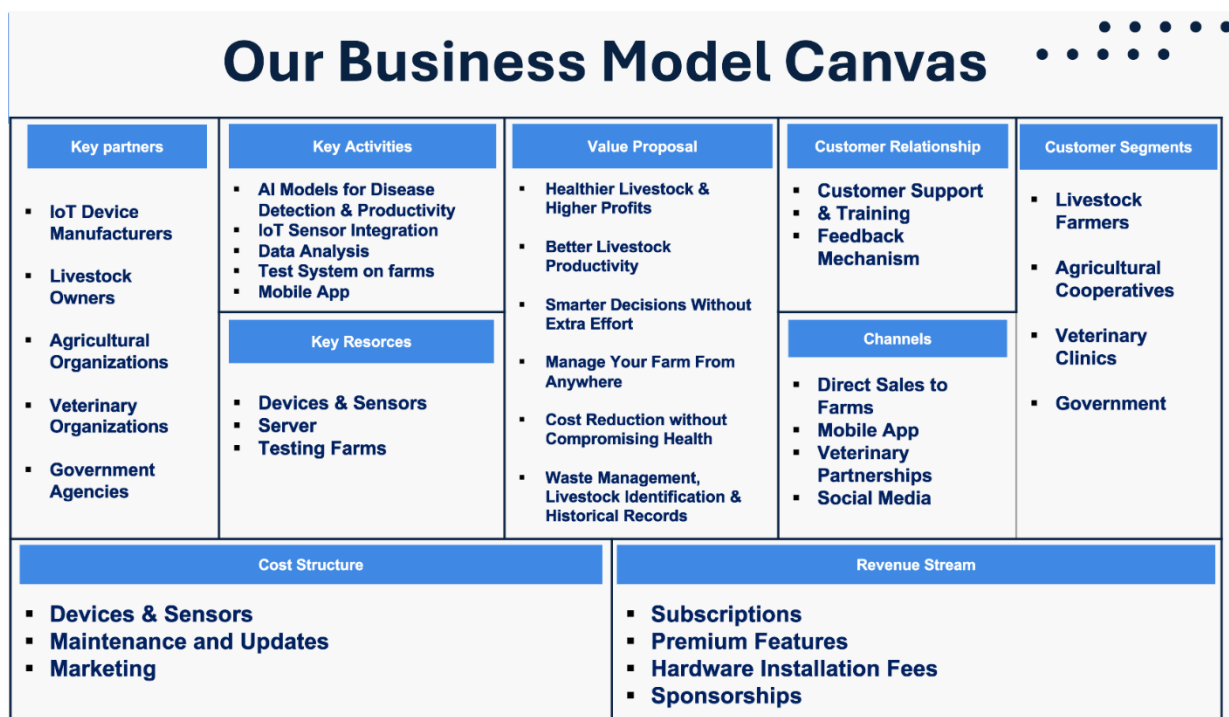


Figure 2.1: Business Model

Chapter 3

MARKET ANALYSIS

3.1 Introduction

This section outlines the global and local livestock monitoring market, focusing on current market size, growth trends, and regional relevance. It also identifies key drivers for adoption and the intended target audience for **MooNet**.

3.2 Industry Overview

The global livestock monitoring market is witnessing significant growth due to the increasing demand for smart farming solutions. Farmers are adopting AI and IoT technologies to boost productivity, improve animal welfare and reduce operating costs.

3.2.1 Global Livestock Monitoring Market

1. Livestock Monitoring Market Size & Growth

The global livestock monitoring market size is estimated at USD 7.42 billion in 2024 and is anticipated to reach around USD 31.08 billion by 2034, expanding at a CAGR of 15.40% from 2024 to 2034.

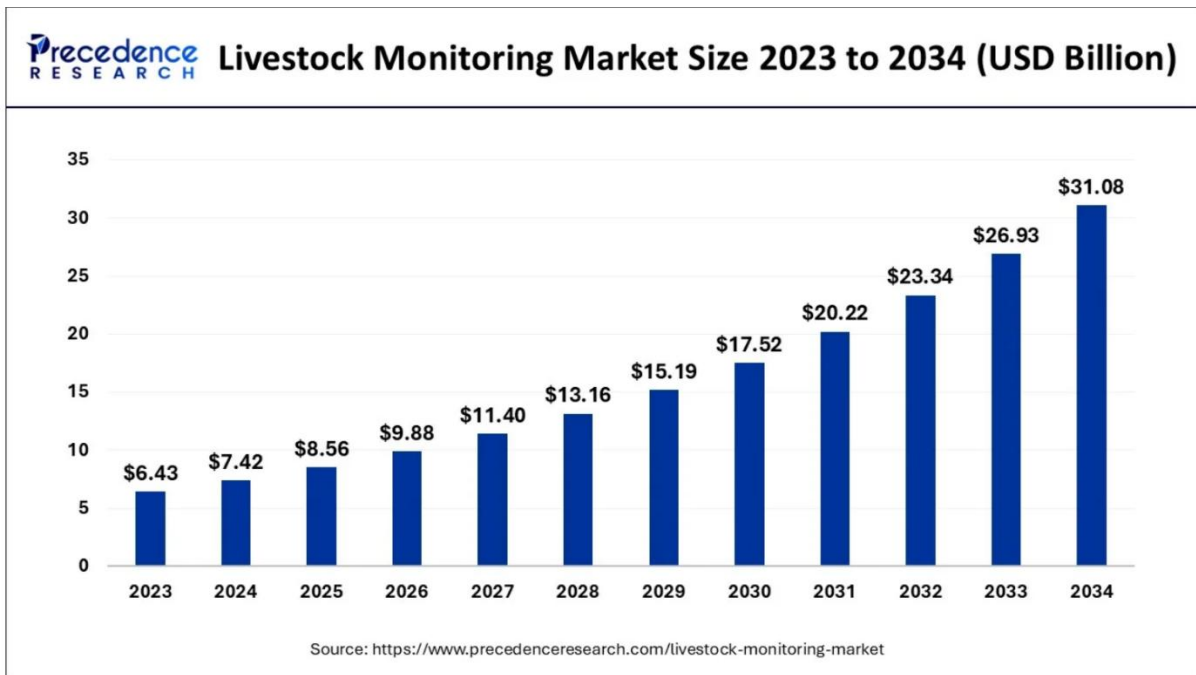


Figure 3.1: Livestock Monitoring Market Size

The U.S. livestock monitoring market size is evaluated at USD 1.71 billion in 2024 and is predicted to be worth around USD 7.34 billion by 2034, growing at a CAGR of 15.60% during the same period.

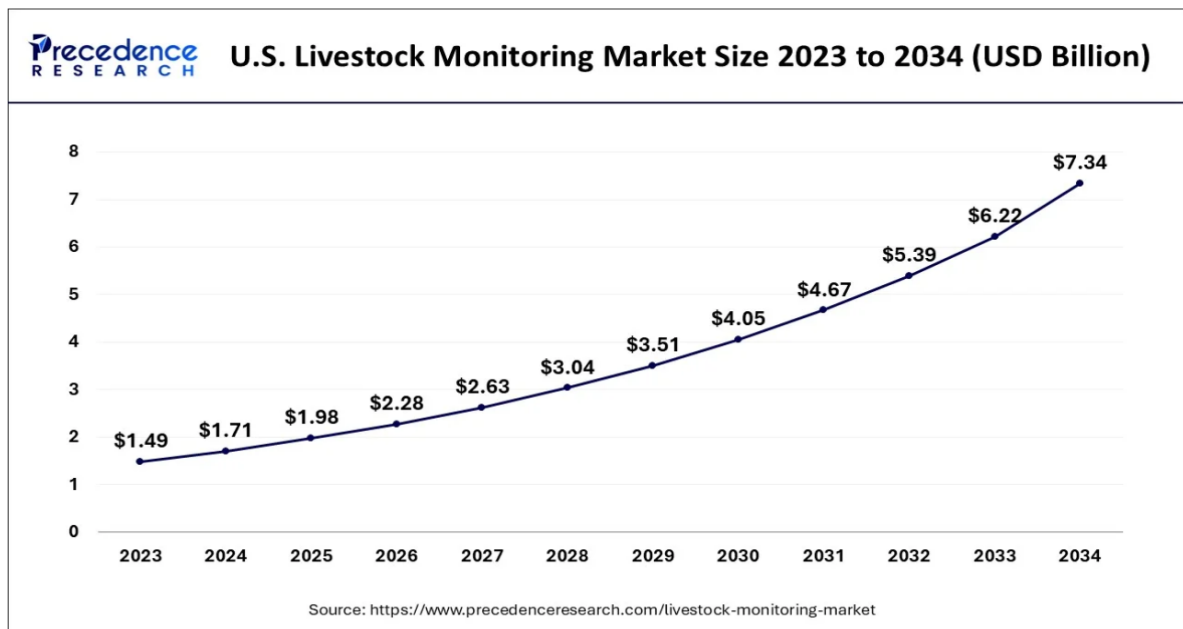


Figure 3.2: Livestock Monitoring Market Size US

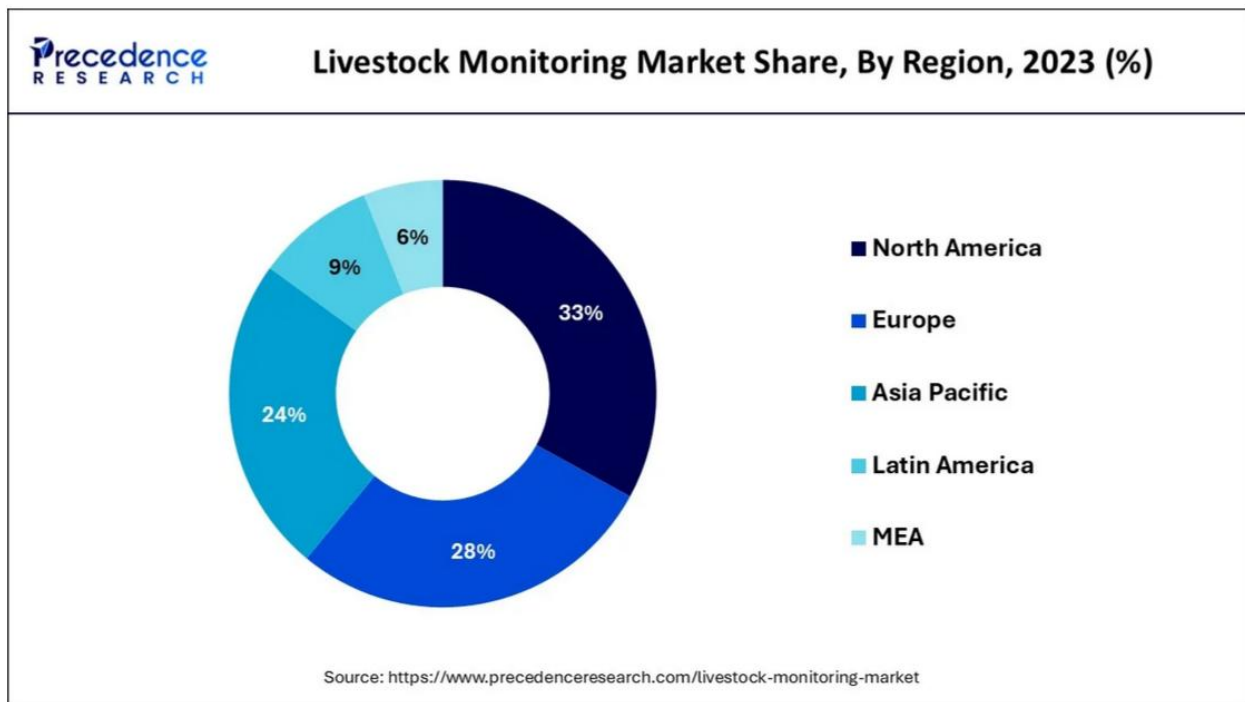


Figure 3.3: Livestock Monitoring Market Share

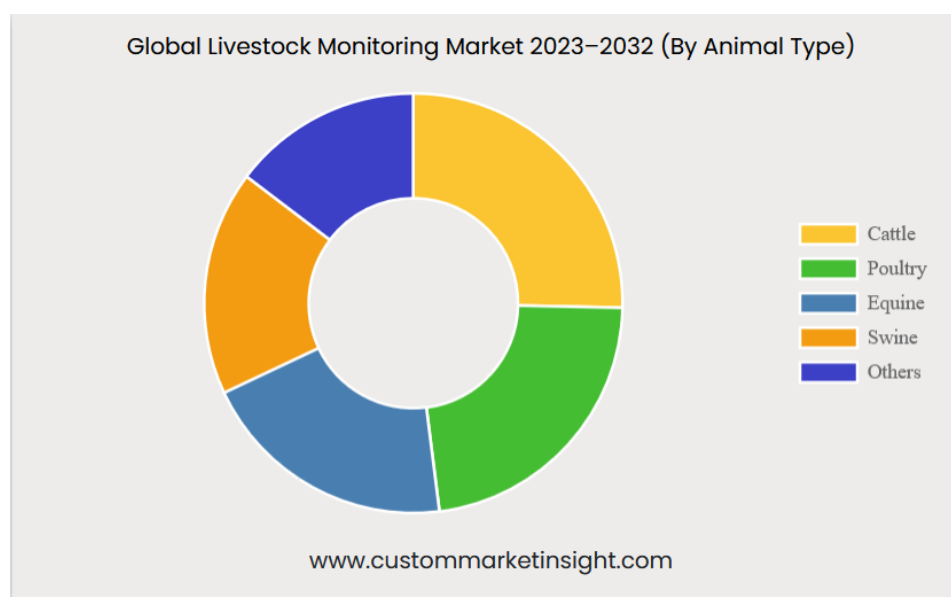


Figure 3.4: Global Livestock Monitoring Market

2. Global Livestock and Meat Market Size & Growth

The Livestock and Meat Market size is estimated at USD 514.37 billion in 2025, and is expected to reach USD 672.90 billion by 2030, at a CAGR of 5.52% during the forecast period (2025-2030)

Livestock And Meat Market

Market Size in USD Billion

CAGR 5.52%



Study Period	2019 - 2030
Base Year For Estimation	2024
Market Size (2025)	USD 514.37 Billion
Market Size (2030)	USD 672.90 Billion
CAGR (2025 - 2030)	5.52 %
Fastest Growing Market	Asia-Pacific
Largest Market	North America

Figure 3.5: Livestock & Meet Market

3.2.2 Local Livestock Market (Egypt & MEA Region)

Egypt is considered one of the most significant livestock producers in the region, playing a key role in the agricultural economy.

1. Middle East and Africa Livestock Monitoring Market

In 2022, the Middle East and Africa (MEA) region accounted for 4.4% of the global livestock monitoring market. The market is expected to grow at a CAGR of 16.1% from 2023 to 2030.

Middle East & Africa livestock monitoring market, 2018-2030 (US\$M)

Filter by country ▾

Download

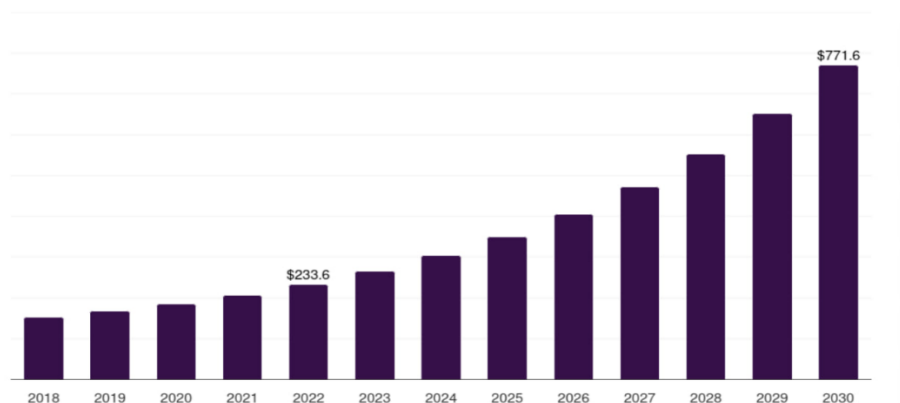


Figure 3.6: Middle East and Africa Livestock Monitoring Market

2. Egypt's Livestock Statistics

In 2023, the cattle stock in Egypt was estimated at approximately 3.1 million heads.

Leading livestock stocks in Egypt in 2023, by species

(in 1,000 heads)

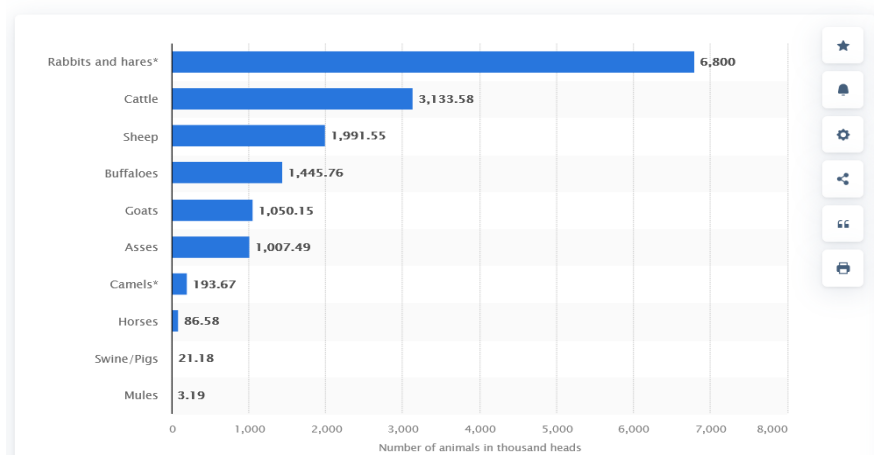


Figure 3.7: Egypt's Livestock

3.2.3 Market Trends Driving Growth

- **Animal Welfare**

There is increasing awareness of animal health, leading to demand for advanced monitoring systems to ensure livestock well-being.

- **Technological Development**

The use of smart sensors and real-time data analytics enhances decision-making and improves overall farm management.

- **Government Initiatives**

Government support for agricultural digitization is encouraging farmers to adopt modern technological solutions.

3.3 Target Audience

MooNet targets a broad range of stakeholders within the livestock and agricultural technology ecosystem:

1. Farmers
2. Livestock Managers
3. Veterinarians
4. Feed Companies
5. Meat and Dairy Production Companies
6. Veterinary Colleges
7. Agricultural Colleges
8. Investors and Stakeholders in Agriculture
9. Funding Organizations
10. Government Agencies concerned with agriculture and animal health
11. Agricultural Cooperatives

3.4 Competitive Analysis

Table 3.3 Competitive Analysis

“We don’t have competitors in Egypt”

Q		Disease Detection	Water Management	Environmental Monitoring	Real-time Monitoring	Automated Data Collection and Analysis	Feed Cost-Recommendations	Waste Management	Cattle Identification	Historical Records
	Farm4Trade SUITE	✓ Record	✗	✗	✗	✗	✓	✗	✓	✓
	Ranch Manager	✓ Record	✗	✗	✗	✓	✗	✗	✗	✓
	Moovement	✓ Record	✓	✗	✗	✓	✗	✗	✗	✓
	MERCK	✓ Record	✗	✗	✓	✓	✗	✗	✗	✗
	AgriERP	✓ Record	✗	✗	✓	✓	✓	✗	✗	✓
	MooNet	✓	✓	✓	✓	✓	✓	✓	✓	✓

3.5 SWOT Analysis

3.5.1 Strengths

- **All-in-One System:**

The system integrates all livestock management needs in one place, such as health monitoring, environmental control, feed optimization, and data analysis, helping users use one system instead of multiple tools.

- **Early detection of diseases:**

Our system allows users to detect diseases as early as possible to save the lives of cows. We focused on detecting Lumpy Skin Disease, Mastitis & Ketosis.

- **Monitoring the environment to improve productivity:**

IoT technologies enable comprehensive monitoring of environmental conditions in animal housing, such as air quality, ventilation, climate control and lighting optimization. By analyzing data collected from environmental systems and automatically adjusting settings to maintain optimal conditions, we can improve animal health and productivity.

- **Improved livestock welfare:**

Continuous monitoring and automatic adjustment of temperature, humidity and behavior improve living conditions, ensuring better livestock health.

- **Improved productivity:**

IoT sensors provide data on activity levels and environmental conditions, feed analysis and early disease detection for treatment, that allow farmers to optimize operations and maximize productivity.

- **Remote Monitoring:**

Farmers can monitor their livestock from anywhere via mobile apps, improving supervision and management efficiency.

- **Scalability:**

The integration of AI and IoT solutions can be easily scaled to manage larger herds without the need for major infrastructure changes.

- **Ease of Use:**

Simple interfaces and user-friendly applications ensure that farmers can operate the system without the need for advanced technical skills.

3.5.2 Weaknesses

- **High initial investment cost:**

Implementing AI and IoT technologies requires a large initial investment in hardware, software, and infrastructure, which can be challenging for small farmers.

- **Internet connectivity Dependency:**

The system is entirely dependent on stable internet access for real-time data transmission and cloud-based analytics, which can be a problem in rural areas with poor internet connectivity.

- **Technical complexity for farmers:**

Farmers with limited technical knowledge may find it difficult to operate, troubleshoot and fully benefit from the system.

- **Environmental Factors Affecting Sensors:**

Weather conditions such as high humidity, dust, or temperatures can affect the accuracy and longevity of IoT sensors.

- **System Maintenance and Development:**

The system needs technicians and developers regularly, to install the system, maintain it, troubleshoot, detect sensors, and develop the system.

3.5.3 Opportunities

- **Growing Demand for Smart Agriculture Solutions:**

In our current era, all fields are moving towards artificial intelligence and development, so there is a growing demand for artificial intelligence and IOT solutions to improve the efficiency of livestock management and productivity.

- **Increased focus on animal welfare:**

Consumers are becoming more aware of animal health, creating an opportunity to market our AI-powered system that improves animal welfare.

- **Expanding into global markets:**

With demand for meat and dairy products growing globally, smart systems can help farmers meet market demands efficiently.

- **Government Support:**

The government provides support and funding for smart farm projects, creating an opportunity to expand AI and IoT applications in livestock management.

- **Partnership with veterinarians:**

Collaboration with veterinary clinics helps improve the efficiency of the system, increase its reliance on early detection of diseases and improve animal welfare.

- **Partnership with educational institutions:**

Cooperation with colleges of agriculture and veterinary medicine to introduce young people to new technology will contribute to increasing its use in the long term.

- **Partnership with agricultural cooperatives:**

Partnerships with agricultural cooperatives will facilitate the dissemination of smart technology among farmers in rural areas.

3.5.4 Threats

- **Rapid evolution of technology:**

Technology is evolving so rapidly that an existing system may become unsuitable after a short period of time, requiring regular updates and additional investments to maintain efficiency.

- **Resistance to new changes:**

Traditional farmers accustomed to traditional methods may be reluctant to adopt new technology.

- **Cybersecurity Risks:**

Smart devices (Internet of Things) and cloud data storage may be vulnerable to hacking and cyber-attacks, which may lead to the leakage of sensitive data and damage trust.

- **Market Competition:**

There are companies that offer some similar solutions.

3.5.5 Solutions for Weaknesses

- **High initial investment cost**

Flexible Payment Plans: Offer installment or rental plans to make the system more accessible to small farms.

Mini versions: Offer affordable basic versions with basic features that can be upgraded later.

- **Internet connectivity dependency**

Partnerships with ISPs: Working with ISPs to provide customized solutions for rural areas.

- **Technical complexity for farmers**

Provide training: Create training courses designed for farmers in easy ways to understand how to use the system.

User-friendly interface: Simplify the system applications so that they contain clear icons and educational videos.

Provide technical support: Create call centers or field technical support that help farmers when they encounter any technical problems.

- **Environmental Factors Affecting Sensors**

Choose environmentally resistant devices: Use water, dust, and temperature resistant sensors.

Schedule periodic inspections: Ensure that devices operate efficiently in different weather conditions.

- **System Maintenance and Development**

Develop maintenance plans: Create a schedule for regular maintenance to avoid sudden breakdowns.

Train farmers on simple maintenance: Train them to do basic maintenance themselves.

3.5.6 Solutions for threats

- **Rapid Technological Advancements**

Continuous Innovation: Establish a dedicated research and development team to stay up to date with the latest technology trends and update features regularly.

User Feedback: Gathering user feedback on existing features, allowing our system to prioritize updates and new developments that align with user needs.

- **Resistance to new changes**

Technology Awareness: Organize awareness campaigns for farmers explaining the benefits of using the system to improve profits.

Real Success Stories: Share successful experiences of farmers who have benefited from technology.

- **Cybersecurity Risks**

Farmer and worker training: Educate users on how to protect data and avoid cyber fraud attempts.

Regular security updates: Ensure that hardware and software are constantly updated to close any security gaps.

Data backup: Maintain regular backups to restore data in case of a breach.

- **Market Competition**

Offering unique and integrated features.

Competitive pricing: Offering different plans to suit all categories.

3.6 Our SWOT

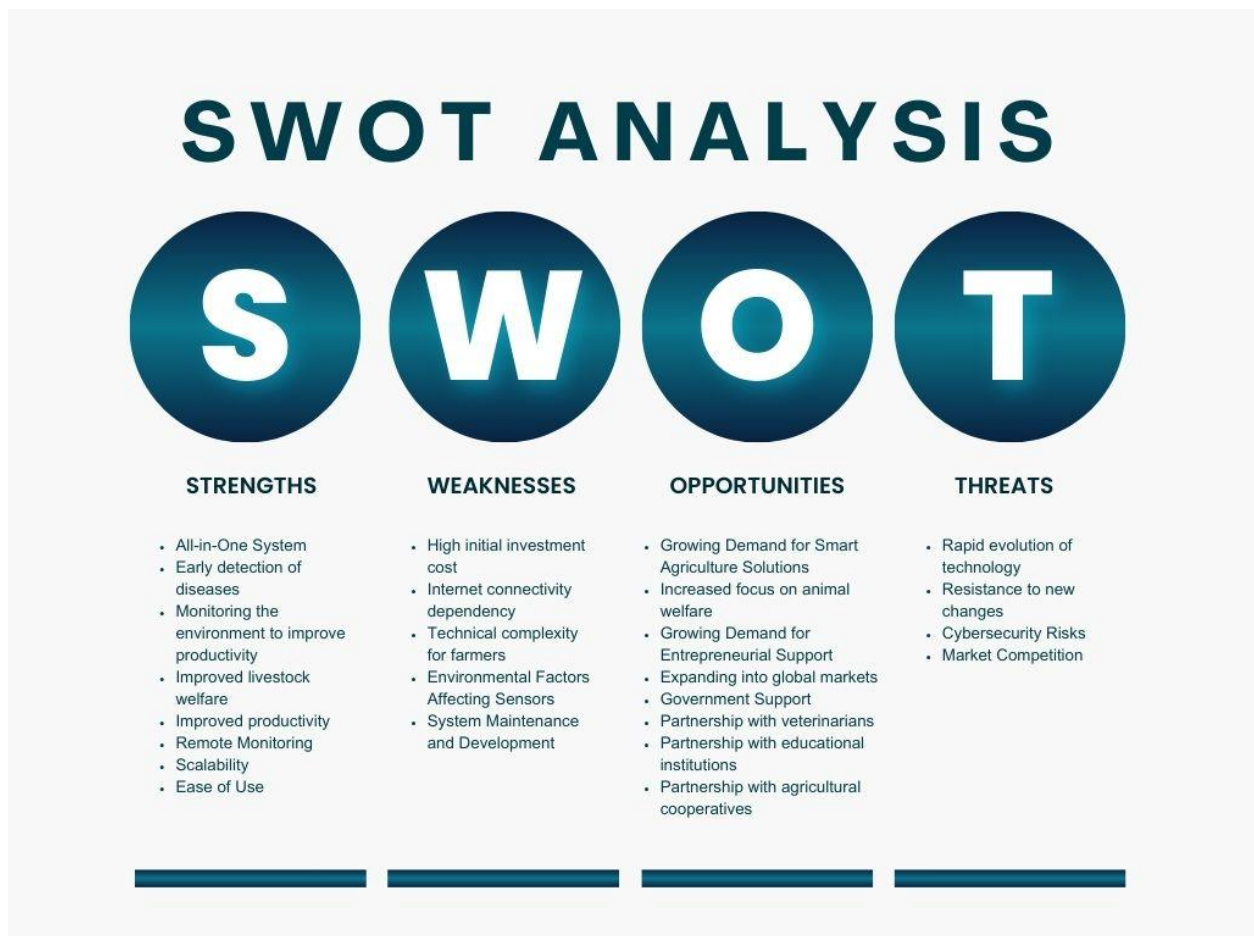


Figure 3.8: SWOT Analysis

Chapter 4

RELATED WORK

4.1 Introduction

In recent years, numerous solutions have emerged to modernize livestock management using digital technologies. Existing systems such as **Farm4Trade SUITE** and **Ranch Manager** focus on basic health tracking and nutrition but lack AI-driven predictions or environmental automation. **Moovement** leverages IoT for tracking but omits vital features like disease detection or water quality control. **MERCK** specializes in genetic analysis but doesn't integrate environmental or feed management. **AgriERP** supports health and feed improvement without predictive intelligence or automation.

These systems typically operate in silos, each targeting isolated problems. None offer a unified platform that integrates health monitoring, environment optimization, and smart decision support. Our proposed system, **MooNet**, stands out by providing a comprehensive AI-IoT hybrid that automates and optimizes every aspect of livestock care. This integrated approach ensures early disease detection, resource efficiency, and improved productivity. Thus, MooNet fills a critical gap left by current market solutions.

4.2 Existing Competitors

The current market offers several digital solutions aimed at supporting livestock farmers through health tracking, feed management, or location-based monitoring. While these tools offer useful features, most fall short in providing a fully integrated, AI-powered, and sensor-driven system. Below is a detailed overview of the most prominent competitors, outlining their core functionalities, advantages, and limitations.

- **Farm4Trade SUITE**

Description:

A mobile and web-based livestock management platform that enables farmers to track animal health, reproduction cycles, and productivity through manual records.

Advantages:

- Provides tools for tracking veterinary events and productivity metrics.
- Supports compliance, animal identification, and traceability.

Disadvantages:

- No artificial intelligence or predictive analytics.
- Entirely dependent on manual data entry.
- No automation or IoT sensor integration.
- Does not support real-time monitoring or alerts.
- Lacks modules for environmental or water management.

- **Ranch Manager**

Description:

A desktop-based software designed to manage livestock records including health, breeding, and inventory tracking.

Advantages:

- Strong record-keeping capabilities.
- Supports multiple species and generates printable reports.

Disadvantages:

- No support for real-time monitoring or AI-based predictions.
- Lacks mobile access and remote control.
- Does not integrate with IoT sensors or environmental controls.
- Manual data input only.
- Outdated interface not suitable for modern users.

- **Moovement**

Description:

An IoT solution that uses GPS ear tags to monitor livestock location, movement, and behavioral patterns on large grazing fields.

Advantages:

- Real-time geolocation tracking of cattle.
- Useful for pasture-based management and grazing optimization.

Disadvantages:

- Only covers location tracking—no health or disease prediction.
- Lacks environmental, feed, or water quality monitoring.
- No integration with AI or automation tools.
- Offers no mobile app for full system interaction.
- High implementation cost for small and mid-sized farms.

- **MERCK**

Description:

Provides electronic identification and performance tracking tools for genetic and productivity analysis in livestock operations.

Advantages:

- Advanced analytics for genetic efficiency and breeding.
- Supports large-scale industrial animal operations.

Disadvantages:

- Limited scope focuses only on genetic and health data.
- No integration of environmental or feed management systems.
- Not AI-driven or predictive in nature.
- High complexity and cost, unsuitable for smaller farms.
- No mobile or unified dashboard for everyday farm operations.

- **AgriERP**

Description:

An enterprise resource planning (ERP) platform that supports farm operations including crops, livestock, finance, and inventory.

Advantages:

- Comprehensive data centralization for large-scale operations.
- Includes modules for feed tracking, cost analysis, and financial reporting.

Disadvantages:

- Complex user interface; requires training and technical support.
- No AI-based prediction or behavioral analysis.
- Not tailored for livestock health as a primary focus.
- Limited or no IoT sensor integration.
- Lacks smart automation or real-time alert systems.

4.3 Competitive Analysis

“ We don’t have competitors in Egypt “

Q		Disease Detection	Water Management	Environmental Monitoring	Real-time Monitoring	Automated Data Collection and Analysis	Feed Cost-Recommendations	Waste Management	Cattle Identification	Historical Records
	Farm4Trade SUITE	✓ Record	✗	✗	✗	✗	✓	✗	✓	✓
	Ranch Manager	✓ Record	✗	✗	✗	✓	✗	✗	✗	✓
	Moovement	✓ Record	✓	✗	✗	✓	✗	✗	✗	✓
	MERCK	✓ Record	✗	✗	✓	✓	✗	✗	✗	✗
	AgriERP	✓ Record	✗	✗	✓	✓	✓	✗	✗	✓
	MooNet	✓	✓	✓	✓	✓	✓	✓	✓	✓

Figure 4.1: Competitive analysis

Chapter 5

SYSTEM DESIGN

5.1 Introduction

This chapter presents the system design of the intelligent cow farm monitoring and control system. It includes various diagrams that illustrate the structure, behavior, and data flow within the system. Key design models such as the Data Flow Diagram (DFD) and System Architecture Diagram are provided to explain how data moves and how components interact. These diagrams help in visualizing the system's functionality from both the user and technical perspectives. The aim is to ensure a clear understanding of the system before implementation begins.

5.2 Data Flow Diagram

5.2.1 System Scenario

The proposed project aims to develop an intelligent system for monitoring and controlling cow farms using **Artificial Intelligence (AI)** and the **Internet of Things (IoT)**. The primary goal is to reduce the spread of diseases and maintain a healthy, controlled environment for the cows. The system relies on a mobile application that allows users to interact with and monitor farm operations in real time.

The **farm manager** uses the mobile application to monitor environmental parameters such as **temperature**, **gas concentrations**, and **lighting intensity**. Additionally, the manager can access detailed reports for each cow, including health records and status updates. The system also integrates a **chatbot** feature that enables the manager to inquire about any farm-related topic.

Similarly, the **veterinarian** can log into the application to monitor the **health status of cows**, detect infections, and access comprehensive reports for decision-making. The veterinarian can also use the chatbot for assistance and inquiries, just like the manager.

Within the farm, a network of **sensors** collects data on environmental conditions, which is then processed by a central **controller**. Based on this data, the controller sends **automated commands to actuators** to maintain optimal farm conditions.

The system uses two main **databases**: one for storing all cow-related health and environmental data, and another for **user account management and login authentication**. This integration of mobile technology, AI, and IoT creates a smart farming solution aimed at improving animal welfare and operational efficiency.

5.2.2 Context Diagram

Illustrates the system's interactions with external entities.

Process:

- Cow Farm Monitoring & Control System

External Entities:

- Farm Manager
- Veterinarian
- Sensors
- Actuators

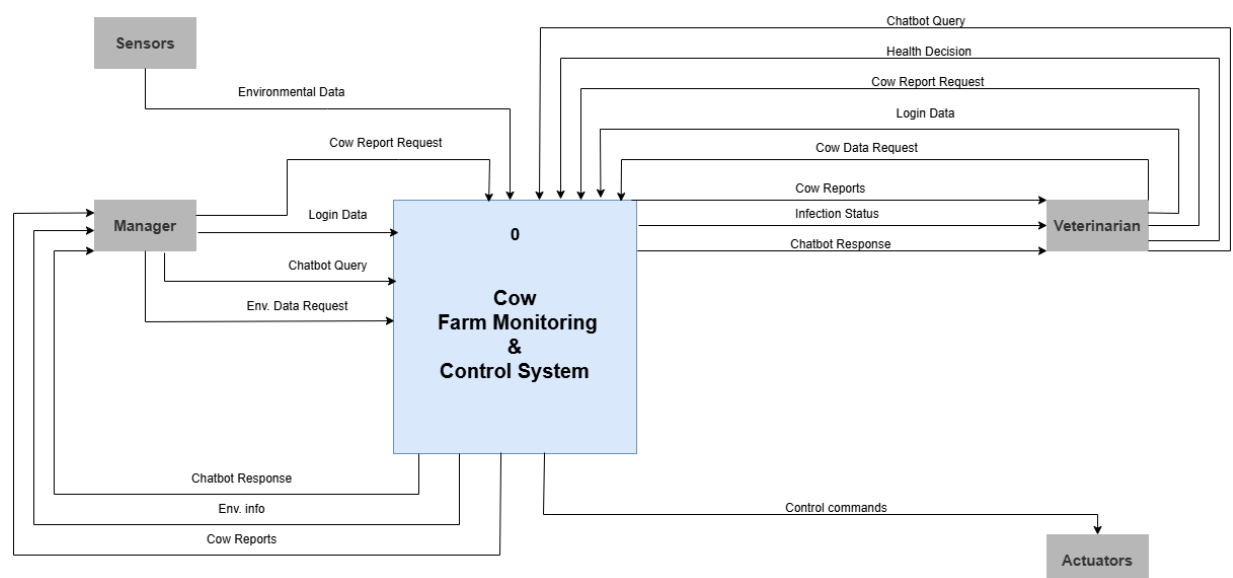


Figure 5.1: Context Diagram

5.2.3 Level 0 Diagram

Expands on the context diagram, detailing internal processes.

Processes:

- 1.0 User Authentication
- 2.0 Environmental Data Processing
- 3.0 Cow Health Monitoring
- 4.0 Report Management
- 5.0 Chatbot Interaction

External Entities:

- Farm Manager
- Veterinarian
- Sensors
- Actuators

Data Stores:

- D1: Cow Database
- D2: User Database

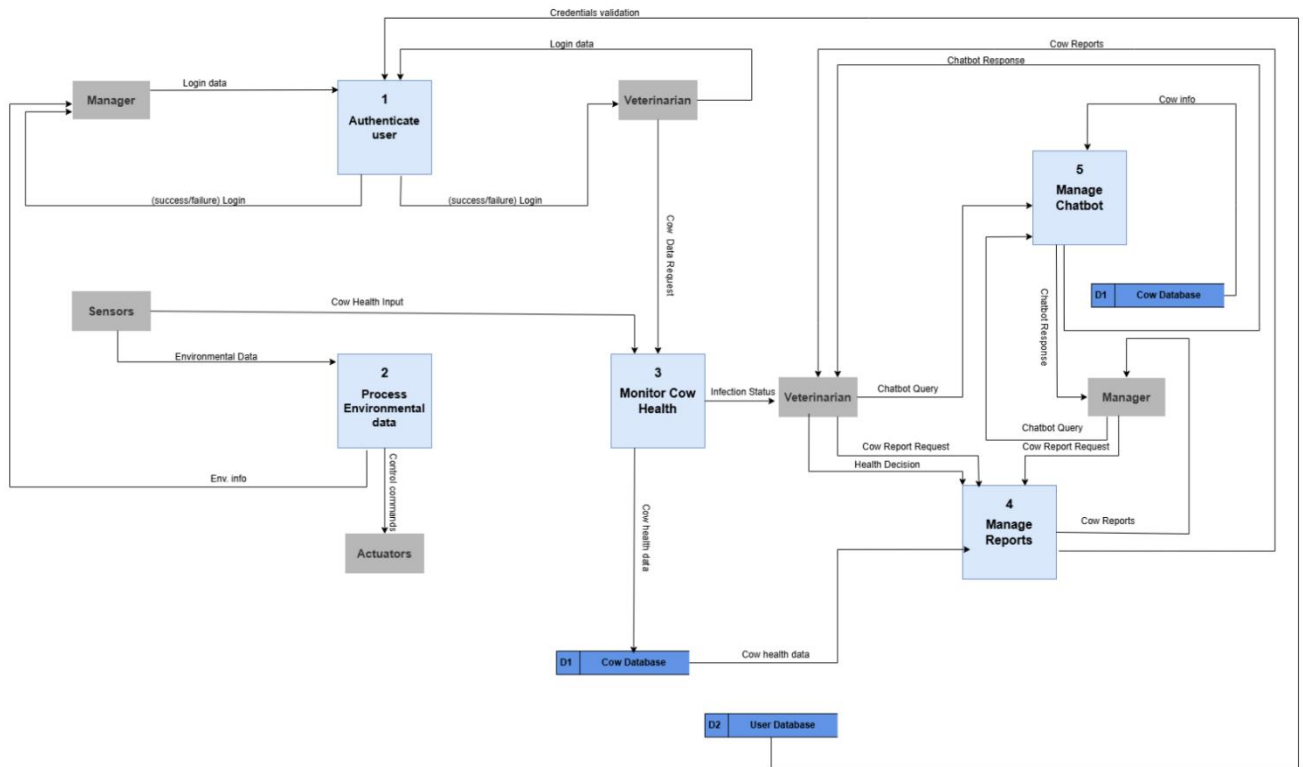


Figure 5.2: Level 0 diagram

5.3 AI System Design

5.3.1 AI Workflow for Disease Detection and Report Generation

This diagram illustrates the end-to-end AI workflow used in the cattle disease detection system. The left and right sections show the training pipelines for the **Mastitis** and **Lumpy Skin Disease** models, respectively. Each model is trained using a five-fold cross-validation strategy to ensure robustness and accuracy. The process includes data splitting into training and testing sets, model construction, and performance evaluation through quality checks. Once the model passes validation, training is considered complete.

In the center, the inference pipeline is shown. When a new image is captured by the farm camera, it is passed through both trained models. The detection is powered by a **YOLOv8-based object detection model**, which localizes cows in the image and extracts the relevant regions for disease classification. Each region is analyzed to determine whether signs of **mastitis** or **lumpy skin disease** are present.

If either model detects a disease, a prompt is sent to a **Large Language Model (LLM)** component that generates a detailed report in JSON format. This report is stored in the central **database** and accessed via the **mobile application**, enabling farm managers and veterinarians to make timely decisions based on AI-generated insights.

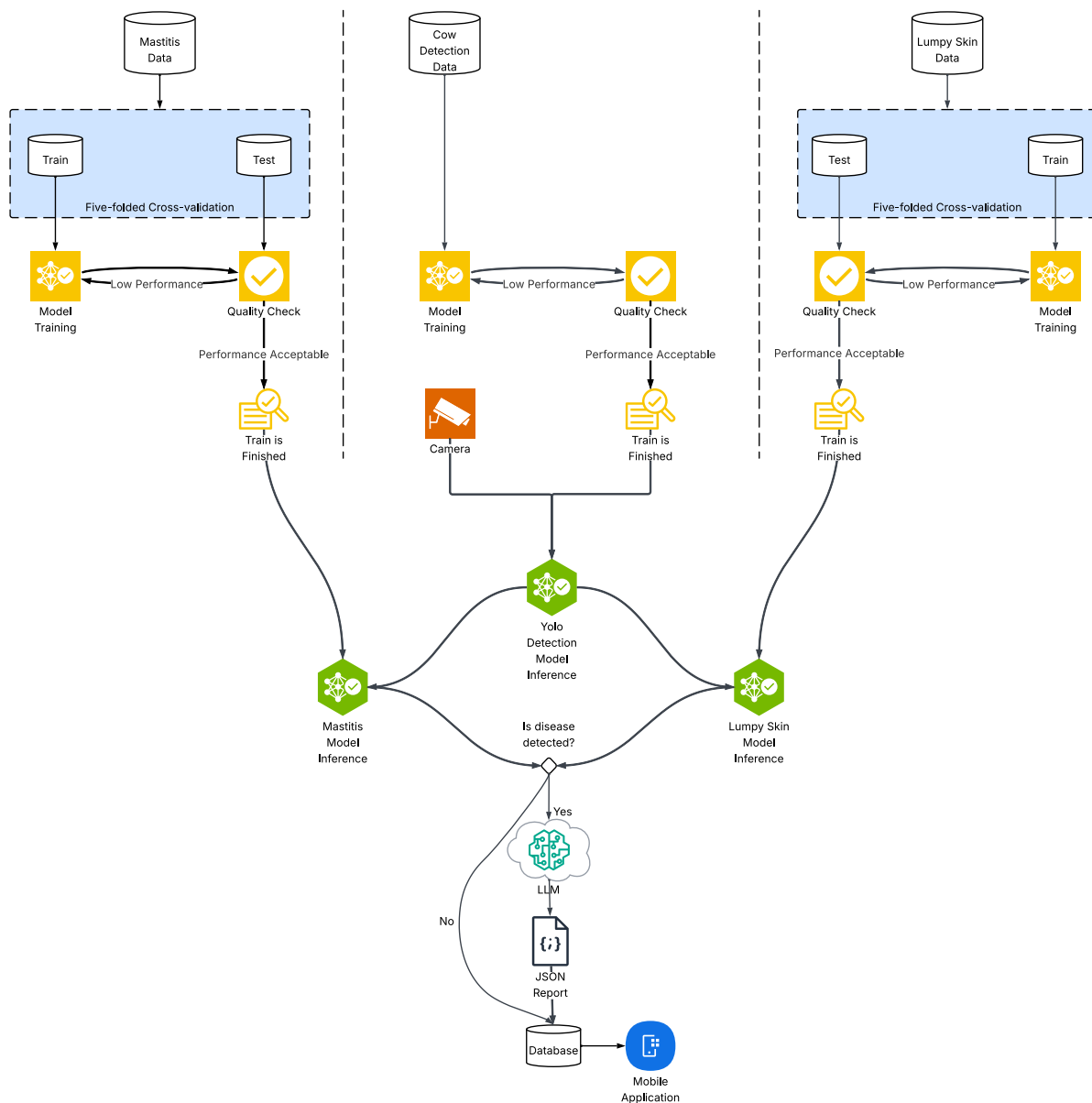


Figure 5.3: AI Workflow for Disease Detection and Report Generation

5.3.2 LLM-Based Multi-Agent Framework for Smart Query Answering

This diagram illustrates the architecture of the LLM-driven assistant used in the system to answer user queries through multiple reasoning tools. When a user submits a question through the mobile application (developed with Flutter), it is passed to a **Primary Agent** that delegates the task to one or more specialized tools depending on the nature of the question.

Three primary tools are integrated:

1. **RAG Tool:** Used for retrieving answers from **unstructured data** sources (e.g., PDF documents). The system uses an embedding model to convert the question into a vector and performs a vector similarity search on the **Vector Database** to retrieve the most relevant content. This content is passed to the LLM to generate a coherent answer.
2. **Pandas Agent Tool:** Used for answering data-driven questions from **structured sources** such as the `environment.csv` file, which stores the sensor readings (temperature, gas, light). The LLM generates the appropriate **Pandas query**, executes it, and interprets the results using the LLM to provide a natural language response.
3. **Internet Search Tool:** For general or external knowledge queries, the system uses **Tavily** to perform web search. The search results are then summarized by the LLM and returned to the user.

This flexible multi-agent framework allows the system to dynamically choose the best reasoning path depending on the question type and available data, offering a comprehensive smart farm assistant experience.

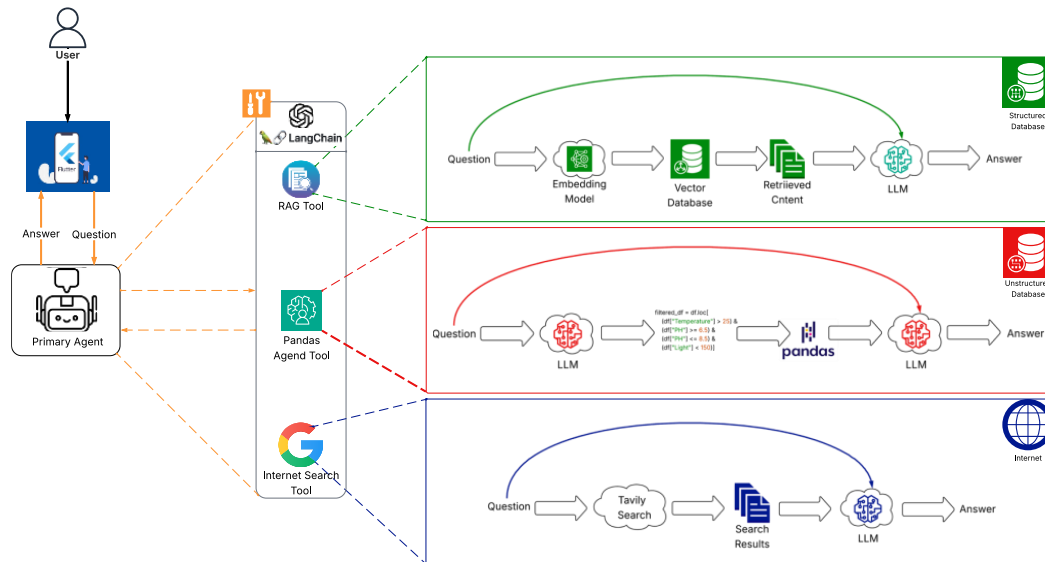


Figure 5.4: LLM-Based Multi-Agent Framework for Smart Query Answering

5.3.3 Feed Optimization System Using CrewAI

This diagram presents the architecture of the Feed Optimization feature in the system, which leverages a multi-agent collaboration framework powered by **CrewAI**. The feature exposes three main API endpoints:

- `/feed-recommendation`: Suggests suitable feed options based on cow attributes.
- `/feed-balance-check`: Verifies whether the current feed meets nutritional balance requirements.
- `/best-feed-recommendation`: Provides the optimal feed choice based on cow profile and market availability.

Each request triggers a workflow coordinated by the **CrewAI crew**, consisting of the following specialized agents:

1. **feed_agent**: Handles the initial request and collects relevant cow data (e.g., weight, breed, age).
2. **balance_check_agent**: Evaluates whether the cow's current feed is balanced or lacking nutrients.
3. **feed_keyword_generator_agent**: Generates contextual keywords to search for feed options, assisted by an **LLM**.

4. **search_engine_agent**: Performs a smart web search (e.g., using Tavily) to find matching commercial feed products.
5. **procurement_report_author_agent**: Compiles a full report with product suggestions, nutritional values, and justification.

The LLM supports the generation and interpretation tasks across agents to ensure the outputs are intelligent and user-friendly. The results are returned in real time to the mobile application user.

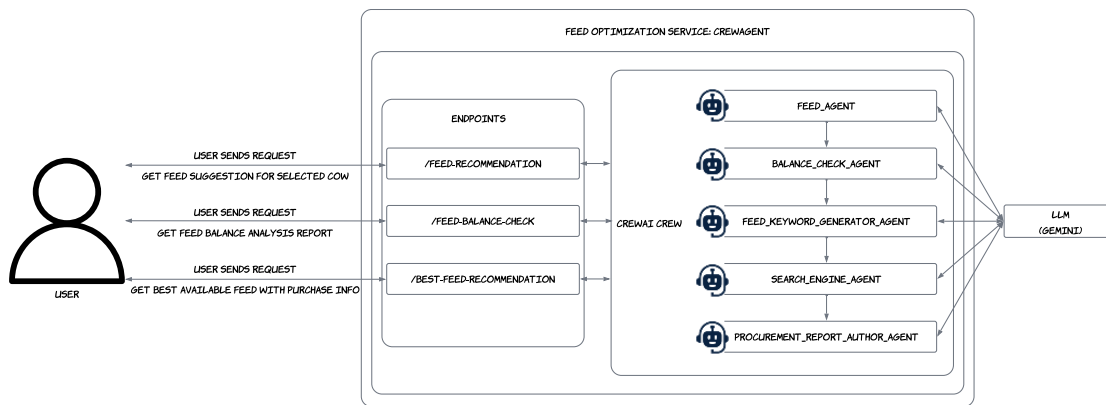


Figure 5.5: CrewAI-Based Multi-Agent Feed Recommendation Workflow

5.4 Conclusion

Chapter 5 outlines the complete design of a smart cow farm monitoring and control system integrating AI, IoT, and mobile technologies. It defines data flow and system architecture using DFDs and diagrams to clarify operations. The chapter presents AI-based disease detection using YOLOv8 and LLM-generated reports. It also introduces intelligent multi-agent systems for query answering and feed optimization. Overall, it provides a comprehensive blueprint for implementing an efficient, automated, and intelligent farm management solution.

Chapter 6

HARDWARE IMPLEMENTATION

6.1 Introduction

The hardware infrastructure is the backbone of the MooNet system, serving as the interface between the physical livestock environment and the digital decision-making layer. It consists of a carefully selected set of IoT sensors, processors, and communication modules that work together to monitor animal health, environmental conditions, and water quality in real time. These hardware components enable the collection of accurate, continuous, and actionable data that feeds into AI algorithms for early disease detection, smart resource management, and automated environmental control.

The use of IoT devices in smart livestock management is a transformative step toward precision farming. By embedding sensors directly into the farm environment, farmers can gain real-time insights into temperature, gas levels, water parameters, and animal behavior. This technology replaces guesswork with data, enabling proactive responses to health issues, optimizing feed and water usage, and ensuring the well-being of animals. In regions like Egypt, where traditional livestock practices dominate, integrating IoT in farming operations not only enhances productivity but also supports sustainability and food security.

6.2 Data Flow Description

- **Data Collection (Sensor Layer):**

The system begins with the sensor network deployed across the livestock environment. Environmental sensors (e.g., DHT11, MQ-137, LDR) continuously monitor temperature, air quality, and lighting. Water sensors (pH, water level) assess the quality and availability of drinking water. The VOC sensor is used for detecting acetone levels in milk to predict ketosis.

- **Data Processing (Processing Layer):**

All sensor data is transmitted via GPIO/analog/digital pins to the processing units. The Arduino handles real-time data collection and initial filtering, this layer also makes immediate automation decisions (e.g., turn on fans or water pump).

- **Data Transmission (Communication Layer):**

Once processed, the data is transferred via Wi-Fi to a local server system, and then made accessible to the user through a mobile application. The communication layer also handles remote command execution (e.g., changing environmental settings).

- **User Interaction (User Layer):**

Users interact with the system through a mobile app that provides a friendly dashboard. They receive real-time alerts and apply data-driven decisions regarding feeding, watering, and disease treatment.

6.3 Sensor Modules

6.3.1 DHT11 (Temperature sensor)



Figure 6.1: DHT11 sensor

Purpose: Measures air temperature and humidity to ensure a healthy environment for livestock.

Placement: Installed inside barns and enclosed areas where animals are housed

Working Principle: Uses a thermistor with a digital output signal.

Specifications:

- Temperature range: 0–50 °C
- Humidity range: 20–90% RH
- Accuracy: ± 2 °C, $\pm 5\%$ RH
- Digital output

Connection to System:

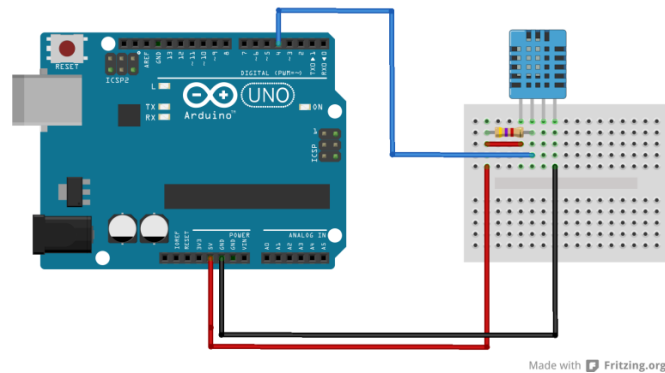


Figure 6.2: DHT11 with Arduino

6.3.2 LDR (Light Dependent Resistor)



Figure 6.3: LDR sensor

Purpose: Measures ambient light to regulate lighting conditions for livestock.

Placement: Fixed near barn ceilings or windows.

Working Principle: Resistance decreases with increasing light intensity, creating a measurable voltage drop.

Specifications:

- Analog output
- Operates at 3.3V or 5V

Connection to System:

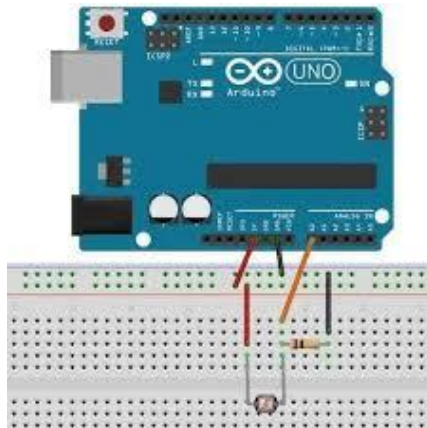


Figure 6.4: LDR with Arduino

6.3.3 MQ-137 (Ammonia (NH₃) Gas Sensor)



Figure 6.5: MQ-137 sensor

Purpose: The MQ-137 sensor is used to detect **ammonia gas (NH₃)** levels in the livestock environment. Elevated ammonia concentrations can cause respiratory problems and stress in animals, negatively affecting their health and productivity.

Placement: Installed in barns or enclosed livestock areas where waste buildup may cause the release of ammonia.

Working Principle: The MQ-137 operates using a **tin dioxide (SnO₂)** sensing layer that changes resistance in the presence of ammonia gas. This change in resistance is converted into a measurable analog voltage signal.

Specifications:

- **Detection Range:** 5 – 500 ppm (NH₃)
- **Sensitivity:** High sensitivity to ammonia
- **Output:** Analog (voltage)

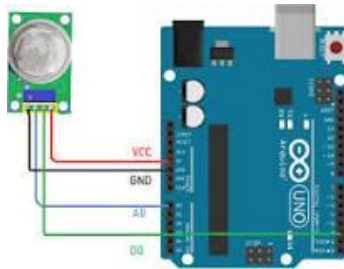
Connection to System:

Figure 6.6: MQ-137 with Arduino

6.3.4 Water Level Sensor



Figure 6.7: Water Level sensor

Purpose: Monitors water availability in animal drinking troughs.

Placement: Submerged inside the water container or trough.

Working Principle: Mechanical float switch changes state (on/off) when water reaches a certain level.

Specifications:

- Low power

Connection to System:

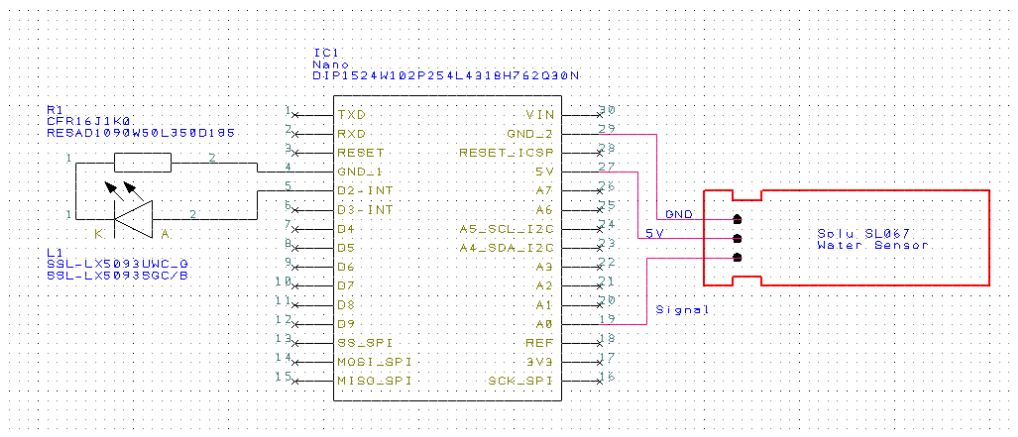


Figure 6.8: Water Level with Arduino

6.3.5 pH sensor



Figure 6.9: PH sensor

Purpose: Measures the acidity/alkalinity of the water to ensure it's safe for animals to drink.

Placement: Installed in water lines or drinking basins.

Working Principle: Uses a glass electrode to generate voltage relative to the pH of the solution.

Specifications:

- Range: 0–14 pH
- Accuracy: ± 0.1 pH
- Analog output

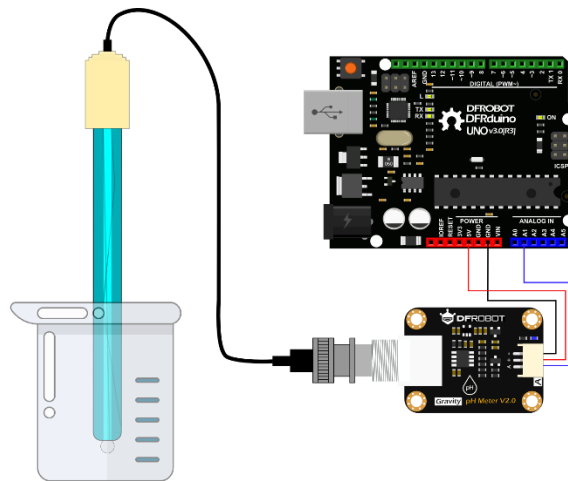
Connection to System:

Figure 6.10: PH with Arduino

6.3.6 VOC Sensor (Acetone Detection)



Figure 6.11: VOC sensor

Purpose: The VOC (Volatile Organic Compounds) sensor is used to detect **acetone vapors** in cow breath or milk—an important indicator of **ketosis**, a metabolic disorder in dairy cows. Early detection helps reduce health complications and economic losses.

Placement: Placed near the cow's **mouth or milking area** to detect acetone emitted through breath or milk. In field tests, it may be fixed within a milking station or handheld for spot checking.

Working Principle: The sensor detects VOCs (like acetone) using a **metal-oxide semiconductor** layer that changes resistance based on gas concentration. The sensor's output is analog and reflects the presence of target gases like acetone.

Specifications:

- **Detectable Gases:** Acetone, alcohol, benzene, methane, etc.
- **Output:** Analog voltage
- **Detection Range (Acetone):** ~5–50 ppm (depending on model)
- **Typical Load Resistance:** 10 k Ω

Connection to System:

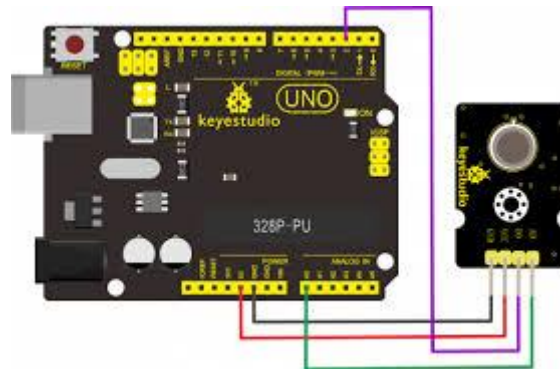


Figure 6.12: VOC with Arduino

6.4 Processing Unit

Arduino Uno



Figure 6.13: Arduino Uno

Purpose: Acts as the **main controller** responsible for reading data from all connected sensors including temperature, humidity, gas concentration, light intensity, pH, water level, and TDS.

Functionality:

- Performs data acquisition from analog and digital sensors
- Processes sensor signals (e.g., filtering, basic thresholds)
- Sends data via serial communication (UART) to the ESP8266 module

Justification for Use:

- Low power consumption
- Easy to program and widely supported
- Suitable for real-time sensor reading with minimal latency
- Supports analog and digital input directly (essential for various sensors)

6.5 Communication Module

ESP8266 NodeMCU



Figure 6.14: ESP8266 NodeMCU

Purpose: Used as a **Wi-Fi communication module** to handle networking tasks and API communication with the mobile application.

Functionality:

- Receives serial data from Arduino
- Formats and sends data as JSON through HTTP APIs
- Hosts or connects to cloud platforms or local servers
- Can receive commands from the app to control actuators

Justification for Use:

- Built-in Wi-Fi capability
- Perfect for IoT applications
- Compatible with HTTP/HTTPS protocols and REST APIs
- Works seamlessly with Arduino for data offloading

6.6 Arduino code

```
#include <DHT.h>

// --- Pin Definitions ---
const int ldrPin      = A0;
const int gasPin      = A1;
const int waterLevelPin = A2;
const int phSensorPin = A3;
const int vocSensorPin = A4;
const int dhtPin      = 2;
const int led1Pin     = 4;
const int led2Pin     = 9;
const int fanRelayPin = 10;
const int coolingRelayPin = 11;
const int refillRelayPin = 12;
const int basePumpRelayPin = 6;
const int acidPumpRelayPin = 7;

// --- Thresholds ---
const int ldrThreshold    = 300;
const int gasThreshold    = 200;
const float tempThreshold = 20;
const int acetoneThreshold = 300;
const int waterLevelThreshold = 600;
const float baseThreshold = 2.5;

// --- DHT Setup ---
#define DHTTYPE DHT11
DHT dht(dhtPin, DHTTYPE);

void setup() {
  pinMode(vocSensorPin, INPUT);
  pinMode(led1Pin, OUTPUT);
  pinMode(led2Pin, OUTPUT);
  pinMode(fanRelayPin, OUTPUT);
```



```
pinMode(coolingRelayPin, OUTPUT);
pinMode(refillRelayPin, OUTPUT);
pinMode(basePumpRelayPin, OUTPUT);
pinMode(acidPumpRelayPin, OUTPUT);

dht.begin();
Serial.begin(9600);
}

void loop() {
    checkLightAndControlLEDs();
    checkGasAndControlFan();
    checkTempAndControlCoolingPump();
    checkWaterLevelAndControlRefillPump();
    checkPHAndControlChemicalPumps();
    checkKetosis();

    // Send required data to ESP8266
    float temp = dht.readTemperature();
    int gas = analogRead(gasPin);
    int voc = analogRead(vocSensorPin);
    Serial.print("DATA,");
    Serial.print(temp);
    Serial.print(",");
    Serial.print(gas);
    Serial.print(",");
    Serial.println(voc);
    delay(2000);
}

// --- LDR ---
void checkLightAndControlLEDs() {
    int ldrValue = analogRead(ldrPin);
    Serial.print("LDR: ");
    Serial.println(ldrValue);

    bool isDark = ldrValue > ldrThreshold;
    digitalWrite(led1Pin, isDark ? HIGH : LOW);
    digitalWrite(led2Pin, isDark ? HIGH : LOW);
}
```

```
// --- MQ-137 ---
void checkGasAndControlFan() {
    int gasValue = analogRead(gasPin);
    Serial.print("Gas: ");
    Serial.println(gasValue);

    bool gasDetected = gasValue > gasThreshold;
    digitalWrite(fanRelayPin, gasDetected ? LOW : HIGH); // Active LOW
}

// --- DHT11 Temperature ---
void checkTempAndControlCoolingPump() {
    float temp = dht.readTemperature();
    if (isnan(temp)) {
        Serial.println("Failed to read DHT");
        return;
    }

    Serial.print("Temp: ");
    Serial.println(temp);

    bool overTemp = temp > tempThreshold;
    digitalWrite(coolingRelayPin, overTemp ? LOW : HIGH); // Active LOW
}

// --- Water Level Sensor ---
void checkWaterLevelAndControlRefillPump() {
    int waterLevel = analogRead(waterLevelPin);
    Serial.print("Water Level: ");
    Serial.println(waterLevel);

    bool needsRefill = waterLevel < waterLevelThreshold;
    digitalWrite(refillRelayPin, needsRefill ? LOW : HIGH); // Active LOW
}

// --- Acetone detection ---
void checkKetosis() {
```

```
int sensorValue = analogRead(vocSensorPin);

Serial.print("Acetone Level: ");
Serial.println(sensorValue);

if (sensorValue > acetonethreshold) {
    Serial.println("⚠ Ketosis detected: Acetone level is above threshold!");
} else {
    Serial.println("✅ No ketosis detected.");
}
}

// --- pH Sensor ---
void checkPHAndControlChemicalPumps() {
    int rawValue = analogRead(phSensorPin);
    float voltage = rawValue * (5.0 / 1023.0);
    float pH = 5 + ((2.5 - voltage) * 3.5);

    Serial.print("pH Voltage: ");
    Serial.print(voltage);
    Serial.print(" | pH: ");
    Serial.println(pH);

    // Control pumps based on pH
    if (pH > baseThreshold) {
        // Alkaline → turn on base pump
        digitalWrite(basePumpRelayPin, LOW);
        digitalWrite(acidPumpRelayPin, HIGH);
    } else if (pH < baseThreshold) {
        // Acidic → turn on acid pump
        digitalWrite(acidPumpRelayPin, LOW);
        Serial.print("RED");
        digitalWrite(basePumpRelayPin, HIGH);
    } else {
        // Neutral → turn off both
    }
}
```

```
digitalWrite(basePumpRelayPin, HIGH);  
digitalWrite(acidPumpRelayPin, HIGH);  
}  
}
```

6.7 ESP8266 NodeMCU code

```
#include <ESP8266WiFi.h>  
#include <ESP8266WebServer.h>  
  
const char* ssid = "";  
const char* password = "";  
ESP8266WebServer server(80);  
  
// Values from Arduino  
String temp = "0";  
String gas = "0";  
String voc = "0";  
  
void setup() {  
  Serial.begin(9600);  
  WiFi.begin(ssid, password);  
  Serial.println("Connecting to WiFi...");  
  
  while (WiFi.status() != WL_CONNECTED) {  
    delay(1000);  
    Serial.print(".");  
  }  
  Serial.println("");  
  Serial.println("WiFi connected.");  
  Serial.println(WiFi.localIP());  
  
  server.on("/temp", []() {  
    server.send(200, "application/json", "{\"temperature\": " + temp + "}");  
  });  
  
  server.on("/gas", []() {
```

```
server.send(200, "application/json", "{\"gas\": " + gas + "}");
});

server.on("/voc", []() {
    server.send(200, "application/json", "{\"voc\": " + voc + "}");
});

server.begin();
Serial.println("HTTP server started");
}

void loop() {
    server.handleClient();
    // Read from Arduino
    if (Serial.available()) {
        String line = Serial.readStringUntil('\n');
        if (line.startsWith("DATA")) {
            int firstComma = line.indexOf(',');
            int secondComma = line.indexOf(',', firstComma + 1);
            int thirdComma = line.indexOf(',', secondComma + 1);
            if (firstComma > 0 && secondComma > firstComma) {
                temp = line.substring(firstComma + 1, secondComma);
                gas = line.substring(secondComma + 1, thirdComma);
                voc = line.substring(thirdComma + 1);
            }
        }
    }
}
```

6.8 Detecting and tracking

UWB and 3D camera for detecting and tracking the specific cow with its ID, even in shared space

6.8.1 Objectives

To accurately detect and analyze the health status of each cow (mastitis + lumpy) diseases using 3D imaging combined with precise location and identity from UWB — in a scenario with multiple cows present at the same time

6.8.2 Scenario Overview

1. Cow Localization via UWB

- Each cow wears a UWB tag on its collar.
- The barn is equipped with UWB anchors
- These anchors receive signals from each tag and triangulate the 3D position of each cow with centimeter accuracy.

```
{  
  "cow_id": "COW_01",  
  "position": [X, Y, Z]  
}
```

Figure 6.15: Cow Localization via UWB

2. Scene Capture via 3D Camera

- A 3D camera captures RGB + Depth data at 30 FPS.
- Multiple cows appear in a single frame.
- Depth map helps in segmenting each cow by distance from the camera.

3. Match UWB Tags to 3D Detections

- The system detects cows in the image using YOLOv8.
- For each bounding box, it calculates the depth/3D center point.
- Then it matches the closest UWB tag location to that bounding box.

A code editor window showing a Python function named `match_cows`. The function takes `uwb_positions` and `detected_cows_3d` as arguments. It initializes an empty dictionary `matches`. It then iterates over `uwb_positions` items. For each `uwb_pos`, it iterates over `detected_cows_3d` items. It calculates the L2 norm distance between the cow's position and the UWB position. If this distance is less than the current `min_dist`, it updates `min_dist` and `closest_match`. Finally, it adds the `closest_match` to the `matches` dictionary for the given `cow_id` and returns the dictionary.

```
import numpy as np
def match_cows(uwb_positions, detected_cows_3d):
    matches = {}
    for cow_id, uwb_pos in uwb_positions.items():
        closest_match = None
        min_dist = float('inf')
        for i, cow in enumerate(detected_cows_3d):
            dist = np.linalg.norm(np.array(cow['position']) - np.array(uwb_pos))
            if dist < min_dist:
                min_dist = dist
                closest_match = i
        matches[cow_id] = detected_cows_3d[closest_match]
    return matches
```

Figure 6.16: Match UWB Tags to 3D Detections

4. Health Analysis via CNN

A JSON object representing health analysis data. It contains fields for cow_id, timestamp, position, mastitis_detected, skin_lesion, and image_path.

```
{
  "cow_id": "COW_05",
  "timestamp": "2025-06-07T13:15:00Z",
  "position": [4.2, 2.3, 1.6],
  "mastitis_detected": true,
  "skin_lesion": false,
  "image_path": "/images/COW_05_20250607_1315.png"
}
```

Figure 6.17: Health Analysis via CNN

6.8.3 System Architecture

1. UWB System (Ultra-Wide Band)

UWB Tags (per cow):

- Qorvo DW3000 attached to each cow's collar.

UWB Anchors:

- 6–8 units, mounted ~3–5 meters high (e.g., barn ceiling) for triangulation of cow position in 3D.

UWB Hub / Gateway:

- Arduino + UWB module (DWM1001-Dev) Time-syncs tags and collects position data over BLE or SPI

2. 3D Camera System

Depth Camera (RGB-D or stereo):

- Intel RealSense D455

specifications:

- Resolution: $\geq 1280 \times 720$
- Depth range: 0.3m to 10m
- Frame rate: ~30 FPS for real-time use

6.8.4 DWM1000 Arduino Connection

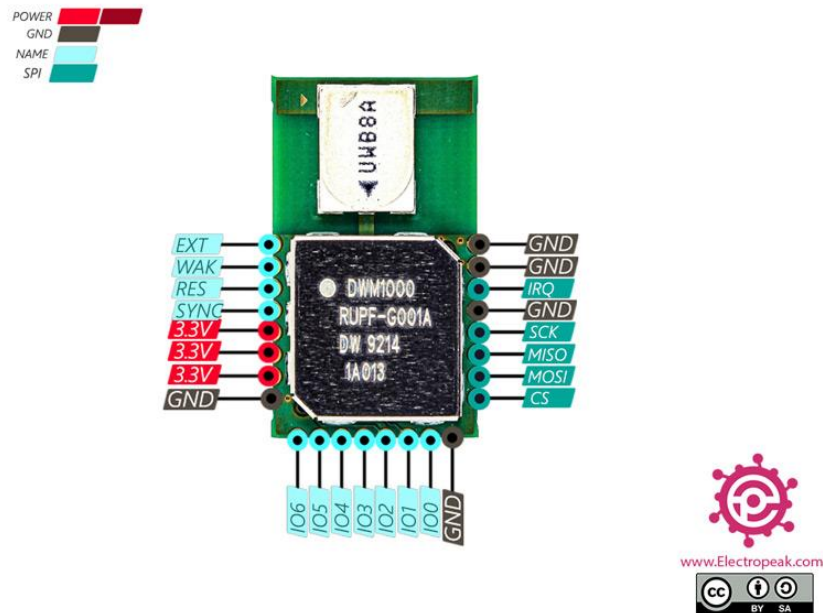


Figure 6.18: DWM1000

Chapter 7

DISEASE DETECTION SYSTEM IMPLEMENTATION

7.1 Introduction to AI System Integration and Implementation

In this section, we transition from the high-level system design to the practical implementation of our AI solutions. The following subsections detail how the conceptual framework described earlier has been translated into a fully functional system. Our AI system is designed to provide comprehensive support for cattle management through timely disease detection, intelligent query answering, and feed optimization.

7.1.1 Overview of the Implemented AI Solutions

Our implementation integrates multiple specialized AI modules, each tailored to a specific aspect of farm management. The key components include:

- 1. Disease Detection and Report Generation**

Two distinct deep learning models are deployed—one for Mastitis and one for Lumpy Skin Disease. Both models are trained using a five-fold cross-validation strategy to ensure robustness and accuracy. A YOLOv8-based object detection model is initially applied to identify and localize cows in the captured images. The detected regions are subsequently passed to the disease classification models, which analyze and predict the presence or absence of specific diseases. If a disease is detected, an integrated Large Language Model (LLM) generates a detailed diagnostic report in JSON format. This report is then stored in the central database and made accessible to farm managers and veterinarians through our mobile application.

2. LLM-Based Multi-Agent Framework for Smart Query Answering

For dynamic and intelligent responses to user queries, the system employs an LLM-driven assistant. Depending on the nature of the query, the assistant delegates the task to specialized tools:

- The **RAG Tool** retrieves answers from unstructured data sources.
- The **Pandas Agent** handles structured data inquiries.
- The **Internet Search Tool** supplements responses with external information.

This multi-agent framework provides an intelligent, real-time assistant capable of addressing both data-driven and general queries effectively.

3. Feed Optimization Using CrewAI

To improve livestock nutrition, a dedicated feed optimization module is integrated. This component leverages CrewAI's multi-agent collaboration framework, where:

- Specific agents (e.g., `feed_agent`, `balance_check_agent`) collect and analyze cow data.
- The system performs a smart web search to find commercially available feed options.
- A comprehensive report is generated recommending the optimal feed based on nutritional balance and market availability.

Together, these modules form a cohesive and comprehensive AI solution. Each component is interlinked to provide a seamless end-to-end process from image capture and disease detection to report generation, smart querying, and feed recommendation. This integrated approach ensures that farm managers receive timely, actionable insights that support both animal health and operational efficiency.

7.1.2 Data Ingestion and Preprocessing

This subsection outlines the data acquisition sources and preprocessing workflows used to prepare input datasets for training the AI models. Three separate datasets were used in this project: one for Mastitis disease classification, one for Lumpy Skin Disease classification, and one for cow detection using the YOLOv8 object detection model. Each dataset underwent tailored transformation processes to ensure compatibility with deep learning pipelines and to enhance model generalization.

7.1.2.1 Mastitis Disease Dataset

Data Source:

The Mastitis dataset was sourced from publicly available repositories on Kaggle and Roboflow. It consists of two classes: Infected Udder (1,698 images) and Healthy Udder (805 images). The dataset was organized into a directory structure with separate folders for each class.

Preprocessing Pipeline:

All images were resized to 224×224 pixels and normalized using standard ImageNet mean and standard deviation values. No data augmentation was applied to preserve the natural visual characteristics of the udders. The complete dataset was passed through a five-fold cross-validation routine to ensure robust performance evaluation without a fixed split.

Code Snippet: Data Transformation for Mastitis

```
input_size = 224

train_transforms = transforms.Compose([
    transforms.Resize((input_size, input_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])

test_transforms = transforms.Compose([
    transforms.Resize((input_size, input_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])
```

Explanation:

These transformations ensure that all images conform to the input size and statistical expectations of the EfficientNet-B0 architecture. The pipeline standardizes pixel intensity distributions and prepares the dataset for reliable and reproducible model training.

7.1.2.2 Lumpy Skin Disease Dataset

Data Source:

The Lumpy Skin Disease dataset was compiled from several Kaggle and Roboflow sources. It comprises 3,215 images of Normal Skin and 2,743 images of Lumpy Skin, organized into class-labeled folders.

Preprocessing Pipeline:

Images were resized to 224×224 pixels and normalized. A random horizontal flip augmentation was applied during training to increase data diversity. The dataset was processed using a five-fold cross-validation approach.

Code Snippet: Data Transformation for Lumpy Skin

```
input_size = 224

train_transforms = transforms.Compose([
    transforms.Resize((input_size, input_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])

test_transforms = transforms.Compose([
    transforms.Resize((input_size, input_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])
```

Explanation:

This preprocessing setup ensures that all input images are resized uniformly and normalized according to the statistical parameters expected by the EfficientNet-B0 model. These consistent transformations improve training stability and make the results more reproducible.

7.1.2.3 Cow Detection Dataset (YOLO)

Data Source:

The cow detection dataset was assembled from the OpenCows2020 dataset provided by the University of Bristol. Each image is accompanied by a YOLO-style annotation file containing normalized bounding box coordinates in the format: [class_id, x_center, y_center, width, height].

Preprocessing and Organization:

Images and their corresponding label files were organized into train, validation, and test subdirectories under the images/ and labels/ folders. This structure is fully compatible with the YOLOv8 object detection framework.

Directory Structure:

```
data/
├── images/
│   ├── train/
│   ├── validation/
│   └── test/
└── labels/
    ├── train/
    ├── validation/
    └── test/
```

Dataset Summary:

- Training Set: 4,930 images and 4,930 labels
- Validation Set: 1,056 images and 1,056 labels
- Test Set: 1,057 images and 1,057 labels

This nearly one-to-one correspondence between image and label files confirms the dataset's integrity and readiness for model training.

Annotation Verification:

To verify the accuracy of the annotations before training, a custom visualization function was developed using Matplotlib. This function renders bounding boxes on the original images using the YOLO annotation format.

Code Snippet: Bounding Box Visualization

```
def visualize_bounding_boxes(image_path, bboxes):

    image = cv2.cvtColor(cv2.imread(image_path), cv2.COLOR_BGR2RGB)
    height, width = image.shape[:2]
    fig, ax = plt.subplots(1)
    ax.imshow(image)

    for bbox in bboxes:
        x_center, y_center, w, h = bbox
        x_center *= width
        y_center *= height
        w *= width
        h *= height
        x = x_center - w / 2
        y = y_center - h / 2
        rect = patches.Rectangle((x, y), w, h, linewidth=2, edgecolor='g', facecolor='none')
        ax.add_patch(rect)

    plt.axis('off')
    plt.show()
```

Explanation:

This function allows for manual validation of annotation accuracy by visualizing bounding boxes on top of each image. It converts normalized YOLO coordinates into pixel values and overlays them, ensuring the quality of labeled data prior to model training.

7.2 Model Architectures and Training Pipelines

This section presents the deep learning models integrated into the cattle health monitoring system, alongside their respective training strategies. The system consists of three key AI components:

- **Mastitis Classification Module:** A binary classifier for identifying mastitis infection in udder images.
- **Lumpy Skin Disease Classification Module:** A skin lesion classifier for detecting Lumpy Skin Disease in cattle.
- **Cow Detection Module:** A real-time object detection model used to localize cattle in images and videos.

Each model was developed using state-of-the-art neural network architectures and optimized using best practices in training, validation, and evaluation. Classification tasks were handled with fine-tuned CNN-based models, while the detection component utilized YOLOv8 for its efficiency and real-time performance. To improve generalization, all classification models were trained using K-Fold Cross-Validation with integrated class imbalance handling and metric tracking.

7.2.1 Mastitis Detection Model

7.2.1.1 Model Objective

The mastitis detection model was designed to classify udder images into two categories:

- Class 0: Healthy Udder
- Class 1: Udder affected by Mastitis

This binary classification facilitates early diagnosis and timely treatment critical in dairy farm management.

7.2.1.2 Architecture Choice

Code Snippet:

```
# Load pretrained EfficientNet-B0 and adapt for binary classification
model = models.efficientnet_b0(pretrained=True)
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 2)
```

Explanation:

The system leverages EfficientNet-B0, a CNN architecture pretrained on ImageNet, chosen for its balance between accuracy and computational efficiency. The final classification layer is replaced with a fully connected layer outputting two classes corresponding to the binary task.

7.2.1.3 Training Strategy

Code Snippet:

```
# Basic training configuration
num_classes = 2
num_epochs = 10
batch_size = 32
splits = 5
learning_rate = 2e-5
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Explanation:

Defines the main training parameters, including number of classes, epochs, batch size, K-Fold splits, and device selection.

7.2.1.4 Dynamic Class Weighting

Code Snippet:

```
def calculate_class_weights(dataloader, num_classes, device):
    class_counts = Counter()
    for _, labels in dataloader:
        class_counts.update(labels.cpu().numpy())
```

```
weights = [ 1.0 / class_counts[ i ] if i in class_counts else 0.0 for i in range(num_classes) ]  
return torch.tensor(weights , dtype=torch.float , device=device)
```

Explanation:

Calculates inverse-frequency class weights dynamically per training fold to mitigate class imbalance and prevent bias toward the majority class.

7.2.1.5 K-Fold Training Loop

Code Snippet:

```
kfold = KFold(n_splits=splits , shuffle=True)  
results = {}  
  
for fold , (train_idx , val_idx) in enumerate(kfold.split(dataset)):  
    print(f'\nFold {fold + 1}/{splits}')  
  
    # Split dataset into train and validation subsets  
    train_subset = Subset(dataset , train_idx)  
    val_subset = Subset(dataset , val_idx)  
    train_loader = DataLoader(train_subset , batch_size=batch_size , shuffle=True)  
    val_loader = DataLoader(val_subset , batch_size=batch_size , shuffle=False)  
  
    # Initialize model and optimizer for each fold  
    model = models.efficientnet_b0(pretrained=True)  
    model.classifier[ 1 ] = nn.Linear(model.classifier[ 1 ].in_features , num_classes)  
    model.to(device)  
  
    optimizer = optim.Adam(model.parameters() , lr=0.001)  
    scheduler = optim.lr_scheduler.StepLR(optimizer , step_size=5 , gamma=0.5)  
  
    # Calculate class weights and define loss function  
    class_weights = calculate_class_weights(train_loader , num_classes , device)  
    criterion = nn.CrossEntropyLoss(weight=class_weights)
```

Explanation:

Implements a 5-fold cross-validation. For each fold, the dataset is split into training and validation, the model and optimizer are reinitialized, and class weights are recalculated to address imbalance.

7.2.1.6 Epoch-wise Training per Fold

Code Snippet:

```

for epoch in range(num_epochs):
    print(f'\nEpoch {epoch + 1}/{num_epochs}')
    model.train()

    total_loss, correct, total = 0, 0, 0
    y_true, y_pred = [], []

    for inputs, labels in tqdm(train_loader, desc=f"Training Fold {fold + 1}", leave=False):
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * inputs.size(0)
        _, preds = torch.max(outputs, 1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(preds.cpu().numpy())

    scheduler.step()

    # Calculate training metrics
    acc = correct / total
    prec = precision_score(y_true, y_pred, average='macro')
    rec = recall_score(y_true, y_pred, average='macro')
    f1 = f1_score(y_true, y_pred, average='macro')

    print(tabulate(
        [ [ "Loss", total_loss / total ], [ "Accuracy", acc ], [ "Precision", prec ], [ "Recall", rec ],
          [ "F1 Score", f1 ] ], headers=[ "Metric", "Train" ], tablefmt="fancy_grid"))

```

Explanation:

Each epoch includes training the model with backpropagation, accumulating losses, and tracking metrics such as accuracy, precision, recall, and F1 score. The learning rate scheduler adjusts the learning rate dynamically.

7.2.1.7 Validation Procedure

Validation is conducted at the end of each training epoch using a dedicated validation set to monitor the model's generalization performance and mitigate overfitting. The associated evaluation logic is implemented in the training pipeline, while detailed performance metrics and analysis are provided in **Section 5.2.4 Performance Evaluation**.

7.2.1.8 Cross-Fold Result Aggregation

Code Snippet:

```
print("\nK-FOLD CROSS VALIDATION RESULTS")
avg_acc = sum(results.values()) / splits
for fold, acc in results.items():
    print(f"Fold {fold+1}: {acc * 100:.2f}%")
print(f"\nAverage Accuracy: {avg_acc * 100:.2f}%")
```

Explanation:

The accuracy results from all folds are aggregated to provide a comprehensive estimate of the model's generalization performance on unseen data.

7.2.2 Lumpy Skin Disease Detection Model

7.2.2.1 Model Objective

This model classifies cattle skin images into two categories:

- **Class 0:** Normal Skin
- **Class 1:** Skin affected by Lumpy Skin Disease

Early identification of Lumpy Skin lesions helps mitigate outbreaks and supports timely veterinary intervention.

7.2.2.2 Architecture Choice

Code Snippet:

```
# Load pretrained EfficientNet-B0 and adapt for binary classification
model = models.efficientnet_b0(pretrained=True)
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 2)
```

Explanation:

EfficientNet-B0 was selected for its high performance-to-complexity ratio. The final classification head is replaced to support binary classification for Lumpy Skin detection.

7.2.2.3 Training Configuration

Code Snippet:

```
# Basic training configuration
num_classes = 2
num_epochs = 10
batch_size = 32
splits = 5
learning_rate = 2e-5
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Explanation:

These hyperparameters were tuned to balance learning stability and training efficiency. Learning rate and batch size were empirically selected for optimal performance.

7.2.2.4 Dynamic Class Weighting

Code Snippet:

```
def calculate_class_weights(dataloader, num_classes, device):
    class_counts = Counter()
    for _, labels in dataloader:
        class_counts.update(labels.cpu().numpy())
```

```
weights = [ 1.0 / class_counts[ i ] if i in class_counts else 0.0 for i in range(num_classes) ]
return torch.tensor(weights , dtype=torch.float , device=device)
```

Explanation:

This function dynamically computes inverse-frequency class weights from each fold to reduce the impact of class imbalance during training.

7.2.2.5 K-Fold Training and Evaluation

Code Snippet:

```
kfold = KFold(n_splits=splits , shuffle=True)
results = {}

for fold , (train_idx , val_idx) in enumerate(kfold.split(dataset)):
    print(f'\nFold {fold + 1}/{splits}')

    # Split dataset into train and validation subsets
    train_subset = Subset(dataset , train_idx)
    val_subset = Subset(dataset , val_idx)
    train_loader = DataLoader(train_subset , batch_size=batch_size , shuffle=True)
    val_loader = DataLoader(val_subset , batch_size=batch_size , shuffle=False)

    # Initialize model and optimizer for each fold
    model = models.efficientnet_b0(pretrained=True)
    model.classifier[ 1 ] = nn.Linear(model.classifier[ 1 ].in_features , num_classes)
    model.to(device)

    optimizer = optim.Adam(model.parameters() , lr=0.001)
    scheduler = optim.lr_scheduler.StepLR(optimizer , step_size=5 , gamma=0.5)

    # Calculate class weights and define loss function
    class_weights = calculate_class_weights(train_loader , num_classes , device)
    criterion = nn.CrossEntropyLoss(weight=class_weights)
```

Explanation:

To ensure robust evaluation, a 5-fold cross-validation strategy is employed. All folds are trained independently and evaluated to provide average performance metrics.

7.2.2.6 Epoch-wise Training Loop

Code Snippet:

```
for epoch in range(num_epochs):
    print(f'\nEpoch {epoch + 1}/{num_epochs}')
    model.train()

    total_loss, correct, total = 0, 0, 0
    y_true, y_pred = [], []

    for inputs, labels in tqdm(train_loader, desc=f"Training Fold {fold + 1}", leave=False):
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * inputs.size(0)
        _, preds = torch.max(outputs, 1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(preds.cpu().numpy())

    scheduler.step()

    # Calculate training metrics
    acc = correct / total
    prec = precision_score(y_true, y_pred, average='macro')
    rec = recall_score(y_true, y_pred, average='macro')
    f1 = f1_score(y_true, y_pred, average='macro')

    print(tabulate(
        [ [ "Loss", total_loss / total ], [ "Accuracy", acc ], [ "Precision", prec ], [ "Recall", rec ],
          [ "F1 Score", f1 ] ], headers=[ "Metric", "Train" ], tablefmt="fancy_grid"))
```


Explanation:

Each training epoch includes full-batch backpropagation with metric tracking (loss, accuracy, precision, recall, F1-score). The training progress is visualized using `tqdm`.

7.2.2.7 Validation Procedure

At the end of every training epoch, the model is evaluated on a separate validation set to assess its ability to generalize and to guard against overfitting. The validation process is integrated directly into the training pipeline, and a comprehensive breakdown of performance metrics and evaluation results is presented in Section 5.2.4 Performance Evaluation.

7.2.2.8 Cross-Fold Result Aggregation**Code Snippet:**

```
print("\nK-FOLD CROSS VALIDATION RESULTS")
avg_acc = sum(results.values()) / splits
for fold, acc in results.items():
    print(f'Fold {fold+1}: {acc * 100:.2f}%')
print(f'\nAverage Accuracy: {avg_acc * 100:.2f}%')
```

Explanation:

Final results are averaged over all folds to summarize the generalization capability of the model.

7.2.3 Cow Detection Using YOLOv8**7.2.3.1 Model Objective**

The objective of this component is to detect and localize cows in real-time from input images or live video streams. Detected regions are used as input for the disease classification models.

7.2.3.2 Dataset Structure and Configuration

Code Snippet:

```
# data.yaml
train: /images/train
val: /images/validation
test: /images/test

nc: 1
names: ['cow']
```

Explanation:

The YOLOv8 object detection model is configured using a data.yaml file that defines training, validation, and test sets, along with the number of classes (one class: “cow”).

7.2.3.3 YOLOv8 Training Routine

Code Snippet:

```
model = YOLO("yolov8n.pt")
model.train(
    data=data_yaml_path,
    epochs=50,
    imgsz=640,
    batch=16,
    project=os.path.join(base_dir, 'runs'),
    name="yolov8-cow-detection"
)
```

Explanation:

The training uses the yolov8n.pt (nano version) checkpoint, suitable for edge devices with constrained resources. The model is trained for 50 epochs at 640×640 resolution. Training results, metrics, and checkpoints are stored in the designated runs/ directory.

7.3 Real-time Inference and LLM Integration

This section outlines the deployment of the trained AI models for real-time inference and how their predictions are processed to support intelligent decision-making. Each model—Mastitis classification, Lumpy Skin Disease classification, and YOLOv8 cow detection—was encapsulated in a custom inference class to facilitate seamless integration. The predictions can be used individually or in combination with an LLM to generate detailed diagnostic reports.

7.3.1 Individual Model Inference

Each trained model is deployed using a custom inference class that loads the pre-trained weights, preprocesses input images, and outputs predictions with confidence scores.

7.3.1.1 Mastitis Classification Inference

This class loads the trained EfficientNet-B0 model and performs inference on udder images to predict the presence of Mastitis.

Code Snippet: MastitisDiseasesClassification

```
class MastitisDiseasesClassification:
    def __init__(self, model_path, class_names, input_size=(224, 224), device=None):
        self.device = device if device else torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.class_names = class_names
        self.input_size = input_size

        # Load EfficientNet-B0 and modify the final layer
        self.model = models.efficientnet_b0()
        self.model.classifier[1] = nn.Linear(
            in_features=self.model.classifier[1].in_features,
            out_features=len(class_names)
        )

        # Load pre-trained weights
        state_dict = torch.load(model_path)
        cleaned_state_dict = {k.replace("module.", ""): v for k, v in state_dict.items()}
```

```

self.model.load_state_dict(cleaned_state_dict)
self.model.to(self.device)
self.model.eval()

# Define image preprocessing steps
self.transform = transforms.Compose([
    transforms.Resize(self.input_size),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

def infer(self, image_path):
    """
    Perform classification inference on a mastitis udder image.
    """
    image = Image.open(image_path).convert('RGB')
    input_tensor = self.transform(image).unsqueeze(0).to(self.device)

    with torch.no_grad():
        outputs = self.model(input_tensor)
        probabilities = torch.softmax(outputs, dim=1)
        confidence, prediction = torch.max(probabilities, 1)

    return {
        "Predicted Label": self.class_names[prediction.item()],
        "Confidence": f'{confidence.item() * 100:.2f}%'
    }

```

Explanation:

This class encapsulates all necessary components to perform Mastitis detection, including model loading, image preprocessing, and real-time classification.

7.3.1.2 Lumpy Skin Disease Classification Inference

The inference pipeline for Lumpy Skin Disease mirrors that of Mastitis, using a fine-tuned EfficientNet-B0 architecture.

Code Snippet: LumpyDiseasesClassification

```
class LumpyDiseasesClassification:
    def __init__(self, model_path, class_names, input_size=(224, 224), device=None):
        self.device = device if device else torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.class_names = class_names
        self.input_size = input_size

        # Load EfficientNet-B0 and configure classifier head
        self.model = models.efficientnet_b0()
        self.model.classifier[1] = nn.Linear(
            in_features=self.model.classifier[1].in_features,
            out_features=len(class_names)
        )

        # Load model weights
        state_dict = torch.load(model_path)
        cleaned_state_dict = {k.replace("module.", ""): v for k, v in state_dict.items()}
        self.model.load_state_dict(cleaned_state_dict)
        self.model.to(self.device)
        self.model.eval()

        # Set up image transforms
        self.transform = transforms.Compose([
            transforms.Resize(self.input_size),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
        ])

    def infer(self, image_path):
        """
        Perform classification inference on a skin lesion image.
        """
        image = Image.open(image_path).convert('RGB')
        input_tensor = self.transform(image).unsqueeze(0).to(self.device)

        with torch.no_grad():
            outputs = self.model(input_tensor)
            probabilities = torch.softmax(outputs, dim=1)
            confidence, prediction = torch.max(probabilities, 1)
```

```

return {
    "Predicted Label": self.class_names[prediction.item()],
    "Confidence": f'{confidence.item() * 100:.2f}%'
}

```

Explanation:

The LumpyDiseasesClassification class processes skin images and predicts whether the cow has Lumpy Skin Disease, returning a labeled result and associated confidence.

7.3.1.3 YOLO Cow Detection Inference

The YOLOv8 model is employed for detecting cows in images, outputting bounding boxes and confidence scores for each detected instance.

Code Snippet: YOLO Cow Detection

```

# Load Trained YOLOv8 Model
base_dir = "/content/drive/MyDrive/MooNet Datasets/Cow_Detection_Dataset/Cow_Data"
model_path = os.path.join(base_dir, 'runs', 'yolov8-cow-detection', 'weights', 'best.pt')
model = YOLO(model_path)

# Create Output Directory for Results
output_dir = os.path.join(base_dir, "inference_results")
os.makedirs(output_dir, exist_ok=True)

def infer_on_image(image_path):
    """
    Perform object detection on a single image using the trained YOLOv8 model.
    """
    if not os.path.exists(image_path):
        return

    results = model(image_path)[0]
    image = cv2.cvtColor(cv2.imread(image_path), cv2.COLOR_BGR2RGB)

    if results.bboxes is None or len(results.bboxes) == 0:
        return

    # Annotate detections
    for box in results.bboxes:

```

```
x1, y1, x2, y2 = map(int, box.xyxy[0])
confidence = float(box.conf[0])
label = f"cow: {confidence:.2f}"
cv2.rectangle(image, (x1, y1), (x2, y2), (0, 255, 0), 2)
cv2.putText(image, label, (x1, y1 - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)

# Show and Save Result
plt.imshow(image)
plt.axis('off')
plt.title("YOLOv8 Inference Result")
plt.show()

output_path = os.path.join(output_dir, os.path.basename(image_path))
cv2.imwrite(output_path, cv2.cvtColor(image, cv2.COLOR_RGB2BGR))
```

Explanation:

The function runs inference using the YOLOv8 object detection model and visualizes the bounding boxes of detected cows, offering real-time feedback for livestock localization.

7.4 Centralized Inference and LLM Report Generation

7.4.1 Overview

The app.py script serves as the **central integration module** that coordinates inference across multiple models and generates structured health reports through a Large Language Model (LLM). This system enables automated medical insight generation for veterinary and farming applications, based on real-time image inputs.

7.4.2 System Flow Overview

1. Cow Detection Phase

A YOLOv8 object detection model is applied to confirm the presence of cows in input images.

2. Disease Classification Phase

Upon successful cow detection, two EfficientNet-B0-based classifiers are invoked:

- **Mastitis Classification**
- **Lumpy Skin Disease Classification**

3. LLM Report Generation Phase

Classifier outputs are passed to a Google Gemini model (via LangChain), which returns a detailed veterinary report in JSON format, including causes, symptoms, and recommendations.

7.4.3 Code Modules and Explanations

7.4.3.1 Mastitis Disease Classification Inference

Code Snippet:

```
def infer_abdominal_lumpy(image_path):
    model_path = r"disease_detection/Models/Model/Lumpy_Skin_Disease_Classification_Model.pth"
    lumpy_detector = LumpyDiseasesClassification(model_path=model_path, class_names = ['Lumpy Skin',
'Normal Skin'])
    result = lumpy_detector.infer(image_path)
    description = (
        "The system monitors key diseases in cows, including Lumpy Skin Disease, providing farmers with early "
        "disease detection alerts. "
        "The output includes a prediction dictionary indicating the presence (1) or absence (0) of the diseases, "
        "along with "
        "confidence scores for each prediction."
    )
)
```



```
return result, description
```

Explanation:

This function loads the trained **Mastitis classification model**, performs inference on a cow udder image, and returns both the predicted label and a detailed textual description for use by the LLM.

7.4.3.2 Lumpy Skin Disease Classification Inference

Code Snippet:

```
def infer_abdominal_mastitis(image_path):
    model_path = r"disease_detection\Models\Model\Mastitis_Disease_Classification_Model.pth"
    mastitis_detector = LumpyDiseasesClassification(model_path=model_path, class_names = ["Normal",
    "Infected"])
    result = mastitis_detector.infer(image_path)
    description = (
        "The system monitors key diseases in cows, including Mastitis Disease, providing farmers with early "
        "disease detection alerts. "
        "The output includes a prediction dictionary indicating the presence (1) or absence (0) of the diseases,
    along with "
        "confidence scores for each prediction."
    )

    return result, description
```

Explanation:

This function loads the pre-trained **Lumpy Skin Disease model**, applies it to the image, and returns the prediction result along with an accompanying explanation that summarizes its purpose and output.

7.4.3.3 LLM-Powered Report Generation

Code Snippet:

```
def generate_disease_report(classifier_description, classifier_outputs, cow_id, timestamp, disease_detected):  
    llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash-exp")  
  
    template = """  
    You are an expert veterinary report generator specializing in livestock disease detection through AI image  
classification and IoT-based monitoring.  
  
    Use the following input to generate a JSON-formatted health report:  
    - Classifier Function: {classifier_description}  
    - Classifier Results: {classifier_outputs}  
    - Cow ID: {cow_id}  
    - Timestamp: {timestamp}  
    - Disease Detected: {disease_detected}  
  
    Output Format:  
    {{  
        "disease_name": "{disease_detected}",  
        "description": "Brief explanation using classifier results and detection logic.",  
        "causes": ["Cause 1", "Cause 2"],  
        "symptoms": ["Symptom 1", "Symptom 2"],  
        "is_contagious": "Yes/No - explanation",  
        "severity": "Low/Moderate/High - explanation",  
        "farmer_advice": "Practical guidance for immediate action.",  
        "needs_vet_attention": "Yes/No - explanation"  
    }}  
  
    Make sure the response is strictly formatted as valid JSON.  
    """  
  
    prompt = PromptTemplate(  
        input_variables=[  
            "classifier_description", "classifier_outputs",  
            "cow_id", "timestamp", "disease_detected"  
        ],  
        template=template  
    )  
  
    chain = LLMChain(llm=llm, prompt=prompt)
```

```
report = chain.run(  
    classifier_description=classifier_description,  
    classifier_outputs=classifier_outputs,  
    cow_id=cow_id,  
    timestamp=timestamp,  
    disease_detected=disease_detected  
)  
  
cleaned = re.sub(r"```json```$", "", report.strip(), flags=re.MULTILINE).strip()  
parsed_report = json.loads(cleaned)  
  
return parsed_report
```

Explanation:

This function defines a prompt template tailored to veterinary reporting. It initializes a **Gemini LLM via LangChain**, formats the output into a structured JSON object, and returns the report for app display, storage, or alerts. This layer adds explainability and medical relevance to the raw classification outputs.

7.5 Performance Evaluation

This section presents the validation and test set performance metrics of the trained models: Mastitis Detection, Lumpy Skin Detection, and YOLOv8 Cow Detection. Metrics such as accuracy, precision, recall, F1-score, and confusion matrices are reported for thorough evaluation.

7.5.1 Mastitis Detection Results and Analysis

7.5.1.1 Evaluation Metrics on Test Set

Code Snippet:

```
model.eval()  
test_loss = 0  
correct_test = 0  
total_test = 0  
y_true_test, y_pred_test = [], []
```

```
with torch.no_grad():
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        test_loss += loss.item() * inputs.size(0)

    _, preds = torch.max(outputs, 1)
    correct_test += (preds == labels).sum().item()
    total_test += labels.size(0)

    y_true_test.extend(labels.cpu().numpy())
    y_pred_test.extend(preds.cpu().numpy())
```

Explanation:

This block sets the model to evaluation mode and disables gradient calculation. It calculates the total test loss and gathers predictions to later compute metrics.

7.5.1.2 Classification Metrics and Confusion Matrix

Code Snippet:

```
acc = correct_test / total_test
prec = precision_score(y_true_test, y_pred_test, average='macro')
rec = recall_score(y_true_test, y_pred_test, average='macro')
f1 = f1_score(y_true_test, y_pred_test, average='macro')

results[fold] = acc

print(tabulate([
    ["Loss", f"{test_loss / total_test:.4f}"],
    ["Accuracy", f"{acc:.4f}"],
    ["Precision", f"{prec:.4f}"],
    ["Recall", f"{rec:.4f}"],
    ["F1 Score", f"{f1:.4f}"],
], headers=["Metric", "Test"], tablefmt="fancy_grid"))
```

Explanation:

Standard metrics are calculated and printed in a structured format. The macro average ensures equal weight across classes.

Cross-Validation Accuracy Summary:

K-Fold Cross-Validation Results:

- Fold 0: 99.20%
- Fold 1: 99.60%
- Fold 2: 99.60%
- Fold 3: 100.00%
- Fold 4: 100.00%

Average Accuracy Across Folds: 99.68%

Aggregate Metrics Summary:

Table 7.1: Aggregate Metrics Summary – Mastitis Detection

Metric	Mean	Std Dev
Accuracy	0.996	0.003
Precision	0.9958	0.0032
Recall	0.9966	0.0028
F1 Score	0.9962	0.0029

Text Summary: "The Mastitis detection model was evaluated using 5-fold cross-validation. The model achieved a mean accuracy of 99.6%, with an F1-score consistently above 99.5% across all folds, indicating high robustness and generalization."

Test Set Summary: "When evaluated on the held-out test sets across 5 folds, the Mastitis model continued to demonstrate high reliability. The test accuracy ranged from 99.2% to 100%, with an average accuracy of 99.68%. Precision, recall, and F1-score remained consistently above 99.1%, affirming the model's robustness and its capability to generalize well on unseen data."

Classification Report:				
	precision	recall	f1-score	support
Healthy Udder	1.00	1.00	1.00	161
Infected Udder	1.00	1.00	1.00	341
accuracy			1.00	502
macro avg	1.00	1.00	1.00	502
weighted avg	1.00	1.00	1.00	502

Figure 7.1: Classification Report – Mastitis

Code Snippet:

```
cm = confusion_matrix(y_true_test, y_pred_test)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title(f'Confusion Matrix - Fold {fold}')
plt.show()
```

Explanation:

The confusion matrix visually illustrates model performance, showing true vs. predicted labels.

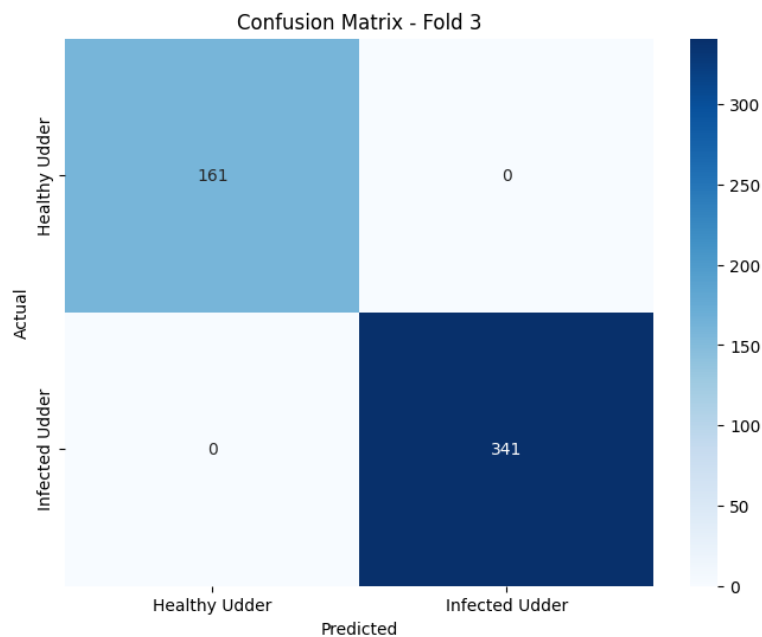


Figure 7.2: Confusion Matrix-Mastitis

7.5.1.3 Test Image Prediction Visualization

Code Snippet:

```
def visualize_all_test_images(model, test_loader, class_names, device="cuda" if torch.cuda.is_available() else
"cpu"):
    was_training = model.training
    model.eval()
    model.to(device)

    fig, axs = plt.subplots(nrows=4, ncols=9, figsize=(16, 9))
    axs = axs.flatten()

    with torch.no_grad():
        index = 0
        for images, labels in test_loader:
            images = images.to(device)
            labels = labels.to(device)
            outputs = model(images)
            _, preds = torch.max(outputs, 1)

            images = images.cpu()
            labels = labels.cpu()
            preds = preds.cpu()

            for i in range(images.size(0)):
                if index >= len(axs):
                    break

                npimg = images[i].permute(1, 2, 0).numpy()
                npimg = npimg * [0.229, 0.224, 0.225] + [0.485, 0.456, 0.406]
                npimg = np.clip(npimg, 0, 1)

                axs[index].imshow(npimg)
                axs[index].axis("off")

                pred_label = class_names[preds[i].item()]
                true_label = class_names[labels[i].item()]
                title_color = "green" if pred_label == true_label else "red"
                axs[index].set_title(f"Pred: {pred_label}\nTrue: {true_label}", fontsize=10, color=title_color)
```

```

    index += 1
    if index >= len(axes):
        break

plt.tight_layout()
plt.show()
model.train(mode=was_training)

```

Explanation:

This function displays 36 test images in a 4×9 grid, each annotated with the predicted and actual class. Titles are color-coded: **green** for correct predictions and **red** for incorrect ones. Images are de-normalized before visualization to restore their original appearance. The function also ensures the model's training state is preserved before and after visualization.

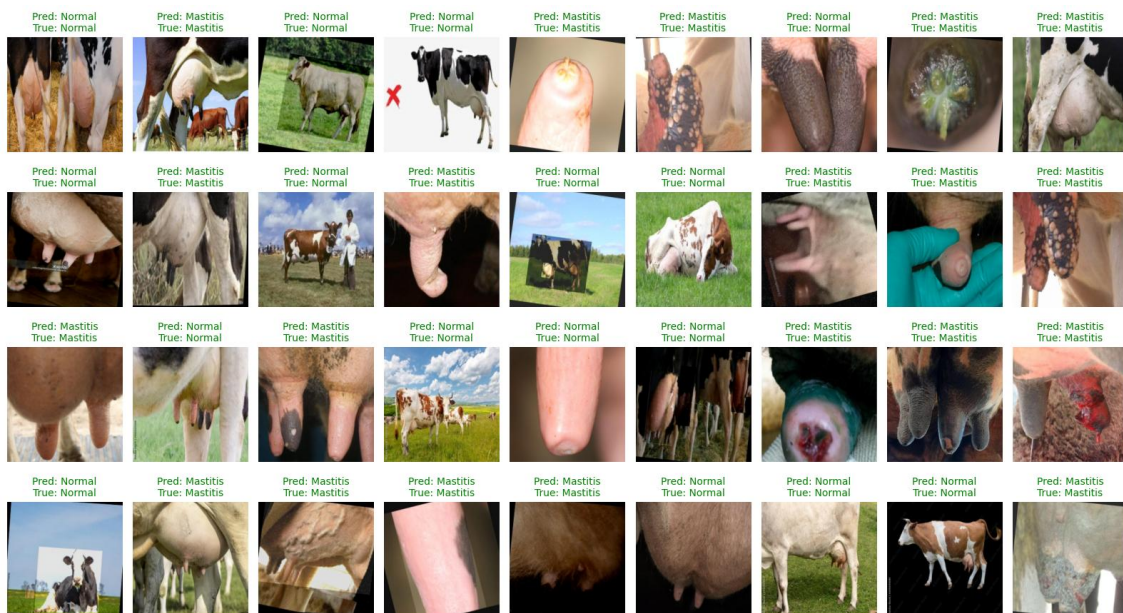


Figure 7.3: Visual Predictions Grid – Mastitis

7.5.2 Lumpy Skin Detection Results and Analysis

7.5.2.1 Evaluation Metrics on Test Set

Code Snippet:

```

model.eval()
test_loss = 0
correct_test = 0

```



```

total_test = 0
y_true_test, y_pred_test = [], []

with torch.no_grad():
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        test_loss += loss.item() * inputs.size(0)

        _, preds = torch.max(outputs, 1)
        correct_test += (preds == labels).sum().item()
        total_test += labels.size(0)

    y_true_test.extend(labels.cpu().numpy())
    y_pred_test.extend(preds.cpu().numpy())

```

Explanation:

Evaluation loop identical to the Mastitis model but trained for Lumpy Skin classification.

7.5.2.2 Performance Reporting and Confusion Matrix**Code Snippet:**

```

acc = correct_test / total_test
prec = precision_score(y_true_test, y_pred_test, average='macro')
rec = recall_score(y_true_test, y_pred_test, average='macro')
f1 = f1_score(y_true_test, y_pred_test, average='macro')
results[fold] = acc
print(tabulate([
    ["Loss", f"test_loss / total_test:.4f"],
    ["Accuracy", f"acc:.4f"],
    ["Precision", f"prec:.4f"],
    ["Recall", f"rec:.4f"],
    ["F1 Score", f"f1:.4f"],
], headers=["Metric", "Test"], tablefmt="fancy_grid"))

```

Explanation:

Results provide a quantitative overview of model performance on the test set.

Cross-Validation Accuracy Summary:

K-Fold Cross-Validation Results:

- Fold 0: 96.31%

- Fold 1: 98.74%

- Fold 2: 99.41%

- Fold 3: 99.41%

- Fold 4: 99.58%

Average Accuracy Across Folds: 98.69%

Aggregate Metrics Summary:

Table 7.2: Aggregate Metrics Summary – Lumpy Skin Detection

Metric	Mean	Std Dev
Accuracy	0.9869	0.0121
Precision	0.9868	0.0125
Recall	0.9868	0.0123
F1 Score	0.9868	0.0123

Text Summary: "The Lumpy Skin detection model was evaluated using 5-fold cross-validation. The model achieved a mean accuracy of 98.7%, with an F1-score consistently above 98.6% across all folds, demonstrating strong performance and generalization across different splits."

Test Set Summary: "The model's performance on the Lumpy Skin test sets was consistent with cross-validation results. Test accuracy ranged from 96.3% to 99.6% across folds, with an average accuracy of approximately 98.69%. Precision, recall, and F1-scores were all well above 96%, reinforcing the model's stability and reliability on real-world unseen data."

Classification Report:				
	precision	recall	f1-score	support
Lumpy Skin	0.99	1.00	1.00	534
Normal Skin	1.00	1.00	1.00	657
accuracy			1.00	1191
macro avg	1.00	1.00	1.00	1191
weighted avg	1.00	1.00	1.00	1191

Figure 7.4: Classification Report – Lumpy Skin

Code Snippet:

```

cm = confusion_matrix(y_true_test, y_pred_test)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title(f'Confusion Matrix - Fold {fold}')
plt.show()

```

Explanation:

The confusion matrix helps assess which classes the model confuses most often.

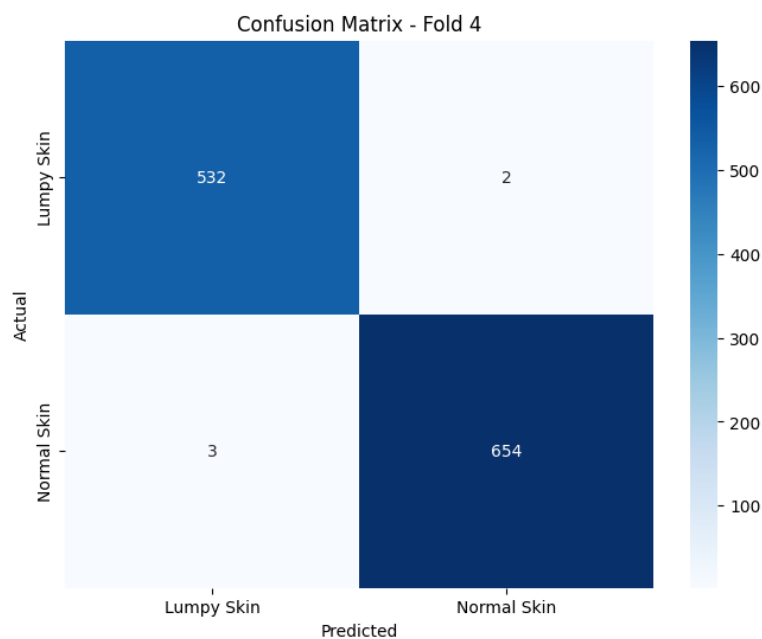


Figure 7.5: Confusion Matrix – Lumpy Skin

7.5.2.3 Test Image Prediction Visualization

Refer to the same `visualize_all_test_images()` function described in Section 7.5.1.3. It is reused here to visually assess the classification quality on the Lumpy Skin dataset.

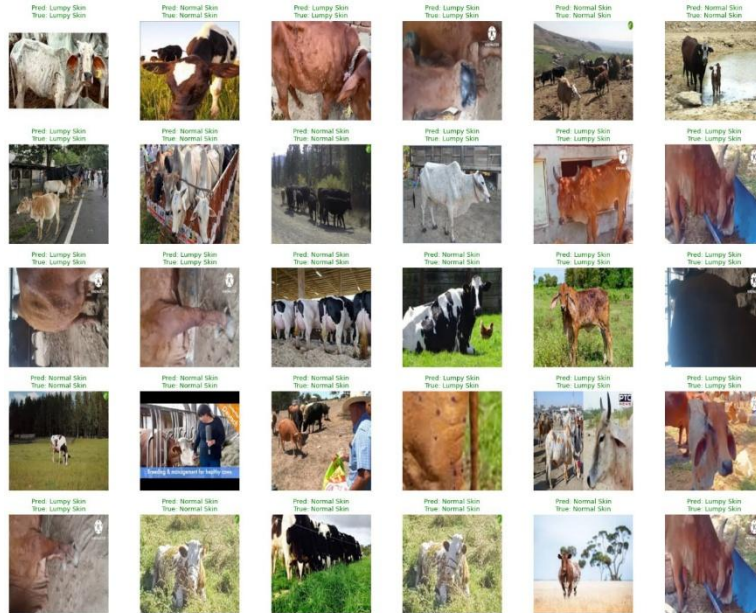


Figure 7.6: Visual Predictions Grid – Lumpy Skin

7.5.3 YOLOv8 Cow Detection Results and Analysis

7.5.3.1 Training Summary

"The YOLOv8 cow detection model was trained for 50 epochs using a dataset of annotated cow images. The model's final evaluation on the validation set yielded a $mAP@0.5$ of 97.5%, and a $mAP@0.5:0.95$ of 77.3%. Training convergence was stable, with consistent improvement in loss and accuracy across epochs.

7.5.3.2 Test Set Metrics Summary

Table 7.3: YOLOv8 Test Set Performance Metrics

Metric	Value
mAP@0.5	0.975
mAP@0.5:0.95	0.773
Precision	0.943
Recall	0.948
Number of Classes	1 (Cow)

Text Summary: "The YOLOv8 model achieved excellent detection performance, with mAP@0.5 reaching 97.5% and precision/recall values exceeding 94%. The model is efficient and capable of detecting cows in various lighting and environmental conditions, making it a robust solution for automated monitoring."

7.5.3.3 Evaluation Strategy

The YOLOv8 object detection model was evaluated on a separate test set using mean Average Precision (mAP), Precision, and Recall metrics. The model was trained on annotated images of cows for detection under various field conditions.

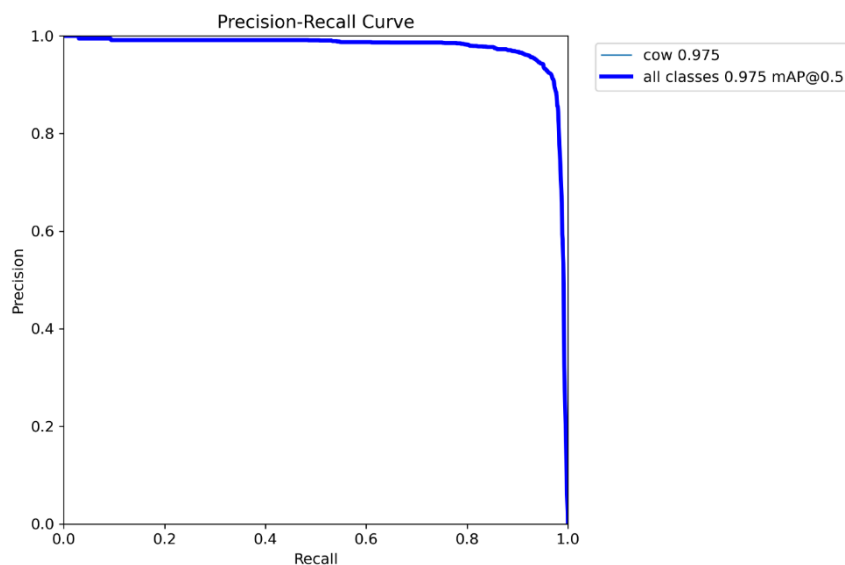
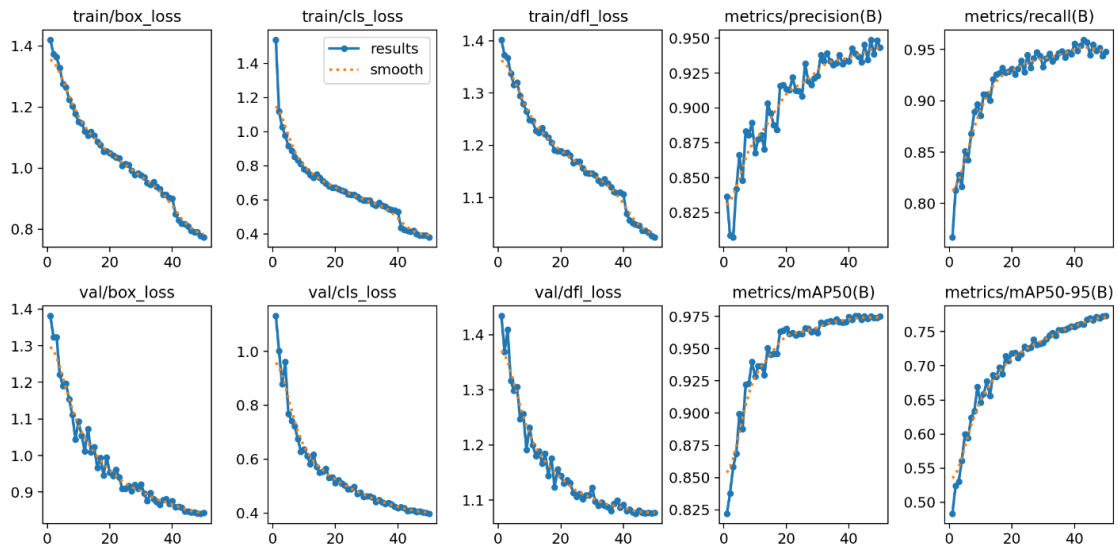


Figure 7.7: Precision-Recall Curve – YOLOv8**Figure 7.8: Training mAP per Epoch Chart – YOLOv8****Figure 7.9: Sample Detection Results – YOLOv8**

Chapter 8

FEED OPTIMIZATION SYSTEM

8.1 System Overview and Motivation

Introduction to Livestock Feeding Challenges

Feeding optimization is a cornerstone of efficient livestock farming. Improper feed formulations can result in reduced productivity, imbalanced nutrition, and increased veterinary costs. Addressing these challenges requires a dynamic, intelligent system that adapts feeding recommendations to the specific needs of each cow based on health, breed, production goal, and environmental factors.

Why Optimization Is Critical

An optimal feeding strategy enhances:

- Milk and meat productivity
- Animal health and immune response
- Feed cost efficiency
- Environmental sustainability

Our system leverages AI agents within the CrewAI framework to generate, analyze, and recommend ideal feed formulations tailored to each cow's profile.

8.2 Agent-Based Architecture with CrewAI

8.2.1 Overview of CrewAI Framework

CrewAI enables task decomposition through multiple collaborating agents, each with a specific role in the feed optimization pipeline. Each agent receives input, performs a focused analysis or generation, and passes results to subsequent agents.

8.2.2 Agents

FEED_AGENT

- **Role:** Cattle Nutrition Expert
- **Goal:** Generate an ideal feed formulation based on animal profile
- **Backstory:** Expert in local feed composition, balancing energy, protein, and seasonal needs.

BALANCE_CHECK_AGENT

- **Role:** Feed Balance Auditor
- **Goal:** Identify nutrient gaps or excesses compared to the ideal formulation.
- **Backstory:** Specialist in nutrition imbalances and their impact on health and productivity.

FEED_KEYWORD_GENERATOR_AGENT

- **Role:** Keyword Generator for Feed Search
- **Goal:** Generate localized and precise search queries for relevant feed products.
- **Backstory:** Trained in optimizing keyword generation based on nutrition components.

SEARCH_ENGINE_AGENT

- **Role:** Online Feed Product Search Tool
- **Goal:** Extract live feed product data including name, ingredients, and price.
- **Backstory:** Web search and extraction expert focused on cattle feed.

PROCUREMENT_REPORT_AUTHOR_AGENT

- **Role:** Product Procurement Evaluator
- **Goal:** Compare products and recommend the best match.
- **Backstory:** Specialist in comparing feed specs and recommending cost-effective solutions.

8.2.3 Agent Workflow & Interaction

The CrewAI workflow executes the following steps sequentially:

1. **Generate Ideal Feed Formula** (FEED_AGENT)
2. **Check Balance vs. Current Feed** (BALANCE_CHECK_AGENT)
3. **Generate Search Keywords** (FEED_KEYWORD_GENERATOR_AGENT)
4. **Search Online Products** (SEARCH_ENGINE_AGENT)
5. **Recommend Best Feed** (PROCUREMENT_REPORT_AUTHOR_AGENT)

All outputs are stored in structured JSON files and consumed downstream by subsequent agents.

8.3 LLM Integration and Implementation Details

8.3.1 LLM for Report Synthesis

Each agent is powered by Google's Gemini 2.0 Flash model, configured via the CrewAI LLM abstraction. The model receives both structured input and domain-specific instructions to generate nutritional insights and feed recommendations.

```
basic_llm = LLM(
    model="gemini/gemini-2.0-flash",
    temperature=0.2,
    api_key=GEMINI_API_KEY
)
```

8.3.2 Code Snippets and Tools Used

Defining the Feed Agent Task

```
feed_task = Task(
    description=f"""
    التالية البقرة بيانات على بناء:
    - النوع: {crew_inputs['type']}
    - العمر: {crew_inputs['age']} سنة
    - الوزن: {crew_inputs['weight']} كجم
    - الهدف: {crew_inputs['goal']}
    - الحليب إنتاج: {crew_inputs['milk_yield']} لتر/يوم
    - السلالة: {crew_inputs['breed']}
    - المستخدم العلف: {crew_inputs['current_feed']}
    على تحتوي مثالية علف توليفة بإنشاء قم:
    - الشامية الذرة البرسيم، مثل) الخشن العلف
    - الصويا فول الذرة، مثل) المركز العلف
    - والفيتامينات المعادن مثل) الغذائية المكملات -
    """
```

```

- اليومية المياه كمية -
بقرة لكل يومية هي الكميات جميع
:على عنصر كل يحتوي أن يجب
- العلف نوع
- اللبتر أو جرام بالكيلو الكمية
- وجدت إن) البروتين نسبة
- {crew_inputs['language']} باللغة وتكتب للمربي عملية ملاحظات
"""
expected_output="JSON",
output_json=IdealFeedFormulation,
output_file=os.path.join(output_dir, "step_1_feed_recommendation.json"),
agent=feed_agent
)

```

Explanation:

This task instructs the FEED_AGENT to generate a feed formulation based on the cow's weight, type, and production goal. The result must include all components of a daily ration.

Creating the Feed Balance Check Task

```

balance_check_task = Task(
    description=f"""
المثالية والتوليفة الحالي العلف بين قارن
غذائية فائضات أو نواقص 3-5 أبرز ** فقط حدد
واضحة نقاط باستخدام {crew_inputs['language']} باللغة التقرير اكتب
الزيادة؟ أو النقص هو ما -
الإنتاج؟ على ذلك يؤثر كيف -
المقترح؟ الحل ما -
المتوقعة؟ الإضافية التكلفة هي ما -
"""
    expected_output="JSON",
    output_json=FeedComparisonReport,
    output_file=os.path.join(output_dir, "step_2_feed_balance_check.json"),
    agent=balance_check_agent
)

```

Explanation:

This agent evaluates the difference between the current and ideal feed, and highlights nutritional imbalances (deficiencies or excesses).

Keyword Generation Task

```

feed_keyword_generation_task = Task(
    description=f"""
الأولى الخطوة في إنشاؤها تم التي المثالية العلفية التوليفة إلى استناداً
- ملف 'step_1_feed_recommendation.json' اقرأ
- (المكملات البروتين، المركز، الخشن، العلف) التوليفة مكونات استخراج
- {crew_inputs['language']} باللغة بحث كلمات ولد
- {crew_inputs['type']} النوع هي التي الأبقار أعلاف على فقط ركز

```

```

- العمر: {crew_inputs['age']} سنة
- الوزن: {crew_inputs['weight']} كجم
- الهدف: {crew_inputs['goal']}
- السلالة: {crew_inputs['breed']} - الأسماك أو الدواجن تشمل لا
- العلف نوع أو البروتين نسبة مثل واضحة مكونات إلى الكلمات تشير أن يجب
مفتاحية كلمات {crew_inputs['no_keywords']} استخراج
"""
expected_output="JSON",
output_json=SuggestedSearchQueries,
output_file=os.path.join(output_dir, "step_3_feed_search_keywords.json"),
agent=feed_keyword_generator_agent
)

```

Explanation:

This task transforms feed components into actionable search terms that will be used to fetch real-world product data.

Searching for Feed Products

```

@tool
def search_engine_tool(query: str):
    results = search_client.search(query=query)
    return json.dumps(results)

task_search_products = Task(
    description=f"""
الأبقار أعلاف عن الإنترنت عبر بحث لتنفيذ المفتاحية الكلمات استخدم
نتيجة كل تشمل أن يجب:
- المنتج اسم
- البروتين نسبة
- الكيلو غرام سعر
- الرئيسية بالمكونات قائمة
- (أمكن إن 5 إلى 0 من) البائع تقييم
- المنتج رابط
كاملة مواصفات بدون أو موثوقة غير نتائج أي تجاهل
{crew_inputs['country_name']} اللغة و {crew_inputs['language']}.
البلد من تستخرج حاجة أهم
"""
    expected_output="JSON",
    output_json=FeedProductResult,
    output_file=os.path.join(output_dir, "step_4_feed_search_results.json"),
    agent=search_engine_agent
)

```

Explanation:

This tool interfaces with Tavily to return live search results for a given query string. It is used by the SEARCH_ENGINE_AGENT.

Product Evaluation Task

```
task_recommend_product = Task(
    description=f"""
المثالية التوليفة مع مواصفاتها تطابق لمدى وفقاً السابق الوكيل من نتجت التي المنتجات جميع بتقييم قم
اهملها شغالة تكن لم لو وشغالة، حقيقة أنها من التحقق يجب وهمية، URL عناوين تجلب لا
**فقط واحد منتج أفضل** اختيار ويتم، **الأقل على منتجات 3** مقارنة يجب
ذلك من التأكد يجب، {crew_inputs['country_name']} دولة داخل تكون URL وعناوين النواتج كل أن تأكد
{crew_inputs['language']} باللغة بسيطة جدول بصيغة التوصية اعرض
""",
    expected_output="JSON",
    output_json=BestFeedRecommendationOutput,
    output_file=os.path.join(output_dir, "step_5_procurement_recommendation.json"),
    agent=procurement_report_author_agent
)
```

Explanation:

This final task compares at least 3 candidate products and chooses the most suitable one based on multiple criteria.

8.4 Output Testing and Validation

To ensure the reliability and accuracy of the feed optimization pipeline, each component in the system was rigorously tested using structured input scenarios. These tests aimed to validate the correctness of the AI-generated outputs and ensure they aligned with expected nutritional logic and scientific principles.

Each test scenario simulated a realistic cow profile, including biological, environmental, and management parameters such as age, breed, weight, production goals, activity level, current feed, and health status. The goal was to observe how the system handled diverse input combinations and whether the resulting recommendations were coherent, contextually relevant, and nutritionally sound.

8.4.1 Sample Cow Profile Input

The following example represents a typical input scenario used to test the system's ability to personalize feeding strategies

```
{
  "type": "dairy",
  "age": 4,
  "weight": 600,
  "goal": "milk",
  "milk_yield": 20.5,
  "pregnant": true,
  "breed": "Friesian",
  "current_feed": ["corn", "barley", "soybean meal"],
  "activity_level": "medium",
  "health_status": "good",
  "season": "spring",
  "animal_type": "البقرة",
  "production_goal": "الحليب إنتاج زيادة",
  "country_name": "Egypt",
  "product_name": "livestock nutrition",
  "no_keywords": 10,
  "language": "Arabic",
  "score_th": 0.7
}
```

8.4.2 Ideal Feed Formulation Output

After processing the input data, the system generates a detailed feed formulation that outlines the optimal daily nutritional plan for the animal. A sample output is shown below:

```
{
  "roughage": {
    "name": "المجفف البرسيم",
    "amount_kg": 12.0,
    "protein_percent": 18.0,
    "fiber_percent": 25.0,
    "energy_mcal": 2.2,
    "notes": "وخلوه البرسيم جودة من التأكد يجب. الحليب وإنتاج الهضمي الجهاز لصحة اللازمين والبروتين الألياف يوفر المجفف البرسيم",
    "sources": [
      "المحلية المراعي",
      "الأعلاف موردو"
    ]
  }
}
```

Explanation:

This output defines the recommended quantity and nutritional characteristics of the roughage component. It also provides practical notes and reliable sourcing suggestions, reinforcing both scientific validity and operational feasibility.

8.4.3 Nutrition Balance Evaluation Output

The system is also capable of analyzing the difference between the animal's current feed and the scientifically ideal formulation. A representative output is as follows:

```
{  
  "balanced": false,  
  "missing_nutrients": [  
    "6 وأوميغا 3 أوميغا) الأساسية الدهنية الأحماض",  
    "للذوبان القابلة غير الألياف",  
    "والكولين البيوتين مثل الفيتامينات بعض"  
  ]  
}
```

Explanation:

The result indicates that the current feed lacks specific essential nutrients. The system intelligently identifies these gaps and flags imbalances, enabling corrective action to be taken. This function plays a critical role in maintaining animal health and optimizing productivity.

8.4.4 Best Feed Recommendation Endpoint

In addition to formulating ideal plans, the system evaluates available commercial feed products and recommends the most suitable option based on nutritional match and supplier credibility. A sample result is shown below:

```
{
  "best product": {
    "product_name": "سينا شركة - للأبقار متكامل علفي مكمل",
    "protein_percent": 0.0,
    "price_per_kg": 15.0,
    "key ingredients": [
      "كالسسيوم",
      "فسفور",
      "(كولين بيوتين، هـ، د، أ) فيتامينات"
    ],
    "matches ideal formula": true,
    "match_reason": "والكولين البيوتين ذلك في بما بها، الموصى والفيتامينات المعادن جميع على يحتوي",
    "supplier_rating": 4.2,
    "url": "https://www.example.com/product3"
  },
  "recommendation reason": "والفيتامينات المعادن جميع يوفر لأنه منتج كأفضل سينا شركة من المتكامل العلفي المكمل اختيار تم",
  "generated_date": "2025-06-24",
  "generated_time": "06:24:07"
}
```

أعلى سعره أن من الرغم على. المحسنة التوليفة في بهما الموصى والكولين البيوتين ذلك في بما الحلوب، الأبقار تحتاجها التي الأساسية،
 كامل بشكل الأبقار احتياجات ويغطي عالية غذائية قيمة يوفر أنه إلا نسيباً،

Explanation:

This output presents a data-driven recommendation of a real-world product. The system justifies its choice by evaluating nutritional completeness and practical considerations such as supplier rating and pricing. This ensures that the user receives not just a theoretically optimal plan, but also a viable and market-aligned solution.

The results across all test scenarios confirm the system's ability to generate consistent, high-quality, and actionable feed recommendations. Furthermore, the system demonstrates strong reasoning capabilities, combining nutritional logic, veterinary guidance, and commercial relevance to support precision livestock nutrition.

Chapter 9

CHATBOT SYSTEM

9.1 Introduction

The Chatbot System represents an integral component of the smart cattle management platform. It provides an intelligent, natural language-based interface that allows users to interact with the platform through free-text queries. This capability empowers various stakeholders, including farm managers, veterinarians, and support staff, to retrieve insights, access documentation, analyze farm data, or seek general knowledge in a seamless and user-friendly manner.

To accomplish these goals, the system leverages a modular multi-agent architecture supported by advanced Large Language Models (LLMs). Each agent within this architecture is tailored to a specific type of reasoning or information source. A centralized routing mechanism, powered by an LLM, dynamically determines which agent to engage based on the nature and intent of the user query.

9.2 System Architecture and Agent Design

The chatbot system is built upon a multi-agent framework that integrates three distinct agents, each with specialized capabilities:

- **RAG Agent (Retrieval-Augmented Generation):** Responsible for answering questions based on unstructured internal documents such as PDF manuals and disease guidelines.
- **Pandas Agent:** Designed to handle structured data queries, particularly over farm sensor datasets stored in CSV format.

- **Web Search Agent:** Facilitates retrieval of real-time knowledge from external web sources using the Tavily API and summarization through an LLM.

These agents work in coordination under the supervision of a central classification and routing component, ensuring that each user query is directed to the appropriate processing module.

9.3 Question Classification and Routing Logic

To determine the appropriate tool for handling a given user query, the system incorporates a classification model based on Gemini 1.5 Flash. The model evaluates the query and assigns it one of the following labels:

- **pandas:** Structured data queries related to farm records or sensor logs.
- **rag:** Questions that require information from internal documents.
- **web:** Open-domain queries that require external search and real-time information.

Code Snippet: Classification Function

```
def classify_question(question: str) -> str:
    prompt = f"""
    You are a smart question classifier.

    You must choose ONLY ONE of the following labels:
    - pandas → for questions about structured tabular data, like cow records or farm environment data
    - rag → for questions based on PDF documents, like diseases, symptoms, or treatments
    - web → for general knowledge or online-related questions that are not covered by internal data

    Think carefully.

    Question: {question}
    Answer (one word only):
    """
    response = model.generate_content(prompt)
    return response.text.strip().lower()
```

Explanation:

This function prompts the Gemini model with a structured instruction to classify the user query into one of three labels (pandas, rag, or web). The result of this classification determines the path of execution in the chatbot system.

This classification output ensures that each query is routed to the most relevant agent in the pipeline.

9.4 RAG-Based Document Reasoning

9.4.1 Objective and Use Case

The Retrieval-Augmented Generation (RAG) module is designed to handle user questions that depend on unstructured documentation, such as veterinary treatment protocols, farm policies, or disease identification guides.

9.4.2 Knowledge Base Construction

PDF documents are preprocessed and transformed into searchable knowledge chunks using the following steps:

- Chunking via RecursiveCharacterTextSplitter.
- Embedding using GoogleGenerativeAIEmbeddings.
- Storage in a Pinecone vector database for semantic retrieval.

Code Snippet: RAG Initialization

```
def initialize_knowledge_base(pdf_dir: str, index_name: str = "moonetbot"):
    """
    Preprocess PDF files and upload embeddings to Pinecone with improved chunking and metadata.
    Run this ONCE.
    """
    # Load PDF documents
    loader = DirectoryLoader(pdf_dir, glob="*.pdf", loader_cls=PyPDFLoader)
    documents = loader.load()

    # Add page metadata if missing
    for doc in documents:
        if "page" not in doc.metadata:
            doc.metadata["page"] = doc.metadata.get("source", "").split("-")[-1].replace(".txt", "")

    # Improved chunking
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1200,
        chunk_overlap=200,
        separators=["\n\n", "\n", ".", " ", ""]
    )
    chunks = splitter.split_documents(documents)

    # Embeddings
    embeddings = GoogleGenerativeAIEmbeddings(model="models/embedding-001")

    # Pinecone setup
    pc = Pinecone(api_key=os.getenv("PINECONE_API_KEY"))
    if index_name in pc.list_indexes().names():
        pc.delete_index(index_name)

    pc.create_index(
        name=index_name,
        dimension=768,
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1")
    )

    # Upload chunks
    batch_size = 50
    for i in range(0, len(chunks), batch_size):
        batch = chunks[i:i + batch_size]
        PineconeVectorStore.from_documents(
            documents=batch,
            index_name=index_name,
            embedding=embeddings
        )

    print(f"Knowledge base initialized with {len(chunks)} chunks.")
```

Explanation:

This function initializes the knowledge base by loading and chunking PDF documents, generating vector embeddings, and storing them in Pinecone. This process is performed once and enables the RAG agent to retrieve relevant information efficiently during query time.

9.4.3 Semantic Answering Pipeline

When a query is submitted:

1. Relevant document chunks are retrieved based on semantic similarity.
2. A structured system prompt is composed and sent to the LLM.
3. The LLM generates a contextually grounded answer in Markdown format.

Code Snippet: Answer Retrieval

```
def get_answer(question: str, index_name: str = "moonetbot"):
    """
    Retrieve the best answer to a user query using Pinecone and Gemini 2.0 LLM.
    """
    # Load vector store
    embeddings = GoogleGenerativeAIEmbeddings(model="models/embedding-001")
    vector_store = PineconeVectorStore.from_existing_index(index_name=index_name,
                                                            embedding=embeddings)

    # Retrieve documents
    results = vector_store.similarity_search_with_score(question, k=30)
    filtered_docs = [doc for doc, score in results if score >= 0.65 and hasattr(doc, "page_content")]
    sources = [doc.metadata.get("source", None) for doc in filtered_docs]

    # General-purpose prompt
    system_prompt = (
        "You are an expert assistant helping users extract accurate and comprehensive information from the\n"
        "provided context.\n"
        "- Use only the given context to answer the question.\n"
        "- If the context does not contain the answer, reply: 'The document does not contain this information.'\n"
        "- Do not speculate or invent information.\n\n"
        "Format the response in **Markdown**, using:\n"
        "- Headings (##)\n"
        "- Bullet points\n"
        "- **Bold** for key terms or data points\n"
        "- Reference page numbers or sections if available.\n\n"
        "CONTEXT:\n{context}"
    )

    prompt = ChatPromptTemplate.from_messages([
        ("system", system_prompt),
        ("human", "{input}")
    ])
    )
```

```

llm = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash-exp",
    temperature=0.4,
    max_output_tokens=1024
)

qa_chain = create_stuff_documents_chain(llm, prompt)
response = qa_chain.invoke({"input": question, "context": filtered_docs})

# Combine response with sources in one string
unique_sources = sorted(set(filter(None, sources)))
sources_str = "\n".join([f"- {src}" for src in unique_sources]) if unique_sources else "- No sources found."

full_output = f"{response}\n\n---\n\n📄 **Sources:**\n{sources_str}"

return full_output

```

Explanation:

This function performs semantic search by retrieving the most relevant chunks from the knowledge base and feeding them to the LLM along with a prompt. The generated answer is contextually accurate and accompanied by source references.

This agent ensures reliability by only using verifiable context and including source references in the output.

9.5 Structured Data Analysis with Pandas Agent

9.5.1 Purpose

The Pandas Agent is designed to answer data-centric queries related to environmental sensor readings (e.g., temperature, humidity, gas levels) stored in structured CSV format. It allows users to ask natural language questions such as “How many days had high gas levels?” and receive precise, code-free analytical responses powered by a Large Language Model (LLM) and the Pandas library.

This module eliminates the need for technical coding knowledge while maintaining the power of data exploration through programmatically generated Python functions.

9.5.2 Workflow and Execution

The execution pipeline for the Pandas Agent consists of the following sequential steps:

1. **Loading and inspecting the dataset** from the farm's environment CSV file.
2. **Extracting column names and sample values** to inform the LLM of the dataset structure.
3. **Prompting the Gemini model** to synthesize a Pandas function capable of answering the user's question.
4. **Executing the generated code** securely using `exec()` to obtain the raw answer.
5. **Refining the output** using an LLM summarization step to return a clear, user-friendly response.

Code Snippet: Data Loading and Structure Setup

```
load_dotenv()

# Configure Gemini API
GEMINI_API_KEY = os.getenv("GOOGLE_API_KEY")
genai.configure(api_key=GEMINI_API_KEY)
model = genai.GenerativeModel('gemini-1.5-flash')

# ☒ File path (adjust if needed)
CSV_FILE = r"chatbot/farm_environment.csv"

def load_data():
    try:
        df = pd.read_csv(CSV_FILE)
        return {"farm_environment": df}
    except Exception as e:
        print(f"Error loading CSV file: {e}")
        return None

dfs = load_data()
```

Explanation:

This code sets up the environment, loads the Gemini LLM, and reads the environmental data CSV into a DataFrame dictionary `dfs`. This dataset will be referenced by the LLM when generating dynamic queries.

Code Snippet: Extracting Table Info and Generating a Pandas Function

```
def generate_pandas_query(question: str, tables_to_use: List[str]) -> str:
    columns_info = ""
    if dfs is None:
        return "def get_answer(dfs):\n    return 'Data not loaded.'"
    for table in tables_to_use:
        if table in dfs:
            df = dfs[table]
            columns_info += f"\nTable '{table}' Columns:\n- " + "\n- ".join(df.columns) + "\n"
            for col in df.columns:
                values = df[col].dropna().astype(str).unique()[:3]
                columns_info += f"    → Examples from '{col}': {', '.join(values)}\n"
            columns_info += "\n"

    prompt = f"""
You are a smart Python assistant working with tabular farm data.

Your task is to write a Python function that answers the question:
"{question}"

Available DataFrames:
{columns_info}

Use DataFrame variable named dfs["farm_environment"] to access the data.
Your answer must:
- Identify relevant columns even if the names don't match exactly.
- Use pandas to extract the full answer.
- Return all matching rows (if multiple).
- Return a short, user-friendly string answer.
- Handle errors gracefully (e.g., data not found).
- Avoid unnecessary explanation.

Return ONLY the function in this format:
```python
def get_answer(dfs):
 # your code
 return "your answer"
```
"""

    response = model.generate_content(prompt)
    return response.text.strip()
```

Explanation:

This function constructs a prompt for the Gemini LLM using metadata extracted from the dataset (e.g., column names and value samples). It instructs the model to return a valid Python function tailored to the user's question and dataset structure.

Code Snippet: Cleaning and Executing the Generated Code

```
def clean_code(code: str) -> str:
    lines = code.strip().splitlines()
    cleaned_lines = [line for line in lines if not line.strip().startswith("```")]
    return "\n".join(cleaned_lines).strip()

def execute_pandas_query(code: str):
    try:
        local_namespace = {'dfs': dfs, 'pd': pd, 'np': np}
        exec(code, globals(), local_namespace)
        function_name = next((name for name in local_namespace if callable(local_namespace[name]) and name
!= 'pd' and name != 'np'), None)
        if function_name:
            return local_namespace[function_name](dfs)
            return "No answer function found."
    except Exception as e:
        return f"Error: {str(e)}"
```

Explanation:

The `clean_code` function removes formatting tags from the LLM's output. The `execute_pandas_query` function runs the generated code safely in a local namespace and captures the result of the dynamic function.

Code Snippet: Summarization and Final Output Delivery

```
def refine_answer_with_llm(question: str, raw_answer: str) -> str:
    prompt = f"""
    You are a helpful assistant summarizing farm data answers.

    Question: {question}
    Raw answer (from data): {raw_answer}

    Rephrase the answer clearly and briefly. If already clear, keep it.
    """
    try:
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception:
        return raw_answer

def get_answer(question: str) -> str:
    categories = extract_categories(question)
    tables_to_use = get_tables(categories)
    pandas_code = generate_pandas_query(question, tables_to_use)
    cleaned_code = clean_code(pandas_code)
    result = execute_pandas_query(cleaned_code)
    final_answer = refine_answer_with_llm(question, result)
    return final_answer
```


Explanation:

The final step involves sending the raw result back to the LLM for rephrasing into a human-friendly sentence. The `get_answer()` function orchestrates the full pipeline from query to final response.

9.6 Real-Time Knowledge via Web Search Agent

9.6.1 Objective

The Web Search Agent is responsible for retrieving and summarizing real-time external information, especially for queries that are beyond the scope of the internal system. This includes treatment updates, new research, or market conditions.

9.6.2 Process Flow

1. Query is sent to Tavily API to retrieve top 10 relevant search results.
2. Duplicates and short entries are filtered out.
3. The remaining content is summarized by Gemini 2.0 Flash using a structured prompt.

There are two variants:

- **Farmer Version:** Optimized for readability with headings and bullet points.
- **Veterinarian Version:** Includes inline citations for traceability.

Code Snippet: Web Search Execution

```
def run_web_search(question: str) -> str:
    try:
        search_results = search_tool.invoke(question)
    except Exception as e:
        return f"❌ Error during search: {e}"

    # Filter & Clean
    seen_urls = set()
    filtered = []
    for res in search_results:
        url, content = res.get("url"), res.get("content", "").strip()
        if url and content and len(content) > 100 and url not in seen_urls:
            seen_urls.add(url)
            filtered.append(res)

    if not filtered:
        return "***The search results do not contain this information.***"

    # Format Context
    formatted_context = "\n\n".join([
        f"- {res['content'].rstrip().rstrip('.')}\n\n (source: {res['url']})"
        for res in filtered
    ])

    heading = f"## {question.strip()}"
    full_input = f"{heading}\n\n{question.strip()}"

    input_vars = {
        "context": formatted_context,
        "input": full_input
    }

    try:
        chain = prompt | llm
        response = chain.invoke(input_vars)
    except Exception as e:
        return f"❌ Error during generation: {e}"

    return response.content.strip()
```

Explanation:

This function uses Tavily to retrieve relevant web content and summarizes the results using the Gemini LLM. It ensures that responses are accurate, well-structured, and grounded in current online sources.

This agent ensures that the chatbot remains dynamically updated and capable of handling external knowledge queries.

9.7 Centralized Smart Answer Routing

Two core functions consolidate the classification and execution process:

- `get_smart_answer_farmer()`
- `get_smart_answer_vet()`

Each function performs the following steps:

1. Classify the question.
2. Route the question to the corresponding agent.
3. Collect and format the answer.

Code Snippet: Central Routing Logic

```
def get_smart_answer_farmer(question: str) -> str:
    query_type = classify_question(question)

    if query_type == "pandas":
        answer = get_agent_answer(question)
        return json.dumps({"answer": answer, "type": "Environment AGENT"})

    elif query_type == "rag":
        answer = rag_get_answer(question)
        return json.dumps({"answer": answer, "type": "RAG"})

    elif query_type == "web":
        answer = chatbot.web_farmer.run_web_search(question)
        return json.dumps({"answer": answer, "type": "WEB"})

    else:
        return json.dumps({"error": "Couldn't classify your question.", "type": None})

def get_smart_answer_vet(question: str) -> str:
    query_type = classify_question(question)

    if query_type == "pandas":
        answer = get_agent_answer(question)
        return json.dumps({"answer": answer, "type": "Environment AGENT"})

    elif query_type == "rag":
        answer = rag_get_answer(question)
```

```
return json.dumps({"answer": answer, "type": "RAG"})

elif query_type == "web":
    answer = chatbot.web_vet.run_web_search(question)
    return json.dumps({"answer": answer, "type": "WEB"})

else:
    return json.dumps({"error": "Couldn't classify your question.", "type": None})
```

Explanation:

This function acts as the central router for handling farmer-specific queries. It classifies the question, then delegates the query to the appropriate agent based on its type and returns the final response.

The returned output is a structured JSON containing both the final answer and the reasoning tool used.

9.8 Output Formatting and User Interaction

The chatbot system returns all results in a structured JSON format, containing both the generated answer and the type of reasoning tool used. The type helps the frontend determine how to format and present the response. Below are three sample queries, each handled by a different agent, and their corresponding outputs:

9.8.1 Example 1 – Structured Data Query (Pandas Agent)

User Query: "How many days had a temperature greater than 25°C?"

```
{
  "answer": "8 days.",
  "type": "Environment AGENT"
}
```

9.8.2 Example 2 – Internal Knowledge Query (RAG Agent)

User Query: "What is the treatment for mastitis?"

```
{
  "answer": ""
}
Answer:
The document provides information on antimicrobial therapy for mastitis, including during lactation and the dry period. Key aspects of the treatment are:

## Antimicrobial Therapy for Mastitis

* **Response Assessment:** Effectiveness should be assessed 2-3 weeks post-treatment using clinical exams, bacterial cultures, somatic cell counts, and milk production levels (p. 397).
* **Factors Affecting Response:** Discrepancies in expected response can be due to host defense, bacterial resistance, altered drug behavior, or poor drug distribution (p. 397).
* **Staph. aureus Infections:** These can be difficult to treat due to the bacteria's location in microabscesses and potential dormancy (p. 397).
* **Bacteriological Cure Rates:** Staphylococcal mastitis has low cure rates during lactation (p. 397).
* **Treatment During Lactation:**
  * Factors to consider include antibacterial spectrum, cost, steroid presence, treatment history, availability of a parenteral preparation, milk withholding time, and personal preference (p. 397).
  * A pretreatment milk sample should be collected for bacterial culture to aid in selecting an antimicrobial for empirical treatment (p. 333).
  * Concurrent use of parenteral and quick-release intramammary preparations may be beneficial (p. 333).
  * Oxytocin can be used to facilitate stripping of milk from the infected quarter (p. 333).
* **Parenteral Preparations:** Severe mastitis is usually treated systemically, often with adjunctive intramammary therapy (p. 395). An ideal antimicrobial should have a low MIC, high bioavailability, and be lipid-soluble (p. 395).
* **Intramammary Preparations:** Available as single-use syringes for 1, 2, or 3 treatments at 12- or 24-hour intervals. Lactating cow products are designed for rapid clearance and shorter milk withholding periods. Dry-cow formulations have extended drug concentrations (p. 395).
* **Strep. agalactiae:** Intramammary treatment is effective, and these bacteria are very sensitive to penicillin antibiotics (p. 335).
* **Coliform Mastitis:** Treatment involves frequent stripping to remove endotoxin, antibiotics, supportive therapy (including hypertonic saline and oxytocin), and anti-inflammatory drugs (p. 337).
* **Peracute Mastitis:** Frequent stripping of the affected quarter following oxytocin administration is important. Antibacterial therapy with gentamicin, polymyxin B, and potentiated sulphonamides may be suitable (p. 398). Intravenous fluids may also be needed (p. 398).

## Additional factors

* **Selective Dry-Cow Therapy:** Recommended when the monthly bulk milk cell count is below 200,000 and the quarter infection rate is less than 15%. Treat cows with peak somatic cell count >250,000/ml, clinical mastitis cases, or major pathogen cultures (p. 400).
* **Teatseal:** A non-antibiotic preparation used in New Zealand and the UK to physically plug the teat canal after the last milking, protecting against infection (p. 401).

📖 Sources:
- /content/agropustaka_id_buku_Bovine_Medicine_Diseases_and_Husbandry_of_Cattle.pdf
  ,
  "type": "RAG"
}
```

9.8.3 Example 3 – External Knowledge Query (Web Search Agent)

User Query: "How to prevent heat stress in cattle?"

```
{
  "answer": ""
  To prevent heat stress in cattle, here are some management practices:
  * Provide **shade**, which can reduce solar radiation by up to 50%. Around 40 square feet per animal is recommended.
  * Ensure adequate **ventilation** using natural or mechanical means to provide air exchange between inside and outside. Aim for 60 to 90 air exchanges per hour.
  * Provide **cooling**, such as fans combined with spraying water. Some specialists suggest wetting cows with 1.5 liters of water over 60 seconds, followed by 4 minutes of drying with a 10 km/hr airflow. Others suggest a spray cycle with 3.5 liters of water when the temperature exceeds 21°C.
  * Provide plenty of **drinking water** at all times for hydration.
  * **Limit heat production** by reducing activity and changing feeding patterns.
  * **Avoid working cattle** during temperature extremes, and if necessary, work in the early morning, use calm handling techniques, and provide water in holding pens.
  * **Alter paddock rotation** so that cattle are in paddocks close to the dairy to reduce walking distance in extreme heat.
  * Allow cattle to **access a dam or other source of cool water**.

  **In summary**, preventing heat stress in cattle involves providing shade, ventilation, and cooling, ensuring adequate hydration, limiting heat production, and modifying management practices to minimize stress during hot weather.

  **Sources:**
  - https://www.nadis.org.uk/disease-a-z/cattle/managing-heat-stress-in-dairy-cows/
  - https://www.ndsu.edu/agriculture/extension/publications/dealing-heat-stress-beef-cattle-operations
  - https://agriculture.vic.gov.au/livestock-and-animals/livestock-health-and-welfare/caring-for-animals-during-extreme-heat
  - https://extension.umn.edu/dairy-milking-cows/heat-stress-dairy-cattle
  ""
  "type": "WEB"
}
```

These outputs are consumed by the mobile and web interfaces, allowing users to receive contextual, well-structured, and role-specific responses in real time. in the mobile or web application interface, enabling users to receive context-aware results instantaneously.

9.9 Conclusion

The Chatbot System successfully integrates multiple intelligent reasoning agents into a unified framework. Through its modular architecture, it enables:

- Retrieval of knowledge from both structured and unstructured data.
- Real-time access to external web-based information.
- Seamless interaction through natural language.

By leveraging state-of-the-art language models and smart routing, the chatbot serves as a powerful decision-support tool within the broader smart farm management ecosystem. It not only simplifies access to information but also elevates the user experience by delivering precise, actionable, and well-referenced responses tailored to diverse user roles.

Chapter 10

MOBILE APPLICATION

10.1 Introduction

The **MooNet mobile application** represents the primary user interface layer within the smart livestock management system. It is designed to provide an intuitive, responsive, and role-specific experience for two main user categories: **Farm Managers** and **Veterinarians**. The mobile application serves as a bridge between users and the intelligent backend services, offering real-time interaction with farm data, AI-based disease detection, environmental monitoring, and personalized feed optimization.

Developed using the **Flutter framework**, the application is cross-platform and supports deployment on both Android and iOS devices. Its design emphasizes usability in field conditions, ensuring that both technical and non-technical users can interact with the system efficiently.

Through the integration of **AI-powered services**, the app supports the following core functionalities:

- **Health Monitoring:** Real-time alerts and detailed health reports for each cow, generated through AI-powered computer vision models and stored in a structured historical format.
- **Environmental Tracking:** Continuous monitoring of critical farm conditions such as temperature, humidity, gas concentration, and water levels.
- **Feed Optimization:** AI-generated feed recommendations and nutrition adjustment tools tailored to individual cow profiles.
- **Intelligent Assistance:** A built-in multilingual chatbot that provides guidance, explanations, and support for both veterinarians and managers.

- **Notifications and Alerts:** Real-time updates on livestock health conditions or abnormal environmental readings to ensure timely action.

The application architecture is modular and scalable, integrating seamlessly with backend APIs developed in **FastAPI**. This enables continuous data flow between IoT sensors, AI services, and user interactions, ensuring reliable performance in real-world agricultural environments.

Overall, the MooNet mobile application transforms traditional farm operations by combining modern software design with artificial intelligence, offering a comprehensive, accessible, and intelligent tool for digital livestock management.

10.2 User Scenario and Interface Design

We present the interaction between different users and the **MooNet** application. It illustrates the app's interface design process from initial sketches to final UI and describes realistic usage scenarios that reflect how users engage with the system to monitor livestock, manage feeding, and respond to environmental conditions.

10.2.1 Personas

The following personas represent the primary users of the **MooNet** system. Each persona was created based on user research to reflect real-world behaviors and needs.

• Farm Manager Persona

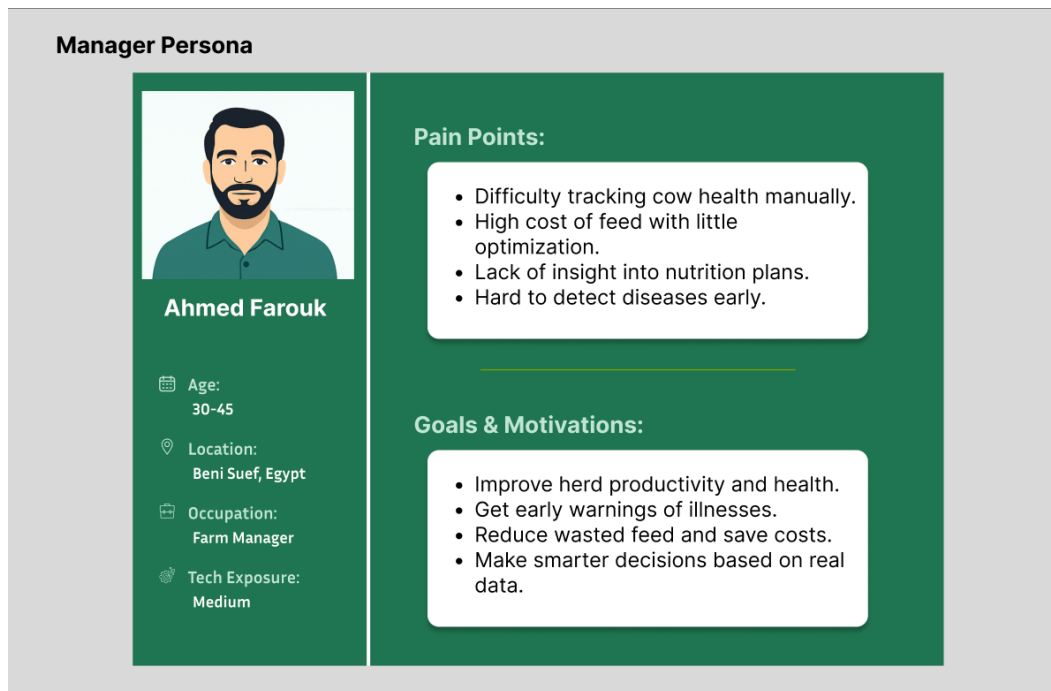


Figure 10.1: Farm Manager Persona

• Veterinarian Persona

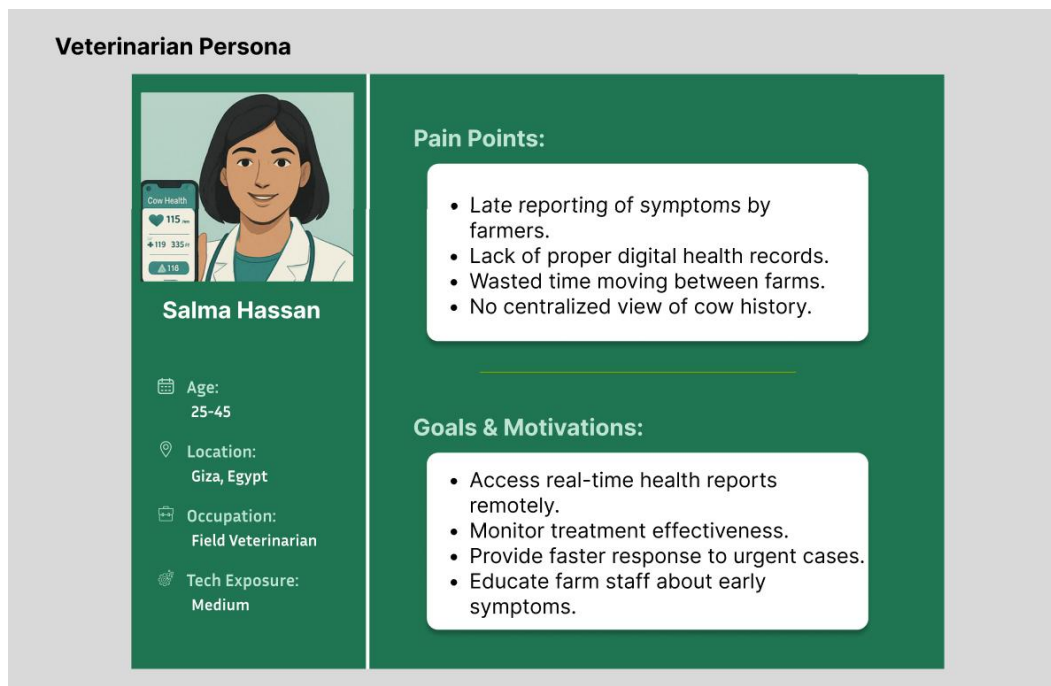


Figure 10.2: Veterinarian Persona

10.2.2 User Journey Maps

User Journey Maps visualize the steps taken by users within the application, helping to identify pain points and opportunities for improvement.

- **Farm Manager Journey Map**

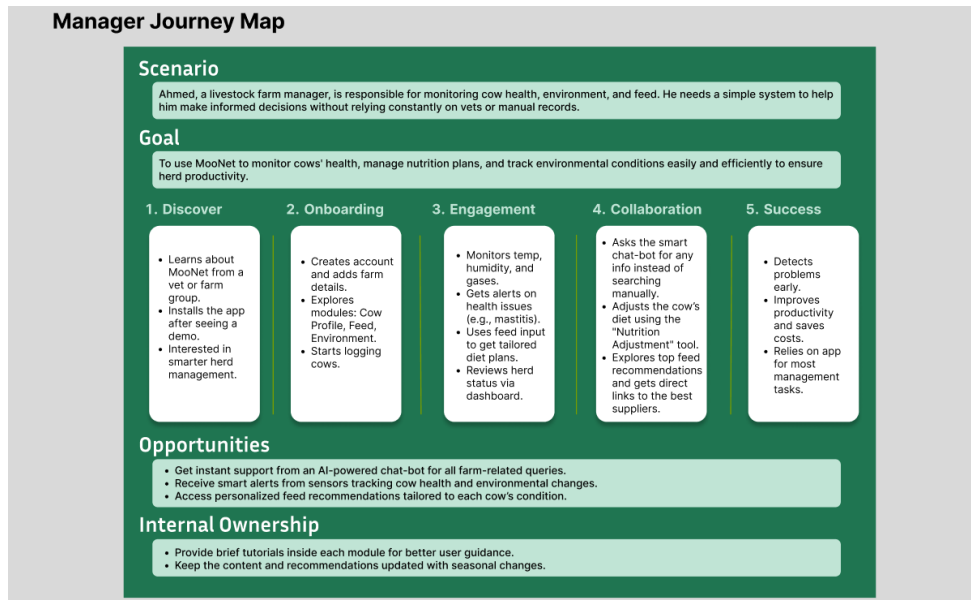


Figure 10.3: Farm Manager Journey Map

- **Veterinarian Journey Map**

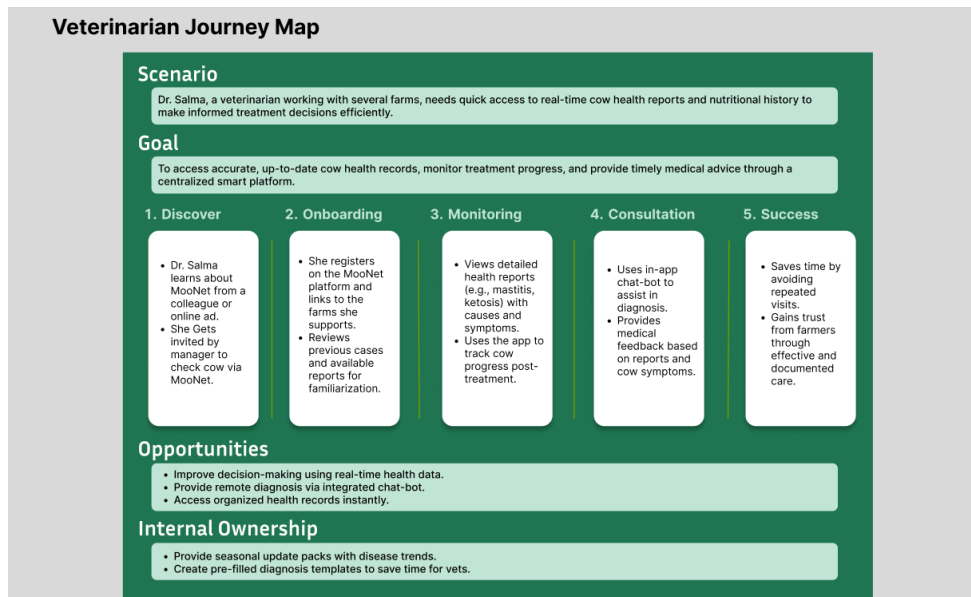


Figure 10.4: Veterinarian Journey Map

10.2.3 Initial Sketches

During the early stages of the design phase, several low-fidelity sketches were produced to explore different layout structures, user flows, and core interaction patterns. These initial hand-drawn sketches played a vital role in rapidly iterating ideas and validating functionality before committing to high-fidelity digital prototypes.

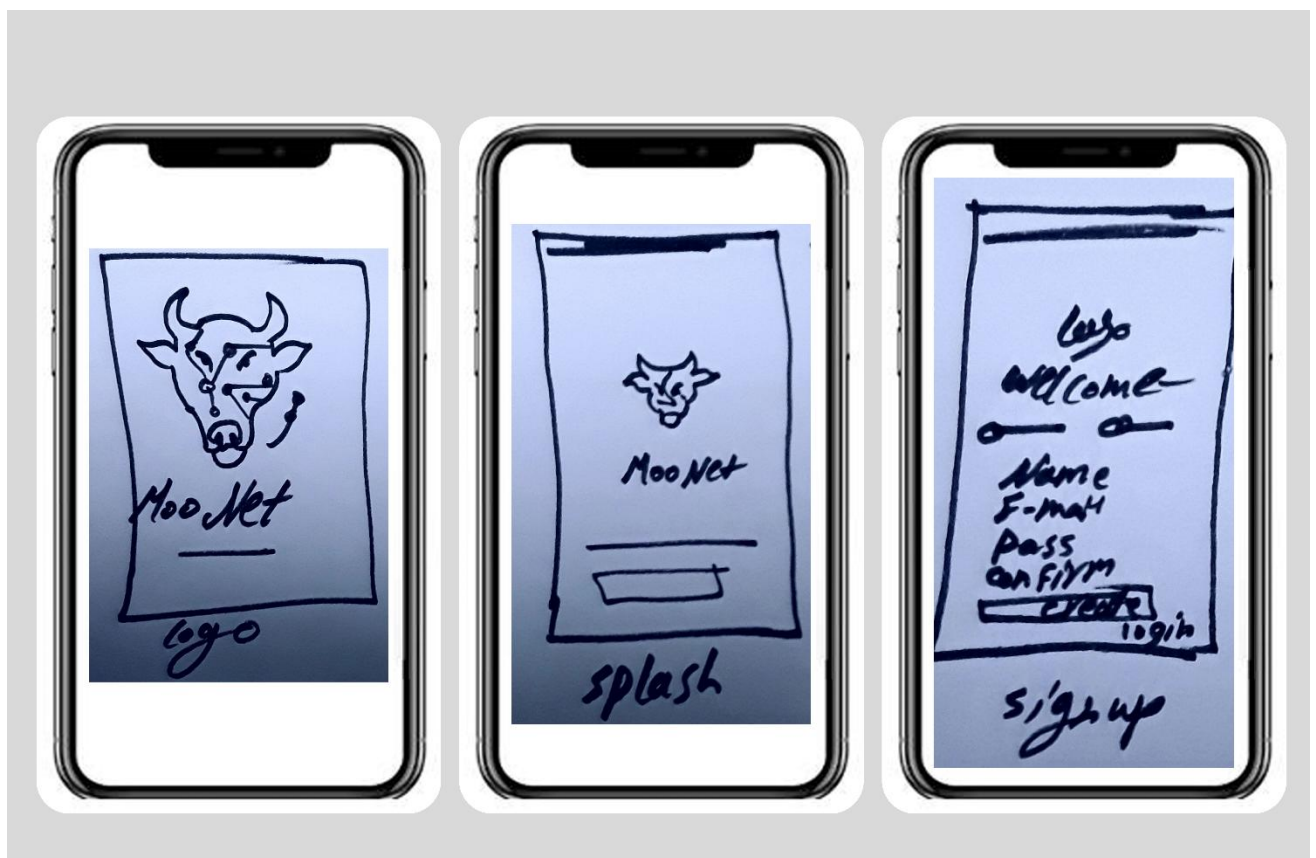


Figure 10.5: Account Registration and Authentication



Figure 10.6: Home Page & some features



Figure 10.7: Chat & Feed & Repots



Figure 10.8: Cow Health & Vet Home Page

10.2.4 Final UI Design

After completing the initial sketching phase and outlining core concepts on paper, we transitioned to designing the User Interface (UI) and User Experience (UX). During this stage, the rough sketches were transformed into detailed digital designs that accounted for both interaction flow and visual appeal. We carefully refined each component to ensure seamless and engaging user experience, with a strong emphasis on optimizing performance and maximizing the app's overall efficiency.

- **Manager Design:**

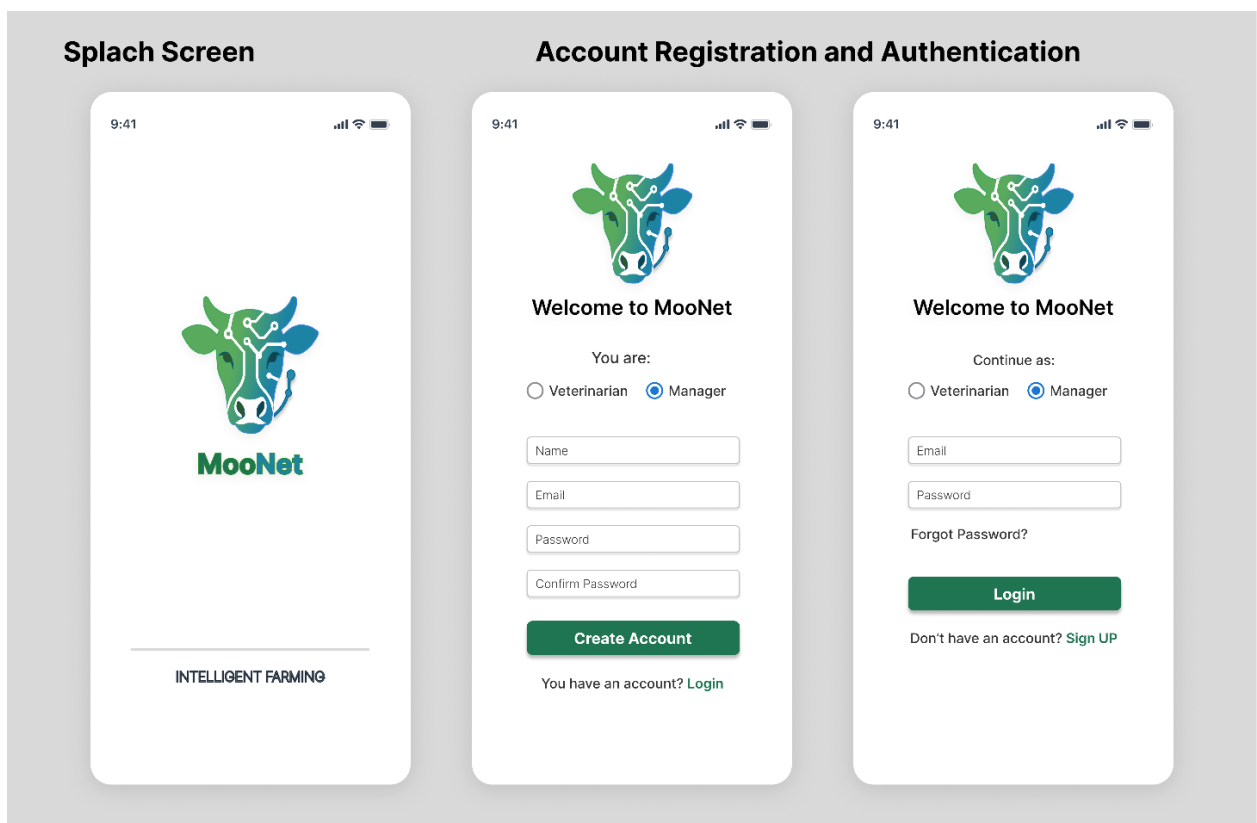


Figure 10.9: Manager Registration

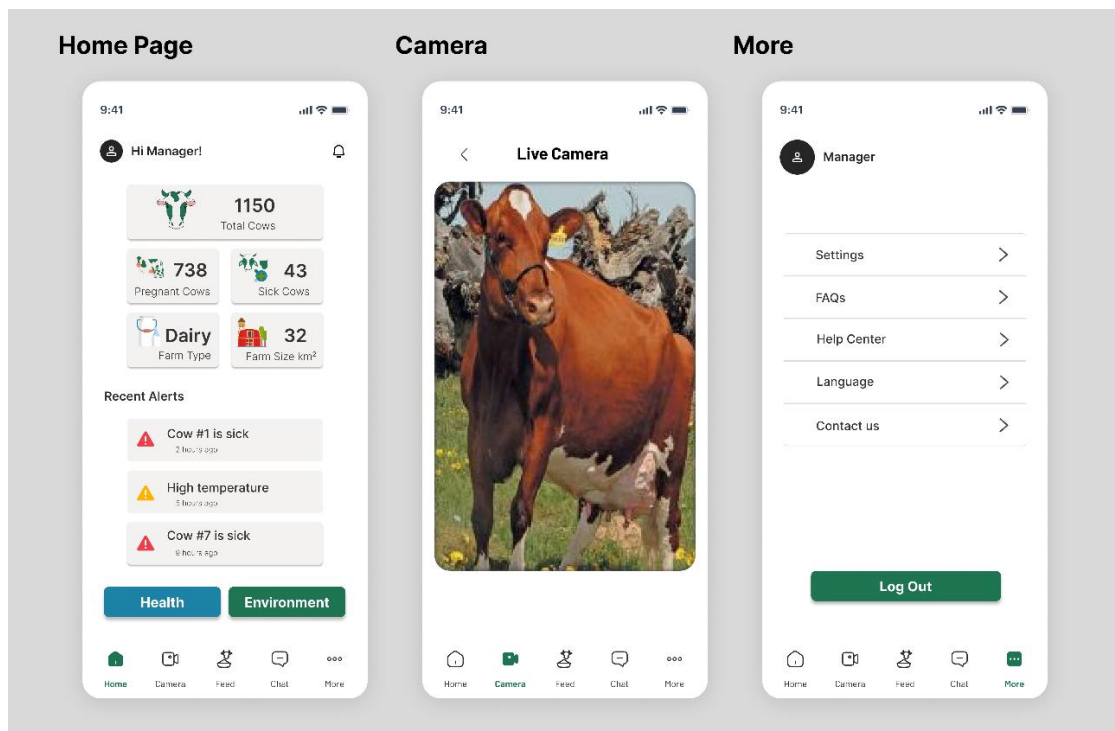


Figure 10.10: Manager Home Page & Cam & More

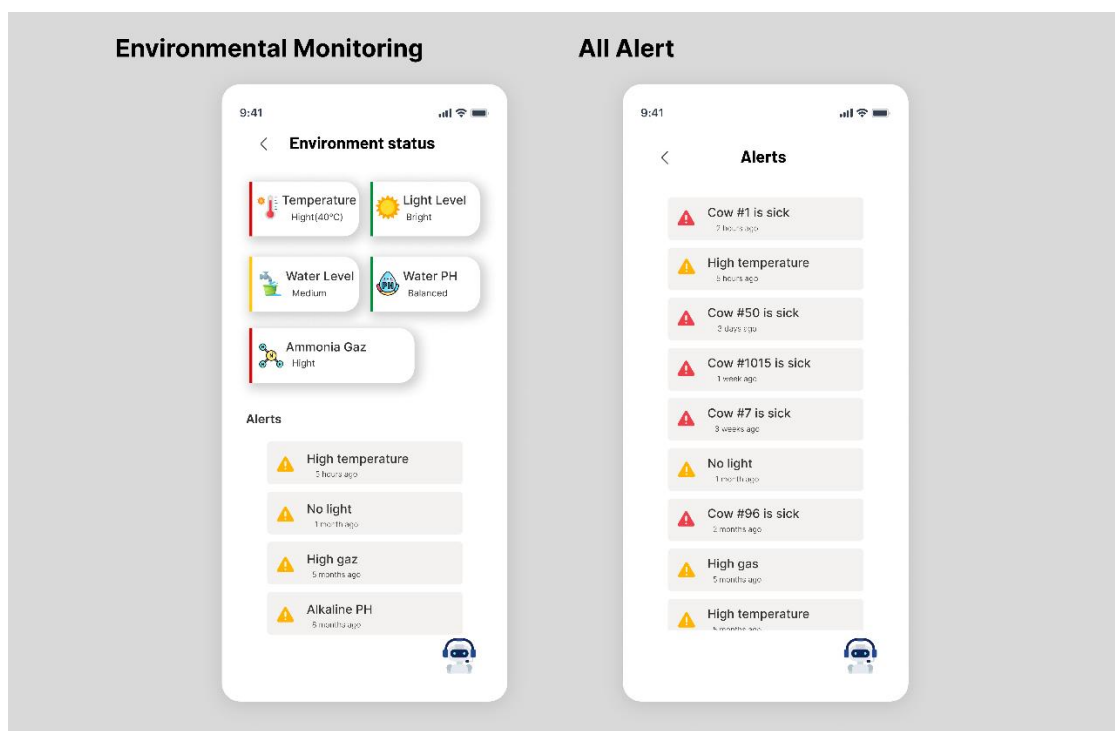


Figure 10.11: Env & Alerts

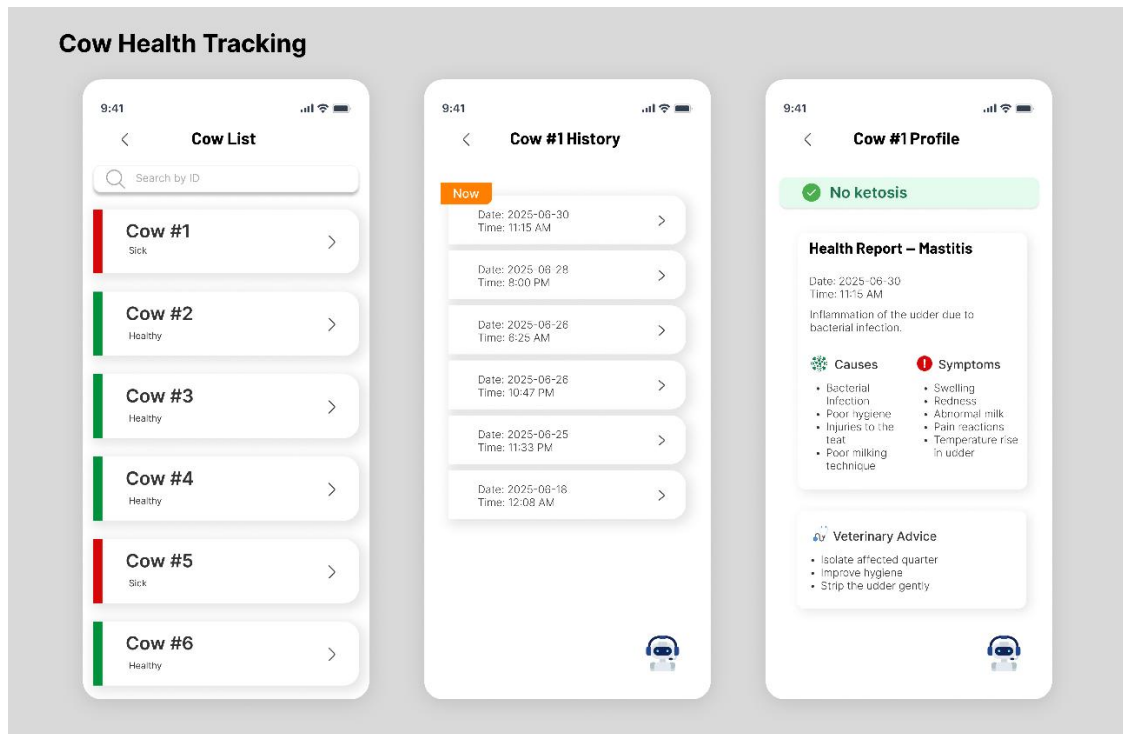


Figure 10.12: Manager Cow Health1

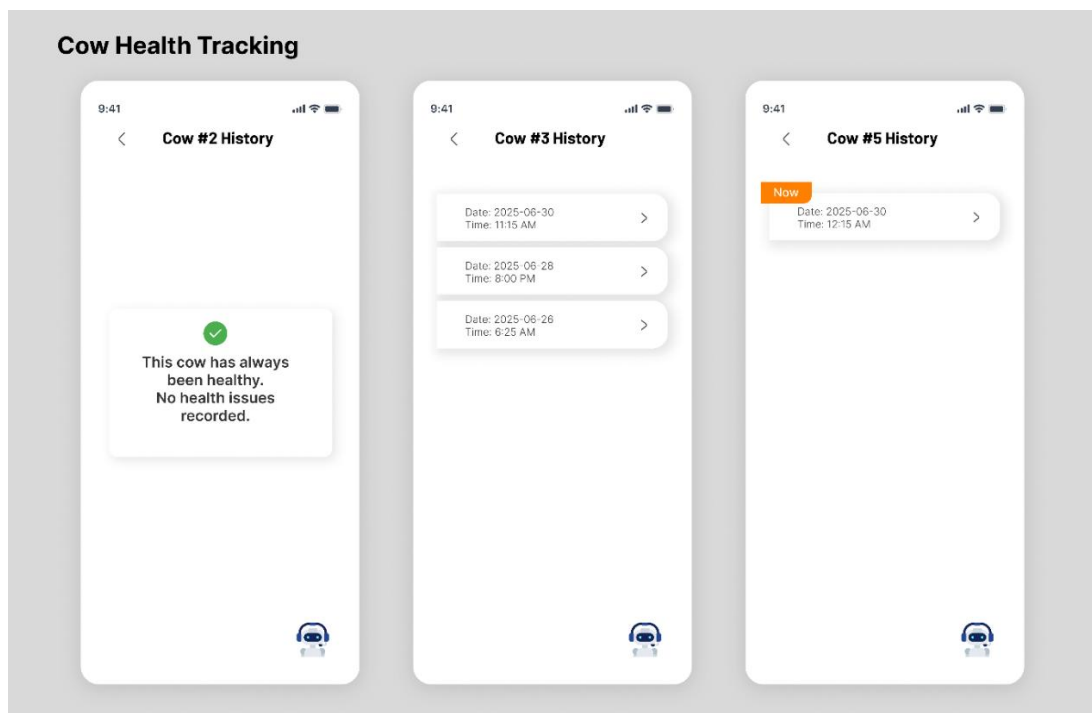
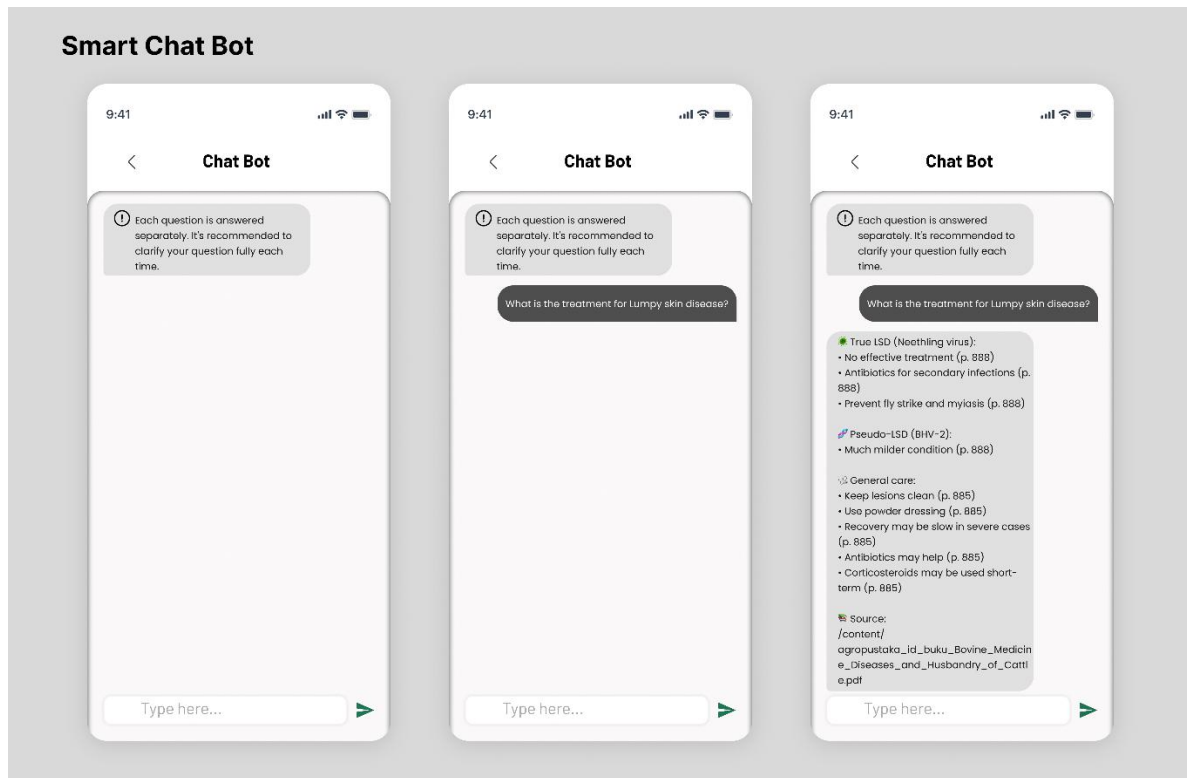


Figure 10.13: Manager Cow Health1

Feed Recommendation

Figure 10.14: Feed

**Figure 10.15: Manager Chat Bot**

- **Veterinarian Design:**

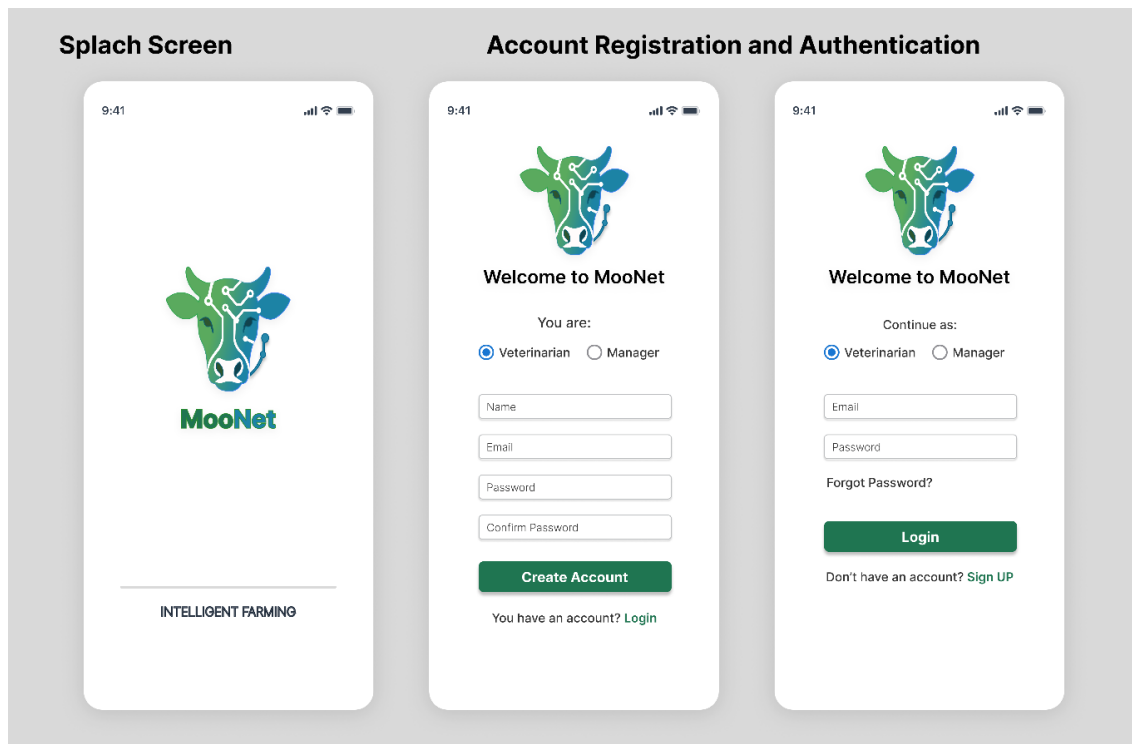


Figure 10.16: Vet Registration

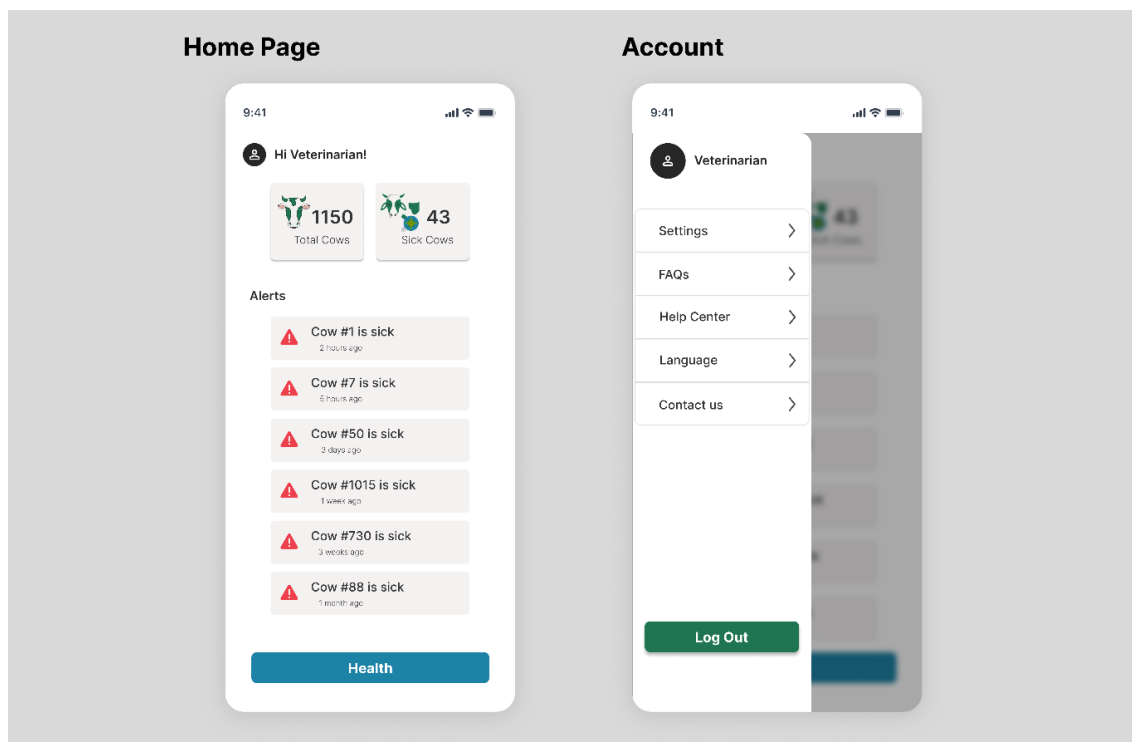


Figure 10.17: Vet Home Page

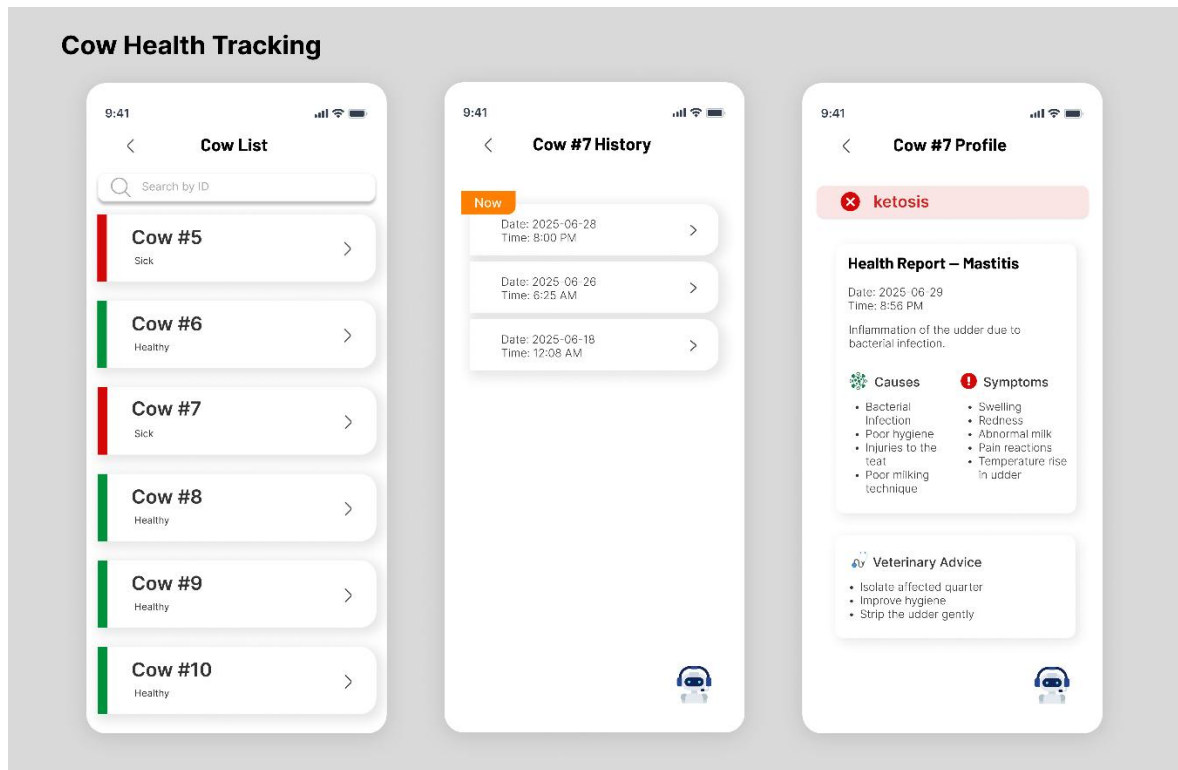


Figure 10.18: Vet Cow Health1

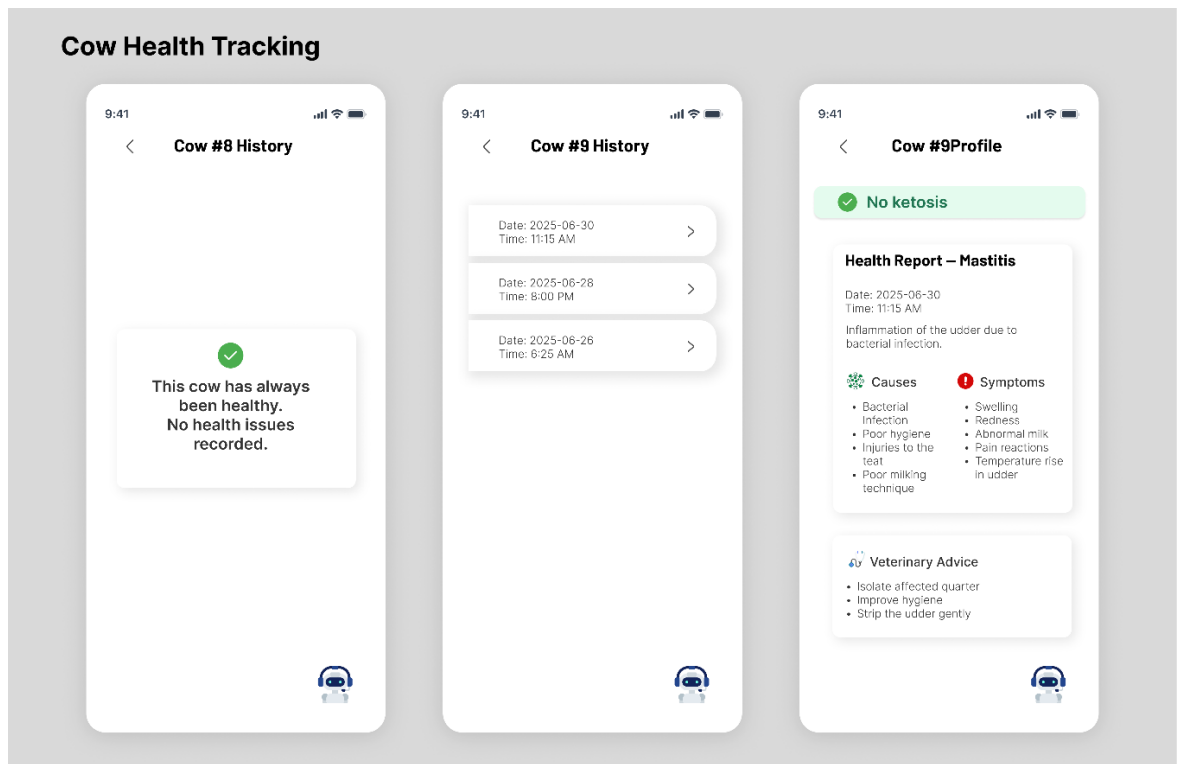


Figure 10.19: Vet Cow Health2

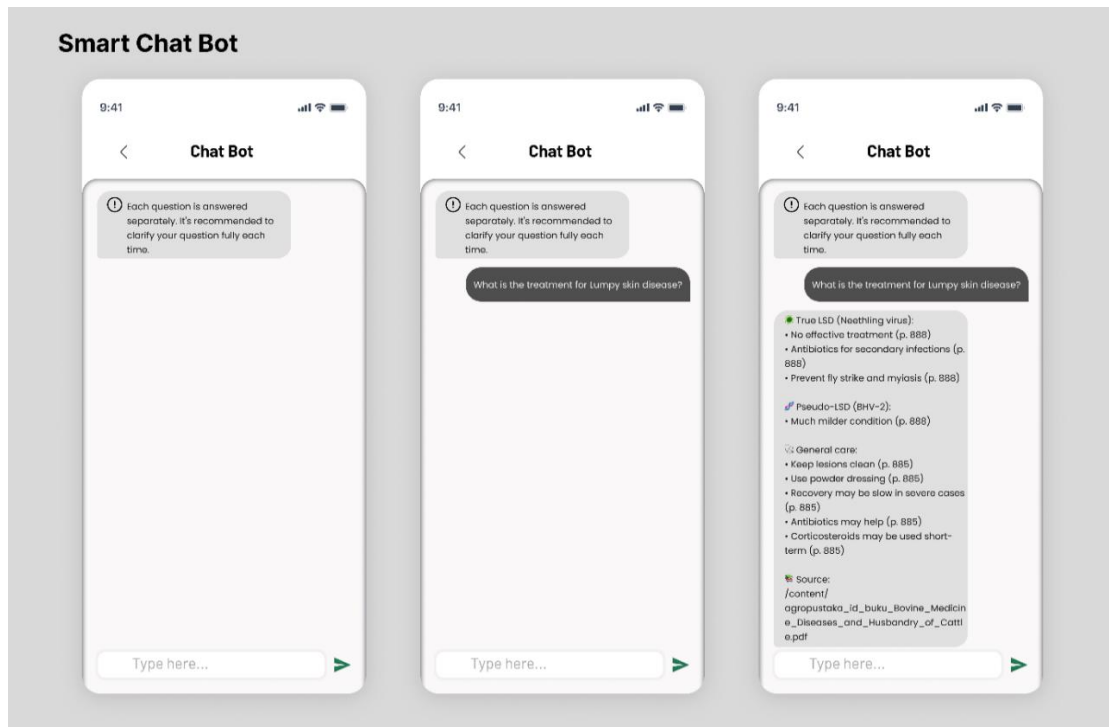


Figure 10.20: Vet Chat Bot

10.2.5 User Scenarios

10.2.5.1 Farm Manager Scenario

This section outlines the complete user journey of a Farm Manager using the **MooNet** application from account creation through to daily farm operations. It emphasizes functionality related to animal health tracking, environmental monitoring, and feed planning.

Account Registration and Authentication:

The Farm Manager begins by either creating a new account or logging in if they already have one.

For New Users:

The registration form requires:

- Full Name
- Email Address
- Password
- Password Confirmation

Once submitted and validated, the system checks email uniqueness. If the email is not registered, a new account is created, and the manager is redirected to the Home Page.

For Existing Users:

Users with an existing account can log in with their email and password. After authentication, the system redirects them to the Home Pa

Home Page Overview:

The Home Page acts as the main dashboard, consisting of three sections:

1. Key Performance Indicators (KPI Cards):

- Total Number of Cows
- Number of Pregnant Cows
- Number of Sick Cows
- Farm Type (Dairy or Beef)
- Farm Area (in hectares)

2. Recent Alerts Section:

- **Cow Alerts:** Clicking the ID card of a sick cow opens its detailed health report.
- **Environmental Alerts:** Clicking the card opens the Environmental Monitoring Page.

3. Action Buttons:

- **Health:** Navigates to the Cow List page.
- **Environment:** Navigates to the real-time Environmental Monitoring module.

A persistent **Bottom Navigation Bar** includes:

- Home (Returns to the main dashboard)
- Camera Access (Opens real-time camera feed for live farm surveillance)
- Feed Planning (Opens the feed input module to manage nutritional plans)
- Smart Chatbot (AI-powered assistant, answer questions and provide guidance)
- More (Settings, Help Center, FAQs, Language, Contact Us, Logout)

A **Notification Icon** provides access to all system alerts and historical notifications.

Cow Health Tracking:

Clicking the Health button directs the manager to the Cow List Page, which displays:

- Cow IDs
- A color-coded health indicator (Green: Healthy, Red: Sick)

Selecting a cow opens its **Health History Page**, showing past health reports in reverse chronological order.

If the cow has always been healthy, the message:

"This cow has always been healthy. No health issues recorded." is displayed.

Clicking on a specific report opens the **Detailed Health Report**, which contains:

- Report Date and Status
- Ketosis Detection Data
- Health Summary
- Veterinarian Notes

Every health page includes a **Chatbot Icon** that connects to the AI assistant for contextual help.

Environmental Monitoring Module:

Clicking the Environment button leads to real-time readings from the following sensors:

- **Temperature & Humidity** – DHT11
- **Light Intensity** – LDR Sensor
- **Ammonia & Gas Levels** – MQ137, MQ138
- **PH Level** – For water analysis
- **Water Level** – For availability tracking

Readings are visualized with:

- Color-coded indicators (green, orange, red)
- Sensor-specific icons
- Live-updating graphs

The system automatically generates alerts when conditions are outside optimal thresholds. The **Chatbot** can be used to interpret sensor data and receive expert recommendations.

Feed Recommendation Module:

In the Feed module, the manager provides cow-specific information:

- Cow Type (Dairy or Beef)
- Age (years)
- Weight (kg)
- Breed (Holstein, Jersey, Local Saidi, Brown Swiss)
- Production Goal (Milk or Meat)
- Milk Yield (liters/day)
- Pregnancy Status (Yes/No)
- Activity Level (Low/Medium/High)
- Health Status (Healthy/Sick)
- Season (Summer, Winter, Spring, Fall)
- Current Feed Used (free text)

Upon submission, the system generates three reports:

1. Cow Nutrition Guide

A complete feeding plan based on the provided cow profile. The plan includes roughage, concentrate, supplements, water needs, notes, and sources.

2. Nutrition Adjustment

An analytical comparison between the current feed and the ideal nutritional requirements. Highlights missing or excessive nutrients and recommends an improved plan with precise amounts and guidance.

3. Top Feed Recommender

A market-based comparison that suggests the top commercially available feed options. Available commercial feed products, nutritional data, pricing, and links to suppliers.

Each report appears on a dedicated screen with clear formatting for easy interpretation and application.

Smart Chatbot Integration:

The Smart Chatbot is available throughout the app via the navigation bar. It supports the

manager with:

- Answering farm-related queries
- Interpreting reports and sensor data
- Providing expert nutritional or environmental advice
- Guiding users through application features

This assistant ensures users receive immediate, reliable, and helpful support during any stage of the farming process.

10.2.5.2 Veterinarian Scenario

This section describes how a Veterinarian uses **MooNet** to monitor animal health, review alerts, and evaluate medical reports for diagnosis.

Account Registration and Authentication:

The Veterinarian begins by either creating a new account or logging in if they already have one.

For New Users:

The registration form requires:

- Full Name
- Email Address
- Password
- Password Confirmation

Once submitted and validated, the system checks email uniqueness. If the email is not registered, a new account is created, and the Veterinarian is redirected to the Home Page.

For Existing Users:

Users with an existing account can log in with their email and password. After authentication, the system redirects them to the Home Page.

Home Page Overview:

The Home Page acts as the main dashboard, consisting of three sections:

1. Key Performance Indicators (KPI Cards):

- Total Number of Cows
- Number of Sick Cows

2. Alerts Section:

- It contains all Alerts about cows; Clicking the ID card of a sick cow opens its detailed health report.

3. Action Buttons:

- **Health:** Navigates to the Cow List page.

An **Account Icon** provides access to Settings, Help Center, FAQs, Language, Contact Us and Logout.

Cow Health Tracking:

Clicking the Health button directs the manager to the Cow List Page, which displays:

- Cow IDs
- A color-coded health indicator (Green: Healthy, Red: Sick)

Selecting a cow opens its **Health History Page**, showing past health reports in reverse chronological order.

If the cow has always been healthy, the message:

"This cow has always been healthy. No health issues recorded."
is displayed.

Clicking on a specific report opens the **Detailed Health Report**, which contains:

- Report Date and Status
- Ketosis Detection Data
- Health Summary
- Veterinarian Notes

The veterinarian uses this to evaluate the cow's condition.

Smart Chatbot Support:

The AI chatbot is accessible at all times in all pages. The veterinarian can:

- Ask for symptom definitions
- Get help interpreting report data
- Receive nutritional or environmental suggestions
- Ask usage questions or get navigation help

Chatbot enhances decision-making and supports efficient medical evaluation.

10.2.6 UI / UX Prototype

To facilitate user access and encourage engagement, a QR code was generated for seamless application access.



10.3 Backend Implementation

10.3.1 Introduction

The backend of the mobile application serves as the central hub that connects the frontend interface with all AI-powered services in MooNet. It is developed using **FastAPI**, a modern web framework known for its performance and easy integration with data validation tools like **Pydantic** and object-relational mapping libraries such as **SQLAlchemy**.

This backend manages essential operations such as:

- User authentication and authorization
- Disease detection and cow health tracking
- Historical report management
- Smart chatbot responses for veterinarians and farmers
- AI-powered feed formulation and optimization
- Real-time notifications and homepage analytics

The system is built using a modular, scalable architecture that supports RESTful APIs for seamless integration with the mobile frontend.

10.3.2 Database Models (SQLAlchemy ORM)

The backend leverages **SQLAlchemy ORM** to define the database schema using Python classes. This allows clean interaction with PostgreSQL, hiding SQL complexity behind well-structured model definitions.

10.3.2.1 User Model

Code Snippet:

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    email = Column(String, unique=True, index=True, nullable=False)
    password = Column(String, nullable=False)
    is_manager = Column(Boolean, default=False)
```

Explanation:

This model defines user credentials and roles. Each user has a unique email and can be optionally marked as a **farm manager**, which influences access permissions on the mobile app.

10.3.2.2 Cow Model

Code Snippet:

```
class Cow(Base):
    __tablename__ = "cows"

    id = Column(Integer, primary_key=True, index=True)
    report = Column(JSON, nullable=True)
    date = Column(Date, nullable=True)
    time = Column(Time, nullable=True)
    is_sick = Column(Boolean, default=False)
```

Explanation:

This model tracks basic health status for each cow, including a reference to the most recent health report. If `is_sick` is set to `True`, the app will trigger a notification to the farmer or vet.

10.3.2.3 Cow Report Model

Code Snippet:

```
class CowReport(Base):
    __tablename__ = "cow_reports"

    id = Column(Integer, primary_key=True, index=True) # Global unique
    cow_id = Column(Integer, ForeignKey("cows.id"))
    per_cow_report_id = Column(Integer) # Unique per cow
    report_json = Column(JSON, nullable=False)
    date = Column(Date, nullable=False)
    time = Column(Time, nullable=False)

    cow = relationship("Cow", backref="reports")

    __table_args__ = (
        # Ensure each cow has unique per_cow_report_id
        UniqueConstraint("cow_id", "per_cow_report_id", name="unique_cow_report"),
    )
```

Explanation:

This model supports storing multiple disease reports for each cow, including historical tracking. Each report is uniquely identified **per cow** using `per_cow_report_id`. The `report_json` field stores detailed AI-generated diagnosis data.

10.3.3 API Schemas (Pydantic Models)

Pydantic models define the shape of request and response payloads for all FastAPI endpoints. They offer built-in type validation and error messaging, ensuring safe communication between the frontend and backend.

10.3.3.1 User Schemas

Code Snippet:

```
class UserBase(BaseModel):
    email: EmailStr
    is_manager: bool = False
class UserCreate(UserBase):
    name: str = Field(..., min_length=2)
    password: str = Field(..., min_length=6)
    confirm_password: str = Field(..., min_length=6)
class UserLogin(BaseModel):
    email: EmailStr
    password: str
class UserResponse(BaseModel):
    is_manager: bool
```

Explanation:

These models support the registration and login process. Validation rules (e.g., password length) are enforced automatically. The login response includes a role flag to personalize the user experience.

10.3.3.2 Cow & Health Report Schemas

Code Snippet:

```
class CowSummary(BaseModel):
    cow_id: str
    is_sick: bool
class CowReportMetaResponse(BaseModel):
    report_id: int
    date: date
    time: time
class CowReportDetailResponse(BaseModel):
    report_id: int
    cow_id: int
    date: date
    time: time
    disease_name: str
    description: str
    causes: List[str]
    symptoms: List[str]
    is_contagious: str
    severity: str
    farmer_advice: str
    needs_vet_attention: str
class CowDetailResponse(BaseModel):
    cow_id: str
    is_sick: bool
    report: Optional[CowReportDetailResponse]
```

Explanation:

- CowSummary is used to populate lists with cow status on the homepage.
- CowReportMetaResponse allows for efficient listing of report history.
- CowReportDetailResponse presents full diagnostic results for any specific report.
- CowDetailResponse merges status and latest report into a single API response.

10.3.3.3 Notification and Statistics

Code Snippet:

```
class NotificationResponse(BaseModel):
    cow_id: str
    since: str
```

```
class HomePageStats(BaseModel):
    total_cows: int
    sick_cows: int
    recent_notifications: List[NotificationResponse]
```

Explanation:

These schemas support **real-time notifications** and **dashboard summaries**. They show how many cows are sick and when the issue was first detected in human-readable terms like "3 hours ago."

10.3.3.4 Chatbot Interface

Code Snippet:

```
class ChatRequest(BaseModel):
    question: str
class ChatResponse(BaseModel):
    answer: str
    type: str
```

Explanation:

These schemas define the interface between the mobile app and the AI-powered **smart assistant**. The type field indicates which reasoning agent (e.g., RAG, WEB, Agent) was used.

10.3.3.5 Feed Optimization Models

Code Snippet:

```
class CowProfileRequest(BaseModel):
    type : Literal["dairy", "beef"] = Field(..., alias="type")
    age: int = Field(..., ge=0, le=20)
    weight: int = Field(..., ge=200, le=1000)
    goal: Literal["milk", "meat"]
    milk_yield: float = Field(0, ge=0)
    pregnant: bool = False
    breed: Optional[str] = None
    feed_used: Optional[List[str]] = Field(None, alias="current_feed")
    activity_level: Optional[Literal["low", "medium", "high"]] = "low"
    health_status: Optional[Literal["good", "fair", "poor"]] = "good"
    season: Optional[Literal["spring", "summer", "autumn", "winter"]] = "winter"
    animal_type: str = "البقرة"
    production_goal: str = "اللحوم إنتاج زيادة"
    country_name: str = "Egypt"
    product_name: str = "livestock nutrition"
    no_keywords: int = 10
    language: str = "Arabic"
    score_th: float = 0.7

class Config:
    populate_by_name = True
```

```

class IdealFeedFormulation(BaseModel):
    roughage: FeedComponent
    concentrate: FeedComponent
    supplements: List[FeedComponent]
    water: WaterComponent
    notes: str

class FeedComparisonReport(BaseModel):
    balanced: bool
    missing_nutrients: List[str]
    surplus_nutrients: List[str]
    comparison_notes: str
    improved_feed_recommendation: IdealFeedFormulation

class FeedProductResult(BaseModel):
    product_name: str
    protein_percent: float
    price_per_kg: float
    key_ingredients: List[str]
    matches_ideal_formula: bool
    match_reason: str
    supplier_rating: Optional[float] = Field(None, ge=0, le=5)
    url: str

class BestFeedRecommendationOutput(BaseModel):
    top_3_products: List[FeedProductResult]
    best_product: FeedProductResult
    recommendation_reason: str

class IdealFeedFormulationResponse(IdealFeedFormulation):
    generated_date: str
    generated_time: str

class FeedComparisonReportResponse(FeedComparisonReport):
    generated_date: str
    generated_time: str

class BestFeedRecommendationOutputResponse(BestFeedRecommendationOutput):
    generated_date: str
    generated_time: str

```

Explanation:

These schemas are used in AI-powered endpoints that analyze a cow's profile and return:

- A balanced feed formulation
- A detailed feed comparison report
- A product recommendation for optimal feeding.

10.3.4 Real-Time Camera Integration and Health Analysis

The system integrates a **simulated camera loop** that captures RGB and depth images of cows at regular intervals. These frames are analyzed by disease detection models (Lumpy Skin Disease and Mastitis), and results are stored in the database. This loop enables **automated health surveillance** of the herd.

10.3.4.1 Camera Simulation and AI Inference Loop

Code Snippet:

```

async def camera_loop():
    while True:
        db: Session = None
        try:
            rgb, depth, timestamp, image_path = simulate_camera_capture("./disease_detection/images")
            cow_id = f"{random.randint(1, 10):02}"

            latest_frame["rgb_image"] = encode_array_to_base64(rgb)
            latest_frame["depth_map"] = encode_array_to_base64(depth, is_depth=True)
            latest_frame["timestamp"] = timestamp
            latest_frame["cow_id"] = cow_id

            lumpy_result, lumpy_description = infer_abdominal_lumpy(image_path)
            mastitis_result, mastitis_description = infer_abdominal_mastitis(image_path)

            lumpy_detected = (
                lumpy_result.get("Predicted Label") == "Lumpy Skin"
                and float(lumpy_result.get("Confidence", "0").replace("%", "")) >= 50.0
            )
            mastitis_detected = (
                mastitis_result.get("Predicted Label") == "Infected"
                and float(mastitis_result.get("Confidence", "0").replace("%", "")) >= 50.0
            )

            db = next(get_db())

```

Explanation:

This loop captures a simulated RGB and depth image, then runs two deep learning models for disease classification. Results are updated to an in-memory dictionary (latest_frame) for quick access via the frontend.

10.3.4.2 Health Status Update and Report Storage

Code Snippet:

```

if lumpy_detected or mastitis_detected:
    # Identify the disease and retrieve its metadata
    disease = "Lumpy Skin Disease" if lumpy_detected else "Mastitis"
    description = lumpy_description if lumpy_detected else mastitis_description
    result = lumpy_result if lumpy_detected else mastitis_result
    dt = datetime.fromisoformat(timestamp)

    # Fetch recent reports for this cow
    recent_reports = db.query(CowReport)\
        .filter(CowReport.cow_id == int(cow_id))\
        .order_by(CowReport.date.desc(), CowReport.time.desc())\
        .all()

    # Find most recent report for the same disease in Python
    last_report = next((r for r in recent_reports if r.report_json.get("disease_name") == disease), None)

    report_already_exists = False

```

```

if last_report:
    last_report_dt = datetime.combine(last_report.date, last_report.time)
    threshold = DISEASE_REPORT_INTERVALS.get(disease, timedelta(hours=24))
    if (dt - last_report_dt) < threshold:
        report_already_exists = True

if not report_already_exists:
    # Generate a new disease report and store it in the database
    report = generate_disease_report(description, result, cow_id, timestamp, disease)

    existing_reports = db.query(CowReport).filter(CowReport.cow_id == int(cow_id)).count()
    next_per_cow_report_id = existing_reports + 1

    cow_report = CowReport(
        cow_id=int(cow_id),
        per_cow_report_id=next_per_cow_report_id,
        report_json=report,
        date=dt.date(),
        time=dt.time()
    )
    db.add(cow_report)

    # Update or create the cow's status in the main cow table
    existing = db.query(Cow).filter(Cow.id == int(cow_id)).first()
    if existing:
        existing.date = dt.date()
        existing.time = dt.time()
        existing.is_sick = True
    else:
        cow = Cow(id=int(cow_id), date=dt.date(), time=dt.time(), is_sick=True)
        db.add(cow)
    else:
        # Skip report generation if a recent report already exists for the same disease
        print(f"[SKIPPED] 🐮 Cow {cow_id} already diagnosed with '{disease}' recently. Skipping duplicate report.")

    else:
        # If the cow is no longer sick, update its status accordingly
        existing = db.query(Cow).filter(Cow.id == int(cow_id)).first()
        if existing and existing.is_sick:
            existing.date = None
            existing.time = None
            existing.is_sick = False
            print(f"[INFO] Cow {cow_id} is now healthy.")
        elif not existing:
            cow = Cow(id=int(cow_id), date=None, time=None, is_sick=False)
            db.add(cow)

db.commit()

```

Explanation:

This code governs how the system manages disease records and cow health status updates in real-time. Once a disease is confidently detected through the image inference pipeline (i.e., either Lumpy Skin Disease or Mastitis), the backend performs several steps to ensure data consistency and prevent redundant reporting.

1. Disease Identification:

Based on the detection result, the system determines which disease is active and retrieves the corresponding diagnostic metadata and timestamp.

2. Report Frequency Check:

To avoid generating duplicate reports for the same disease, the system looks up the cow's previous reports from the `cow_reports` table. It verifies whether a report for the same disease already exists within a predefined time threshold (e.g., 24 hours for Mastitis or 72 hours for Lumpy Skin Disease). If such a report exists, the system skips report generation.

3. Report Creation and Storage:

If no recent duplicate exists, a new JSON-structured report is created using `generate_disease_report()`. It is then added to the `cow_reports` table with a unique per-cow report ID. This ensures historical traceability of all detected cases.

4. Health Status Synchronization:

Simultaneously, the `cows` table is updated to reflect the cow's current health status and the timestamp of diagnosis. If the cow does not exist in the table yet, it is added.

5. Recovery Handling:

If no disease is currently detected, the system checks whether the cow is still marked as sick. If so, it clears the diagnosis date, time, and health flag, marking the animal as recovered.

By enforcing these steps, the backend guarantees medical accuracy, prevents redundant records, and maintains a clean and traceable health history for each individual cow.

10.3.4.3 Start Camera Loop on Application Launch

Code Snippet:

```
@app.on_event("startup")
async def startup_event():
    asyncio.create_task(camera_loop())
```

Explanation:

When the FastAPI server starts, the asynchronous camera loop begins execution, allowing continuous and non-blocking image analysis.

10.3.5 Cow Management and Notification Endpoints

These endpoints provide an interface to manage and retrieve information about cow health, reports, and real-time alerts.

10.3.5.1 Retrieve Latest Camera Frame

Code Snippet:

```
@app.get("/camera/latest")
def get_latest_capture():
    if not latest_frame:
        return JSONResponse(status_code=404, content={"error": "No capture yet"})
    return JSONResponse(content=latest_frame)
```

Explanation:

Returns the most recent RGB and depth images captured, along with metadata like timestamp and cow ID. The frontend uses this for live monitoring.

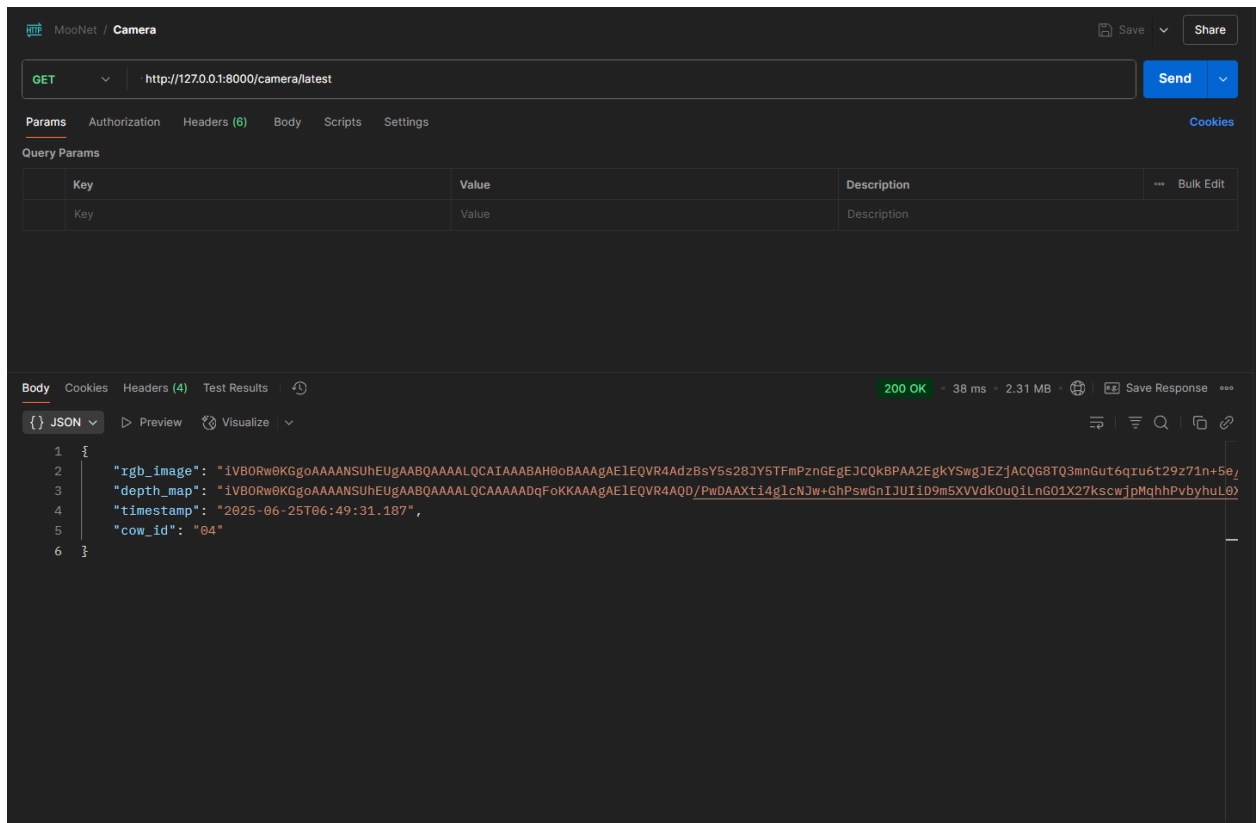


Figure 10.21: Retrieve Latest Camera Frame Endpoint

10.3.5.2 Get Sick Cow Notifications

Code Snippet:

```
@app.get("/notification", response_model=List[NotificationResponse])
def get_notifications(db: Session = Depends(get_db)):
    sick_cows = db.query(Cow).filter(Cow.is_sick == True)\
        .order_by(Cow.date.desc(), Cow.time.desc()).all()

    notifications = []
    for cow in sick_cows:
        timestamp = datetime.combine(cow.date, cow.time)
        human_readable = NotificationResponse.get_human_readable_duration(timestamp)
        notifications.append(NotificationResponse(cow_id=str(cow.id), since=human_readable))

    return notifications
```

Explanation:

This endpoint fetches all currently sick cows and returns a human-readable timestamp indicating how long ago each was detected.

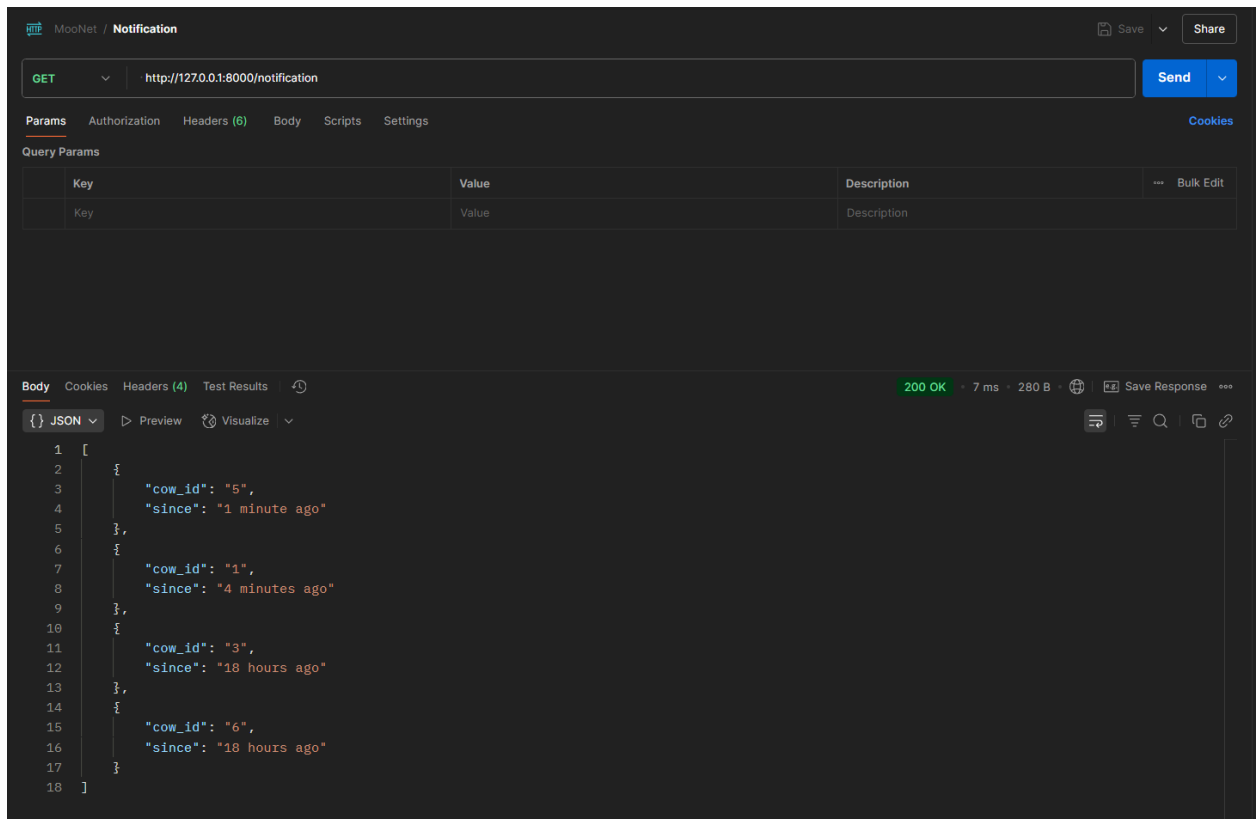


Figure 10.22: Get Sick Cow Notifications Endpoint

10.3.5.3 Cow Status and Reports

Code Snippet:

```

@app.get("/cows", response_model=List[CowSummary])
def get_all_cows(db: Session = Depends(get_db)):
    cows = db.query(Cow).all()
    return [{"cow_id": str(c.id), "is_sick": c.is_sick} for c in cows]

```

Explanation:

Returns a list of all cows and their sickness status. Used for displaying cow cards on the app homepage.

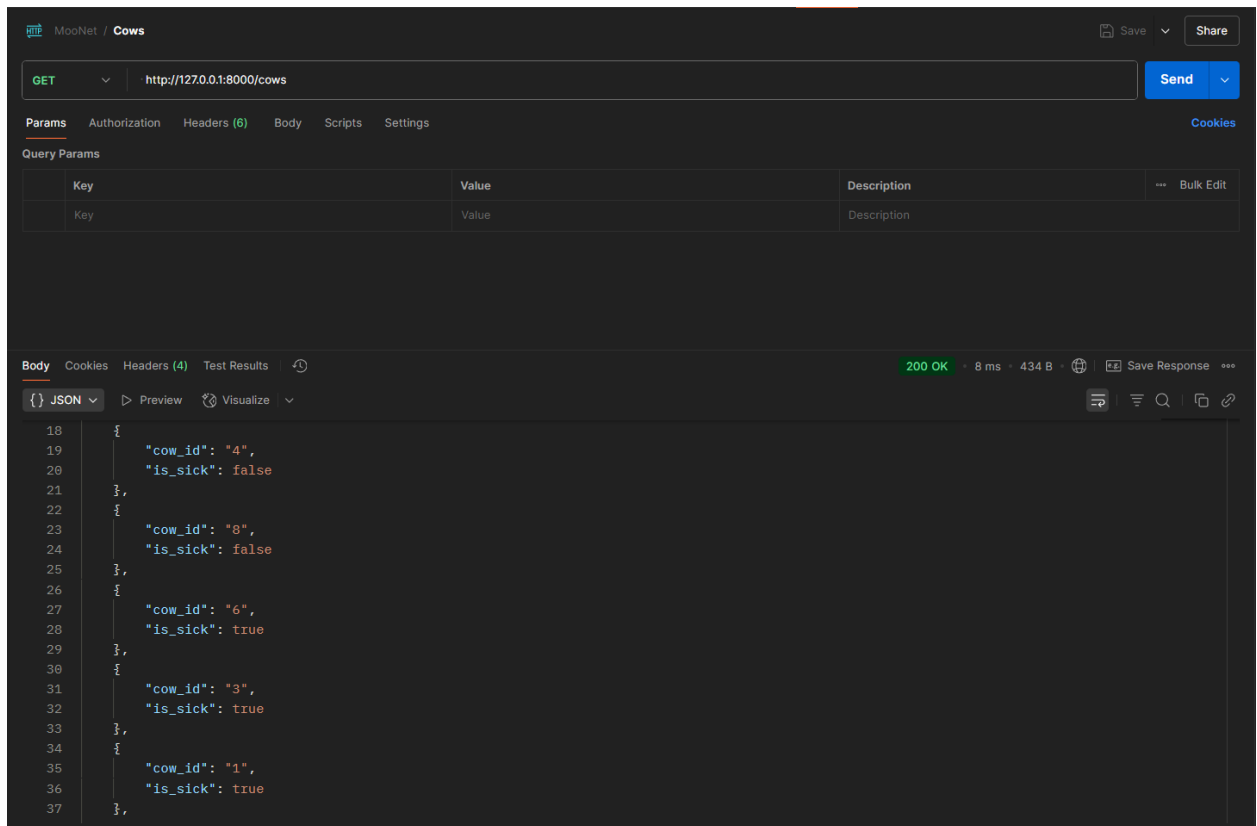


Figure 10.23: Cow Status Endpoint

10.3.5.4 Fetch Cow Report History and Details

Code Snippet:

```

@app.get("/cows/{cow_id}/reports", response_model=List[CowReportMetaResponse])
def get_cow_reports(cow_id: int, db: Session = Depends(get_db)):
    reports = db.query(CowReport).filter(CowReport.cow_id == cow_id)\
        .order_by(CowReport.date.desc(), CowReport.time.desc()).all()

    if not reports:
        raise HTTPException(status_code=404, detail="No reports found for this cow")

    return [{"report_id": r.id, "date": r.date, "time": r.time} for r in reports]

@app.get("/cows/{cow_id}/reports/{report_id}", response_model=CowReportDetailResponse)
def get_specific_cow_report(cow_id: int, report_id: int, db: Session = Depends(get_db)):
    report = db.query(CowReport).filter(
        CowReport.cow_id == cow_id,
        CowReport.per_cow_report_id == report_id
    ).first()

    if not report:
        raise HTTPException(status_code=404, detail="Report not found")

    return CowReportDetailResponse(
        report_id=report.per_cow_report_id,
        cow_id=report.cow_id,
        date=report.date,
        time=report.time,
        disease_name=report.report_json.get("disease name"),

```

```

description=report.report_json.get("description"),
causes=report.report_json.get("causes"),
symptoms=report.report_json.get("symptoms"),
is_contagious=report.report_json.get("is_contagious"),
severity=report.report_json.get("severity"),
farmer_advice=report.report_json.get("farmer_advice"),
needs_vet_attention=report.report_json.get("needs_vet_attention")
)

```

Explanation:

These endpoints enable report browsing per cow. The first lists timestamps and report IDs; the second retrieves full report content for any specific entry.

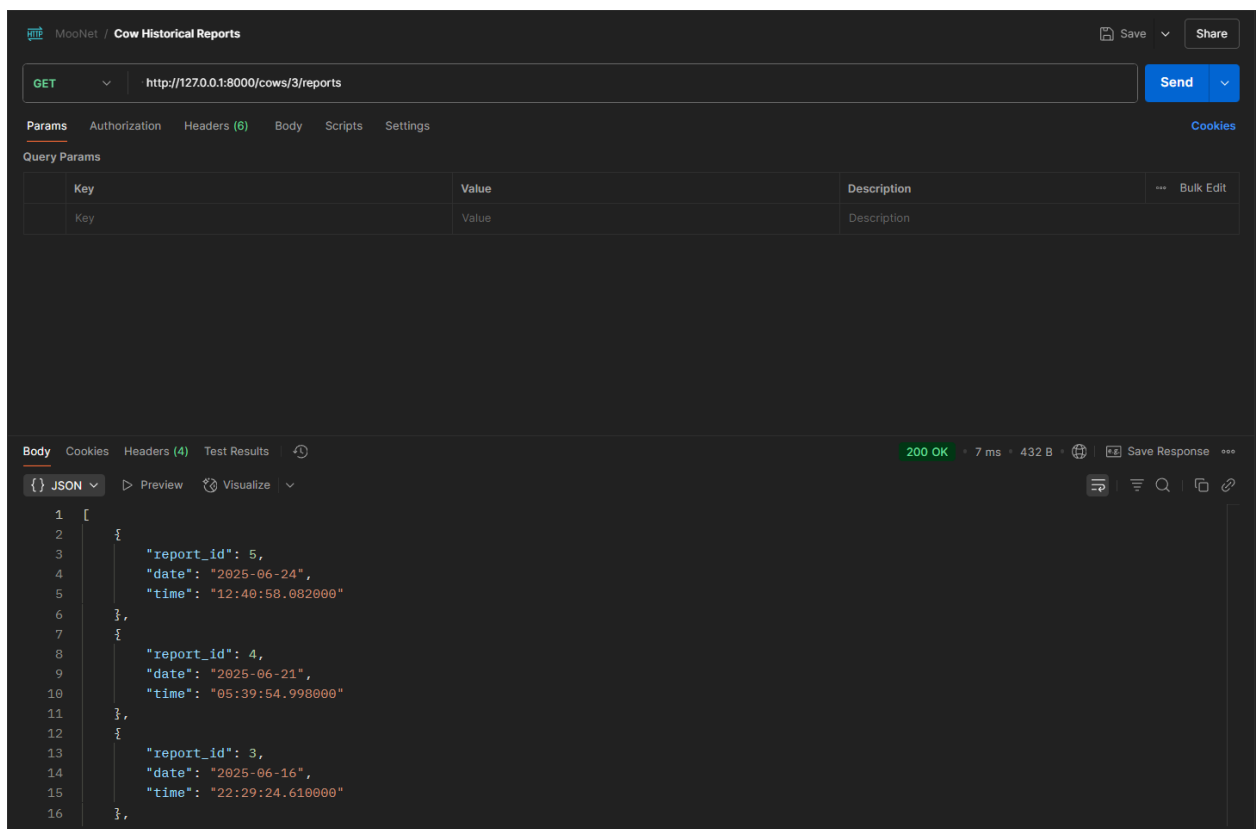


Figure 10.24: Report Browsing Per Cow Endpoint

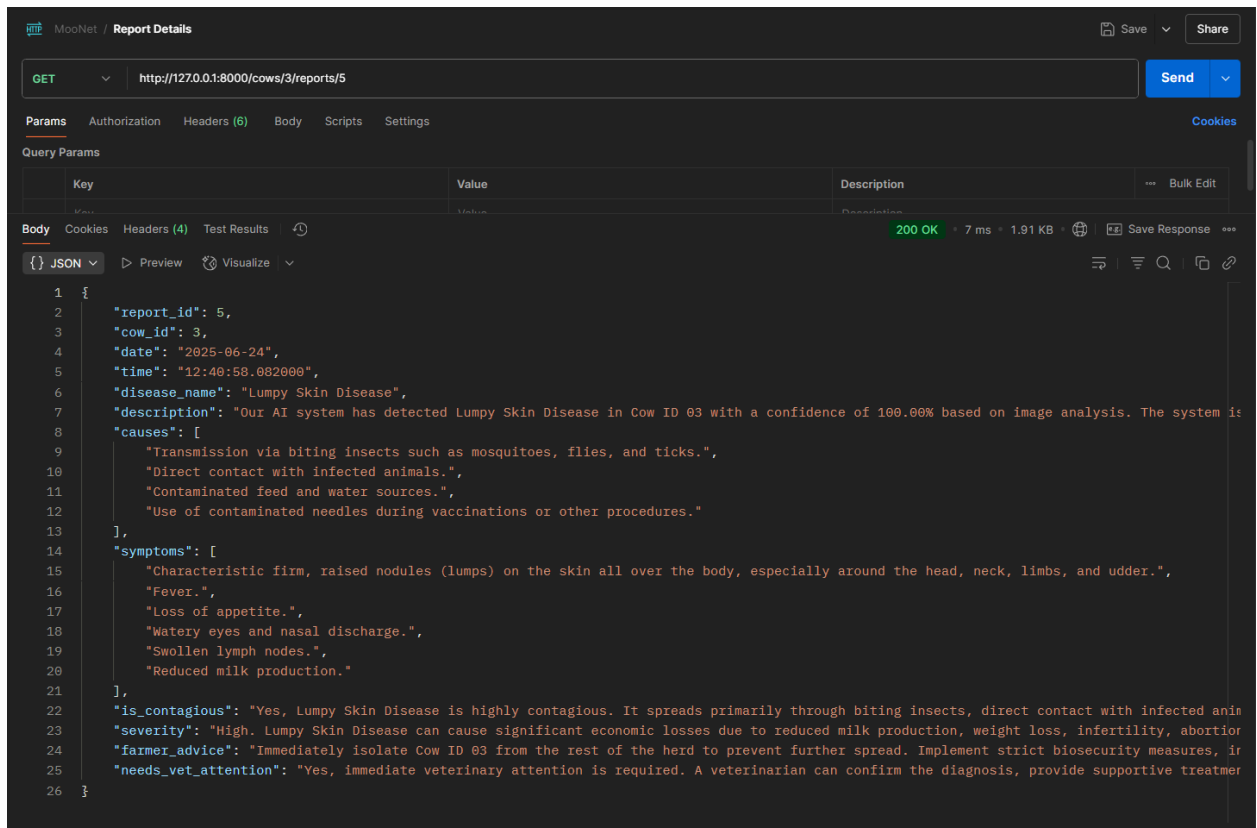


Figure 10.25: Retrieve Full Report Content Endpoint

10.3.5.5 Homepage Overview Stats

```

@app.get("/homepage", response_model=HomePageStats)
def get_homepage_stats(db: Session = Depends(get_db)):
    total = db.query(Cow).count()
    sick = db.query(Cow).filter(Cow.is_sick == True).count()

    recent_sick_cows = db.query(Cow)\
        .filter(Cow.is_sick == True)\
        .order_by(Cow.date.desc(), Cow.time.desc())\
        .limit(10).all()

    notifications = []
    for cow in recent_sick_cows:
        timestamp = datetime.combine(cow.date, cow.time)
        human_readable = NotificationResponse.get_human_readable_duration(timestamp)
        notifications.append(NotificationResponse(cow_id=str(cow.id), since=human_readable))

    return HomePageStats(
        total_cows=total,
        sick_cows=sick,
        recent_notifications=notifications
    )

```

Explanation:

Aggregates total number of cows, sick count, and a brief list of recent alerts. The frontend uses this for the main dashboard display.

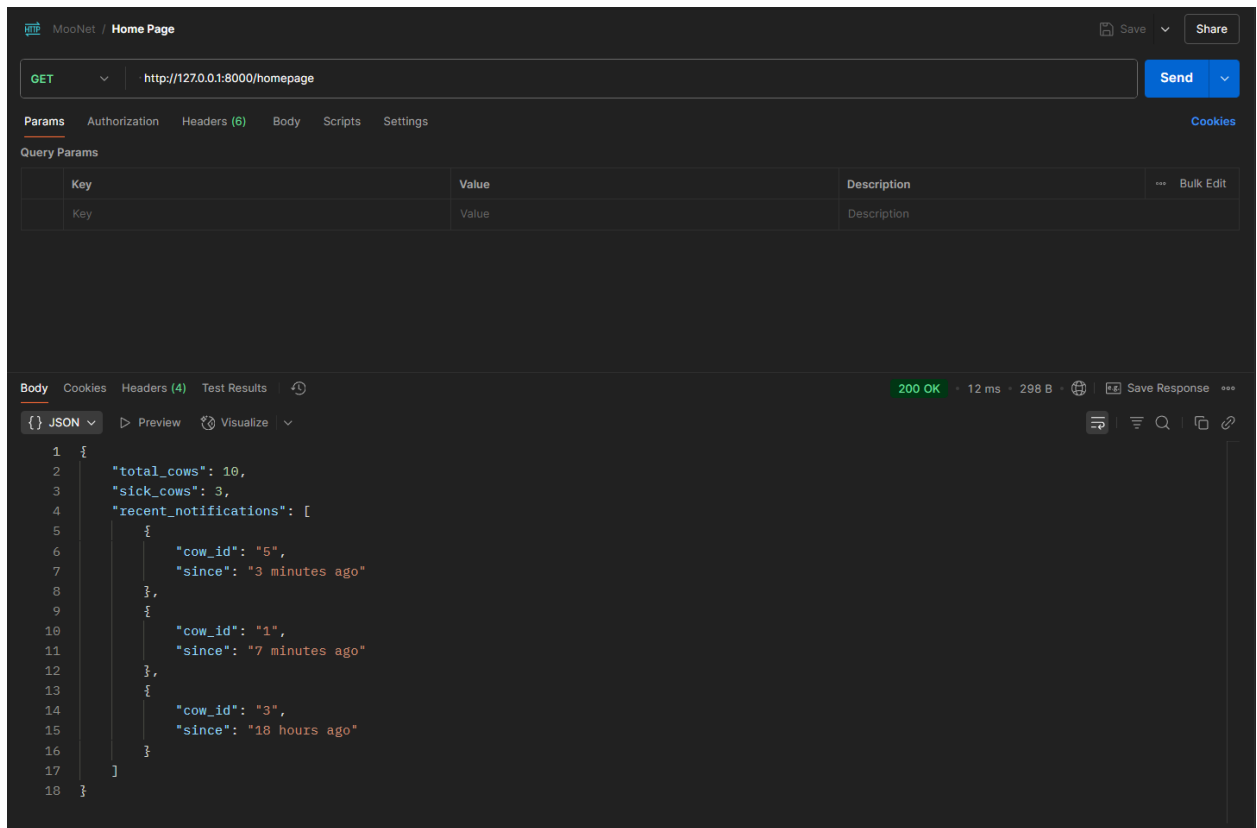


Figure 10.26: Homepage Overview Stats Endpoint

10.3.6 User Registration and Authentication

Secure user management is essential for role-based access in the application. The backend supports registration and login for both farmers and managers.

10.3.6.1 Password Hashing Utilities

Code Snippet:

```

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str):
    return pwd_context.hash(password)

def verify_password(plain_password, hashed_password):
    return pwd_context.verify(plain_password, hashed_password)

```

Explanation:

Passwords are hashed using bcrypt before being stored in the database. During login, plain-text passwords are compared with their hashed counterparts for verification.

10.3.6.2 User Signup

Code Snippet:

```
@app.post("/signup")
def signup(user: UserCreate, db: Session = Depends(get_db)):
    if user.password != user.confirm_password:
        raise HTTPException(status_code=400, detail="Passwords do not match")

    existing_user = db.query(User).filter(User.email == user.email).first()
    if existing_user:
        raise HTTPException(status_code=409, detail="Email already registered")

    new_user = User(
        name=user.name,
        email=user.email,
        password=hash_password(user.password),
        is_manager=user.is_manager
    )
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
```

Explanation:

Handles new user registration. It checks for matching passwords, ensures email uniqueness, hashes the password, and stores the user in the database.

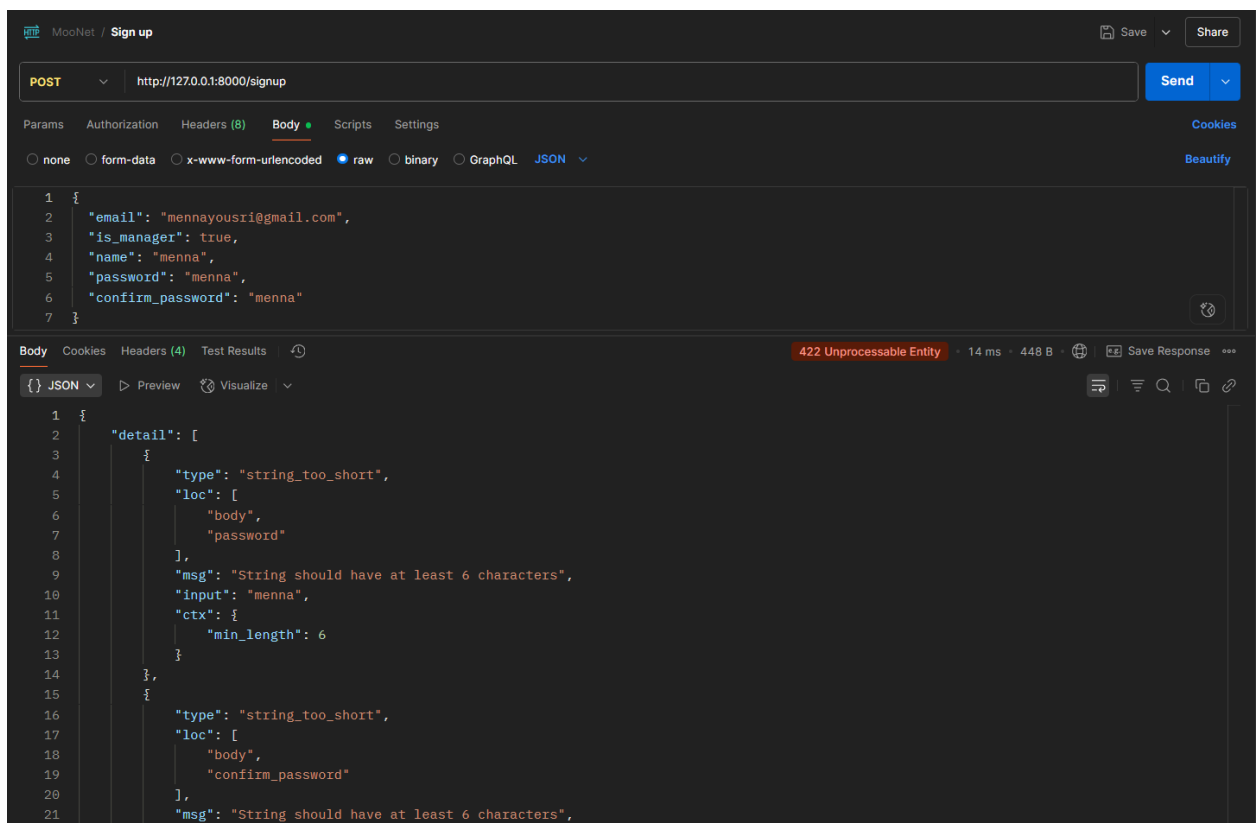


Figure 10.27: Sign up error: Password too short

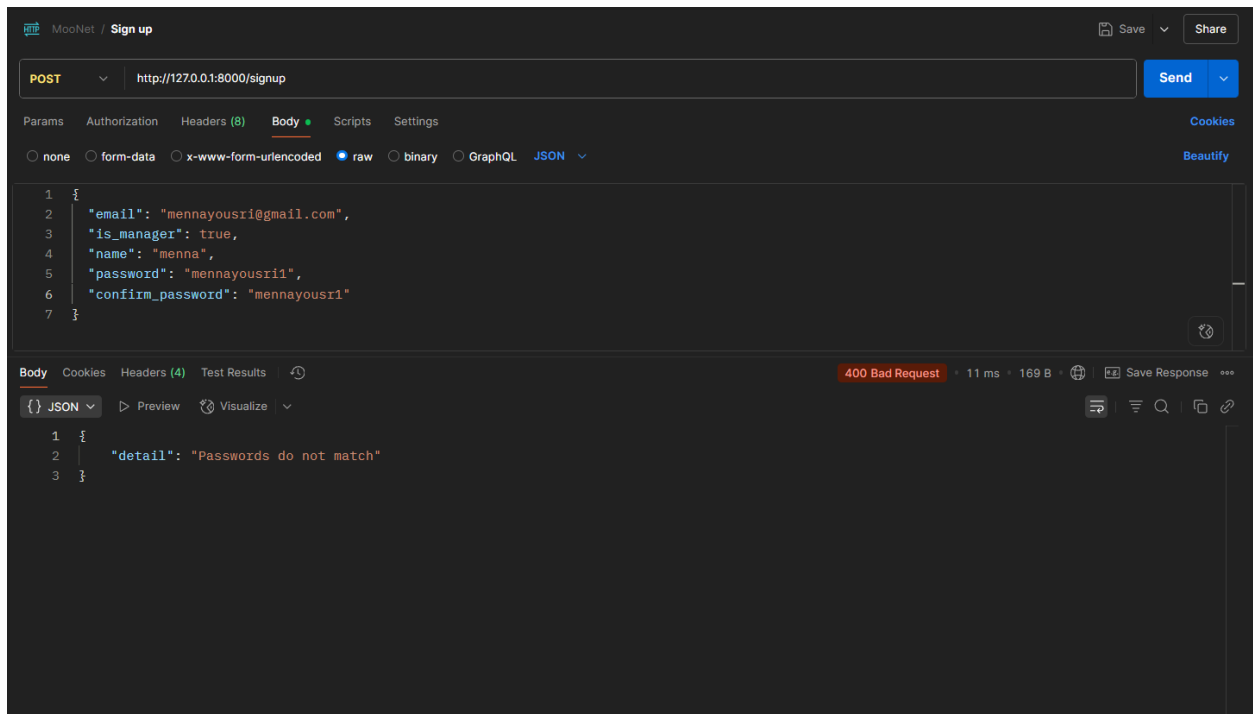


Figure 10.28: Sign up error: Passwords do not match

10.3.6.3 User Login

```

@app.post("/login", response_model=UserResponse)
def login(user: UserLogin, db: Session = Depends(get_db)):
    db_user = db.query(User).filter(User.email == user.email).first()
    if not db_user:
        raise HTTPException(status_code=401, detail="Invalid email or password")

    if not verify_password(user.password, db_user.password):
        raise HTTPException(status_code=401, detail="Invalid email or password")

    return UserResponse(is_manager=db_user.is_manager)

```

Explanation:

Verifies credentials against database records. If the credentials are valid, it returns the user's role to tailor the mobile experience.

Query

Query History

1 SELECT * FROM public.users
2 ORDER BY id ASC

Data Output

Messages

Notifications

+▼▼

| | id
[PK] integer | name
character varying | email
character varying | password
character varying | is_manager
boolean |
|---|--------------------|---------------------------|----------------------------|--|-----------------------|
| 1 | 1 | menna | mennayousri@gmail.com | \$2b\$12\$SLSVjcojnISzQ9oXrC/i5.8wVYqpiCirFfbfhICINhTfKbhkfZJm | true |
| 2 | 2 | string | user@example.com | \$2b\$12\$S86t.84Xu4GtNKEQA1ckKOnnUHcG2FYYSu3D7zb4HgLHYauAr... | false |

Figure 10.29: User Table

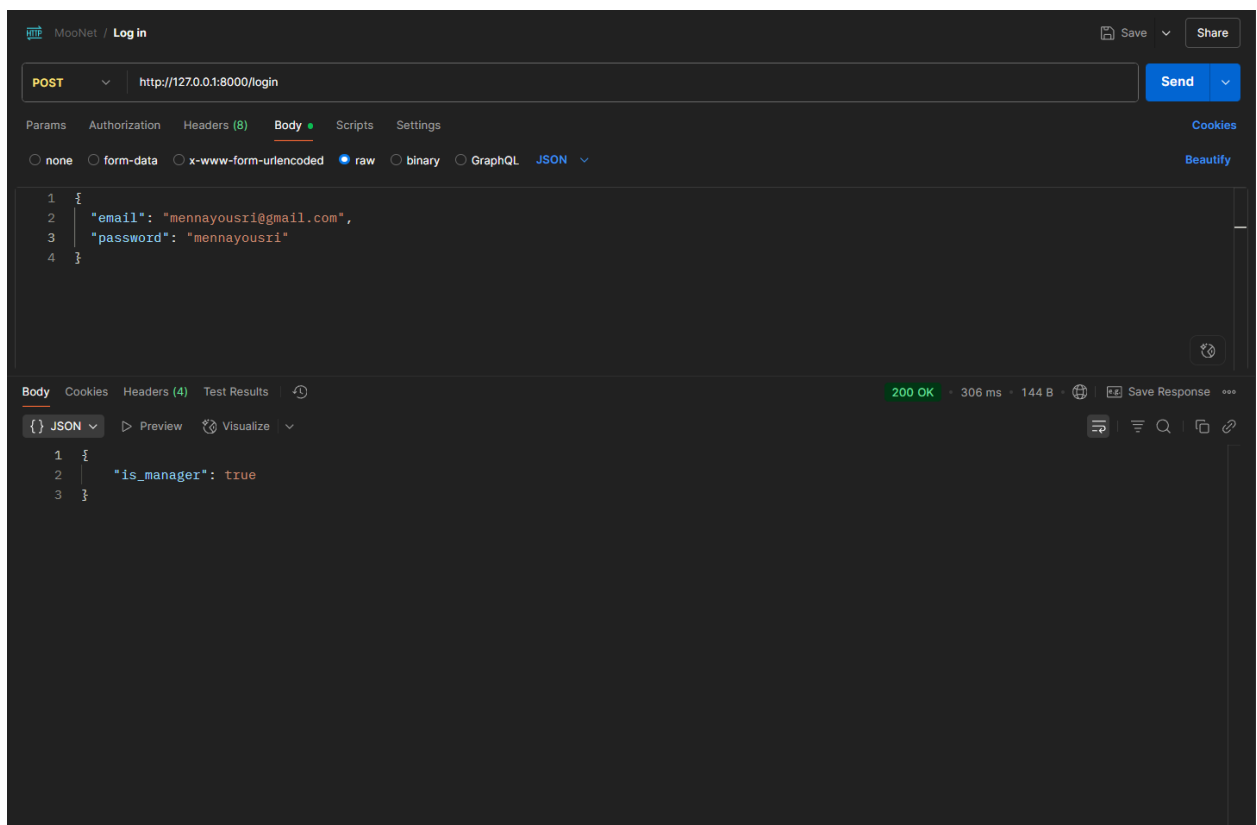


Figure 10.30: Log in Endpoint

10.3.7 Chatbot and Smart Answer APIs

The backend integrates a multilingual smart chatbot that supports veterinary and farming-related inquiries. Based on the nature of the question, the backend intelligently routes the query to one of three reasoning agents:

- **RAG Agent** (for document-based answers)
- **Pandas Agent** (for structured farm data queries)
- **Web Agent** (for real-time internet knowledge)

10.3.7.1 Farmer Chat Endpoint

Code Snippet:

```
@app.post("/chat/farmer", response_model=ChatResponse)
def chat_farmer(request: ChatRequest):
    try:
        result = json.loads(get_smart_answer_farmer(request.question))
        return ChatResponse(answer=result.get("answer", ""), type=result.get("type", "unknown"))
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Explanation:

When a farmer submits a natural language question, this endpoint classifies the intent using an LLM, invokes the corresponding agent, and returns a well-structured answer along with its source type (e.g., WEB, RAG, or Environment AGENT).

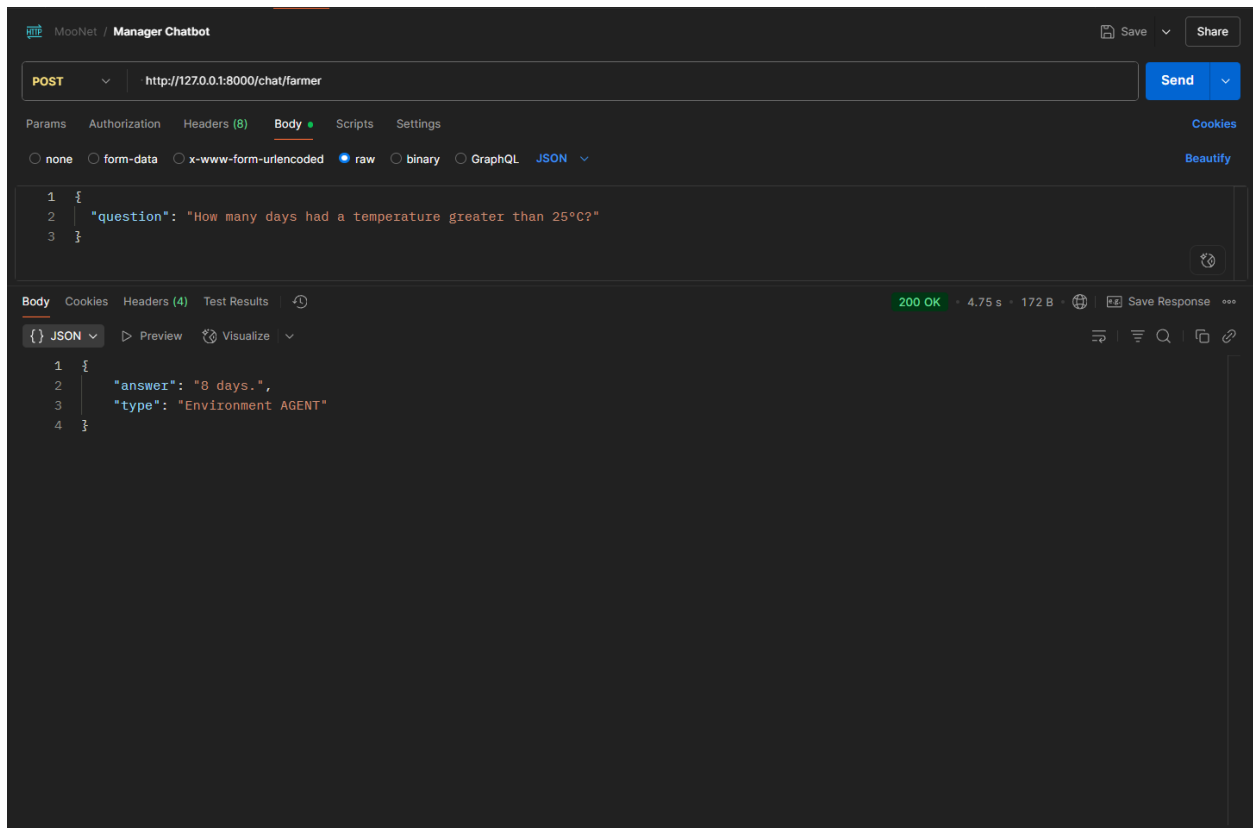


Figure 10.31: Farmer Chat Endpoint

10.3.7.2 Veterinarian Chat Endpoint

```
@app.post("/chat/vet", response_model=ChatResponse)
def chat_vet(request: ChatRequest):
    try:
        result = json.loads(get_smart_answer_vet(request.question))
        return ChatResponse(answer=result.get("answer", ""), type=result.get("type", "unknown"))
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Explanation:

Similar to the farmer chat API, this endpoint handles queries for veterinarians and invokes a specialized variant of the smart answering engine. The LLM response is tailored with deeper technical terminology and citation requirements when needed.

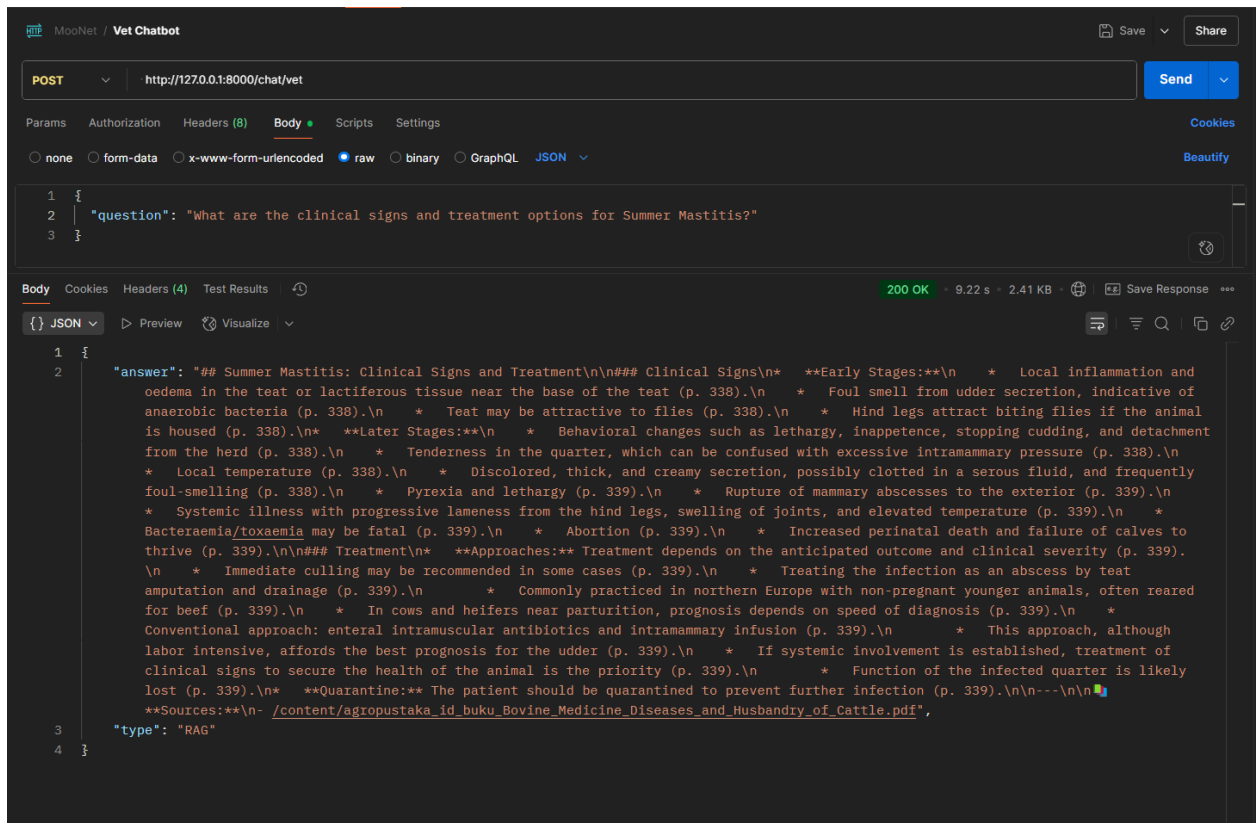


Figure 10.32: Veterinarian Chat Endpoint

10.3.8 Feed Recommendation and Optimization Endpoints

To assist in precision livestock nutrition, the system integrates **CrewAI-powered feed analysis**, delivering three key services:

1. Ideal feed formulation generation
2. Feed balance comparison
3. Market-based best product recommendation

10.3.8.1 Feed Recommendation with Caching

Code Snippet:

```
@app.post(
    "/feed-recommendation",
    response_model=IdealFeedFormulationResponse,
    summary="Get Ideal Feed Formulation",
    description="Analyzes cow profile and returns an ideal feed formulation. Results are cached."
)
async def get_feed_recommendation(cow_profile: CowProfileRequest):
    profile_hash = get_cow_profile_hash(cow_profile)

    if profile_hash not in _cached_results:
        print(f'Cache miss for profile: {profile_hash[:50]}... Running CrewAI process.')
        results = await run_crew_process(cow_profile)
        _cached_results[profile_hash] = results
    else:
        print(f'Cache hit for profile: {profile_hash[:50]}... Serving from cache.')
        results = _cached_results[profile_hash]

    return results["feed_recommendation"]
```

Explanation:

This endpoint analyzes the cow's profile (weight, type, production goal, etc.) and generates an ideal feed plan using CrewAI agents. Results are cached to optimize performance for repeated inputs.

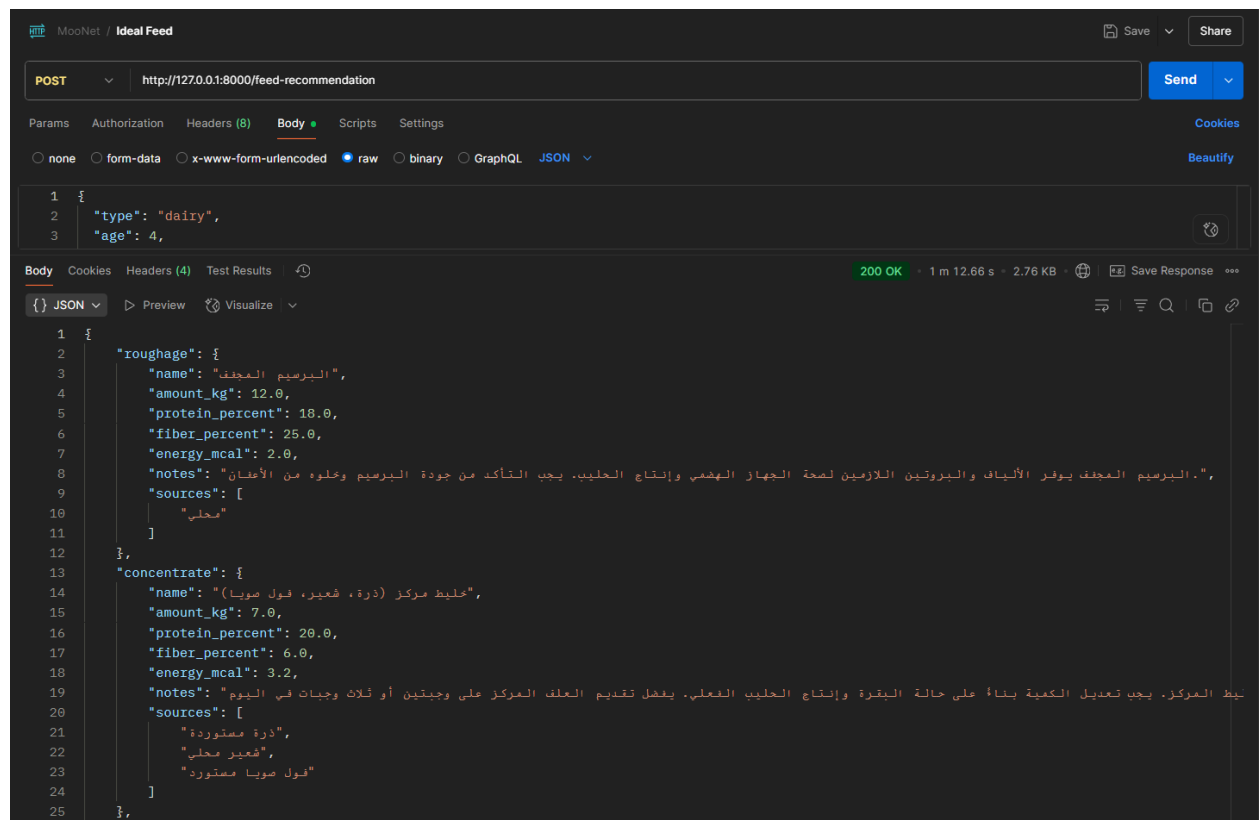


Figure 10.33: Feed Recommendation Endpoint

10.3.8.2 Feed Balance Check Endpoint

Code Snippet:

```
@app.post(
    "/feed-balance-check",
    response_model=FeedComparisonReportResponse,
    summary="Get Feed Balance Check Report",
    description="Compares current feed with ideal formulation and provides a detailed balance report. Results are cached."
)
async def get_feed_balance_check(cow_profile: CowProfileRequest):
    profile_hash = _get_cow_profile_hash(cow_profile)

    if profile_hash not in _cached_results:
        print(f'Cache miss for profile: {profile_hash[:50]}... Running CrewAI process.')
        results = await run_crew_process(cow_profile)
        _cached_results[profile_hash] = results
    else:
        print(f'Cache hit for profile: {profile_hash[:50]}... Serving from cache.')
        results = _cached_results[profile_hash]

    return results["feed_balance_check"]
```

Explanation:

Compares the cow's current feed against the ideal formulation. The report identifies **missing nutrients, surpluses**, and provides actionable advice on how to balance the feed more effectively.

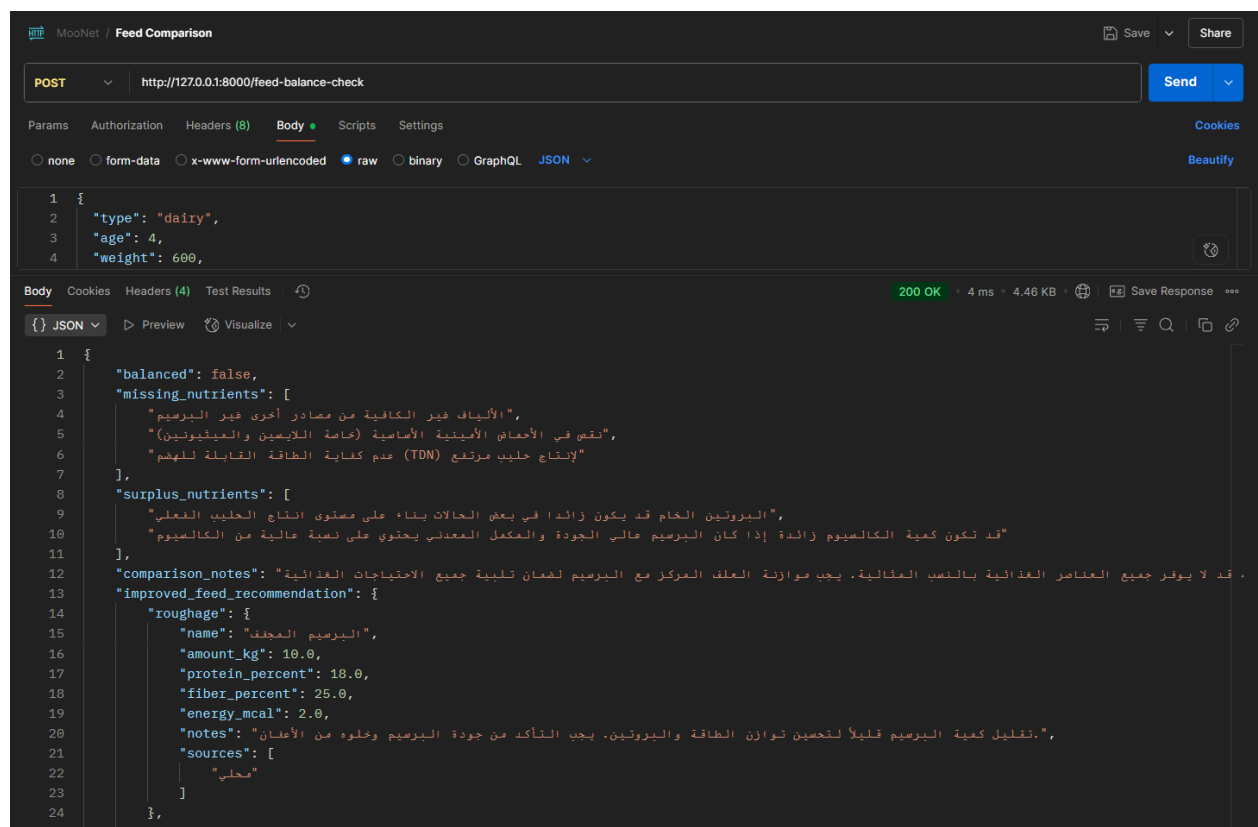


Figure 10.34: Feed Balance Check Endpoint

10.3.8.3 Best Feed Product Recommendation

Code Snippet:

```
@app.post(
    "/best-feed-recommendation",
    response_model=BestFeedRecommendationOutputResponse,
    summary="Get Best Feed Product Recommendation",
    description="Evaluates available feed products and recommends the best option based on value and suitability. Results are cached."
)
async def get_best_feed_recommendation(cow_profile: CowProfileRequest):
    profile_hash = _get_cow_profile_hash(cow_profile)

    if profile_hash not in _cached_results:
        print(f'Cache miss for profile: {profile_hash[:50]}... Running CrewAI process.')
        results = await run_crew_process(cow_profile)
        _cached_results[profile_hash] = results
    else:
        print(f'Cache hit for profile: {profile_hash[:50]}... Serving from cache.')
        results = _cached_results[profile_hash]

    return results["best feed recommendation"]
```

Explanation:

This endpoint searches available products (e.g., from e-commerce or partner APIs) and recommends the best one based on price, nutritional match, and user rating. It helps farmers make informed purchasing decisions.

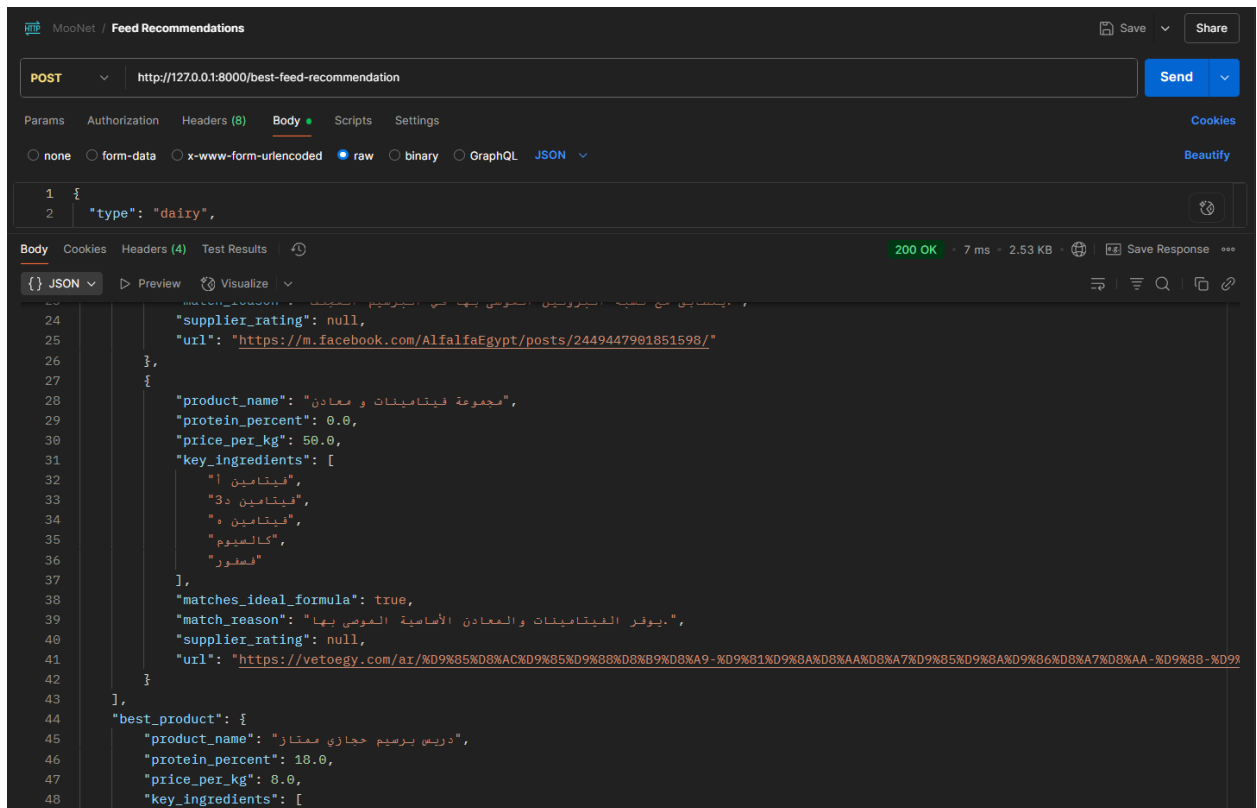


Figure 10.35: Best Feed Product Recommendation

10.3.9 Conclusion

The backend architecture of the mobile application is designed to deliver **real-time AI-driven cattle management**, encompassing:

- Continuous disease monitoring using deep learning and image simulation
- Role-specific smart chatbot answering powered by multi-agent LLM orchestration
- Structured database for cows, reports, and users using SQLAlchemy
- Modern, efficient API schemas via Pydantic and FastAPI
- Precision nutrition support through CrewAI-based feed planning modules

This modular, scalable backend system serves as the **core infrastructure** of the MooNet ecosystem, bridging sensor data, user interaction, and artificial intelligence into a unified decision-support platform tailored for modern smart farming in Egypt and beyond.

10.4 Flutter Implementation

MOONET is a smart mobile application designed to enhance livestock management and veterinary operations. Upon launch, the app displays a custom splash screen with the application's logo, followed by a secure interface for user login or account creation.

After successful authentication, users are directed to role-specific dashboards:

- **Managers** are provided with a dashboard that includes real-time statistics on cow health, alert notifications, and access to feed and environmental data.
- **Veterinarians** are directed to a medical dashboard focused on diagnostic reports, tracking of sick cases, and health recommendations.

The application's interface is organized around a **Bottom Navigation Bar**, featuring the following core sections:

- **Home** Offers an overview of the farm's status, including key cow metrics and recent alerts.
- **Camera** Enables users to capture cow images for AI-powered disease detection.
- **Feed** Delivers AI-generated suggestions for feed optimization and nutritional planning.
- **Chat** Provides access to an intelligent assistant for veterinary and managerial inquiries.
- **More** Includes user profile, settings, and logout functionality

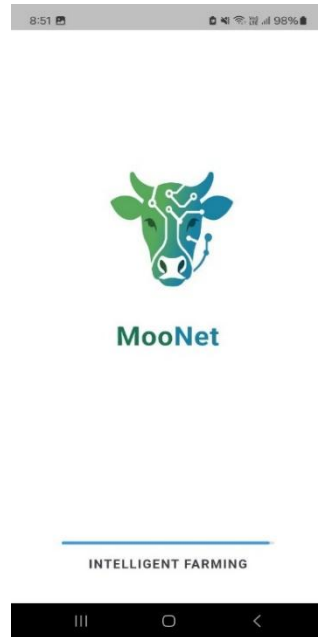


Figure 10.36: Splash Screen and Icon



Figure 10.37: Splash Screen and Icon-2

10.4.1 Account Creation and Login Pages

The **MOONET** application offers a straightforward and user-friendly interface for both account creation and login. At the top of the screen, users can select their role by choosing either **Veterinarian** or **Manager** through radio buttons. Below this, a toggle interface allows users to switch between **Sign in** and **Create an Account** modes.

When the "Sign In" option is selected, the screen displays input fields for **Email** and **Password**, along with additional features such as a "**Forgot Password?**" link and a "**Don't have an account? Sign Up**" prompt at the bottom. Alternatively, selecting "Create an Account" reveals input fields for **Full Name**, **Email**, **Password**, and **Confirm Password**, with a prompt that reads "**Already have an account? Login**".

This structure provides a seamless and intuitive user experience, allowing users to easily access the system based on their role while ensuring a smooth transition between login and registration.

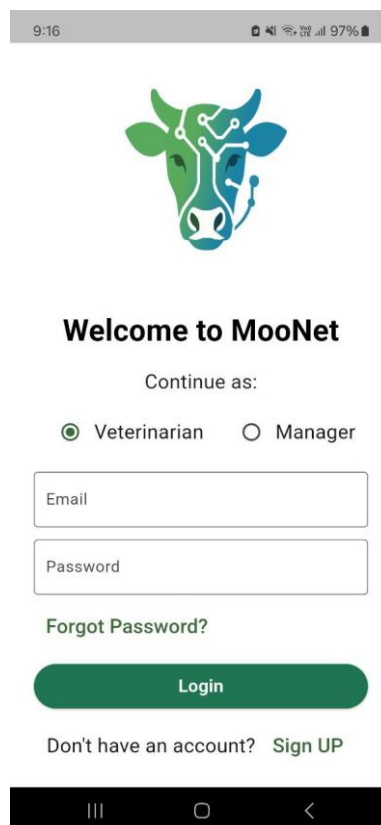


Figure 10.38: Login and Registration Pages

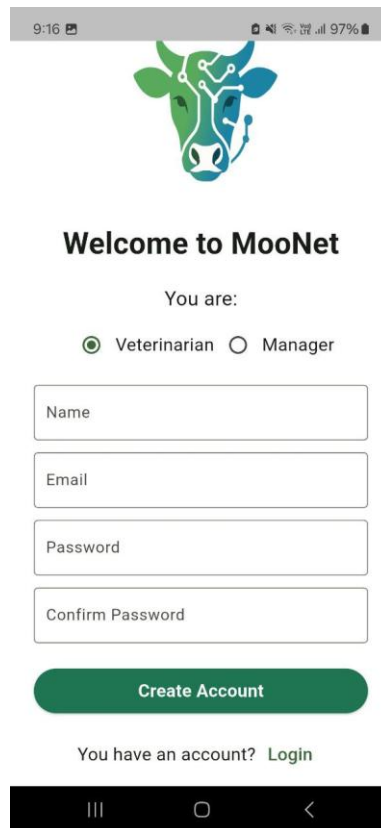


Figure 10.39: Login and Registration Pages-2

10.4.2 Home Page

- **Manger Home Page:**

The Home Page of the Cow Farm Manager application serves as the central dashboard for farm managers. It provides a clear overview of the farm's key statistics, recent health alerts, and quick access to important features. At the top, the manager is greeted with a personalized message ("Hi Manager!"), along with a profile icon and notification bell for user interaction and alerts.

Below the greeting, the page displays essential farm data in a card-based layout. This includes the total number of cows, the number of pregnant cows, sick cows, the type of farm (e.g., Dairy), and the farm's size in square kilometers. Each card is visually represented with an icon and bold numeric value to allow at-a-glance monitoring of farm health and operations.

The “Recent Alerts” section notifies the manager of any urgent issues, such as sick cows or environmental conditions like high temperature. Each alert includes an icon, a brief description (e.g., “Cow #1 is sick”), and a timestamp (e.g., “2 hours ago”), enabling quick responses to emerging problems.

Beneath the alerts, two large action buttons labeled “Health” and “Environment” provide immediate access to deeper insights into livestock health data and environmental factors. These buttons are styled with clear colors and spacing, making them easy to tap and use in real-life farm settings.

At the bottom, a fixed navigation bar allows the user to switch between the main sections of the app: Home, Camera, Feed, Chat, and More. This ensures that managers can quickly navigate to various features like taking photos, managing feed schedules, or chatting with support or veterinary services.

Overall, the Home Page balances simplicity and functionality. It provides a real-time snapshot of the farm’s status and acts as a command center for farm monitoring, health management, and navigation across the app’s ecosystem.

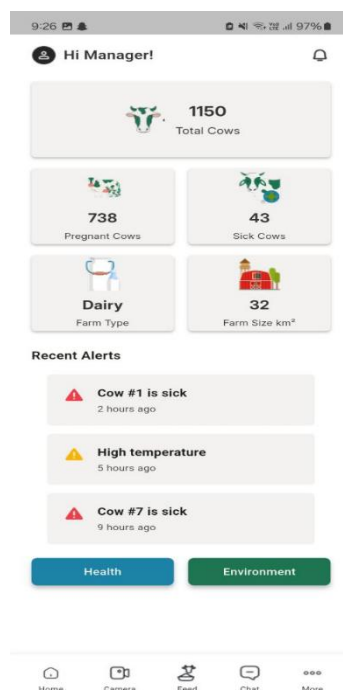


Figure 10.40: Manger Home Page

- **Veterinarian Home Page:**

The veterinarian dashboard in the Moonet Smart Livestock Monitoring System is a mobile user interface designed to assist veterinarians in monitoring the health of cattle in real time. The interface displays a welcome message along with vital statistics, including the total number of cows (1150) and the number of sick cows (43), both represented with intuitive icons for quick interpretation. Below the statistics section, a list of health alerts is presented in chronological order, highlighting individual cows that are reported as sick. Each alert includes a red warning icon, the cow's unique ID, and the time since the alert was generated (e.g., "Cow #1 is sick - 2 hours ago"). This helps the veterinarian prioritize which animals require immediate attention. The interface also includes a prominent "Health" button at the bottom of the screen, which can be used to navigate to more detailed health information or initiate specific actions such as treatments or medical history review. The design emphasizes clarity and usability, with a clean layout and color-coded indicators to make critical data easily accessible. This feature significantly improves the veterinarian's ability to respond quickly to health issues, reduce disease spread, and manage the herd efficiently. Future enhancements may include filtering alerts by severity or date, linking to detailed health records for each cow, and logging treatments for better case tracking.

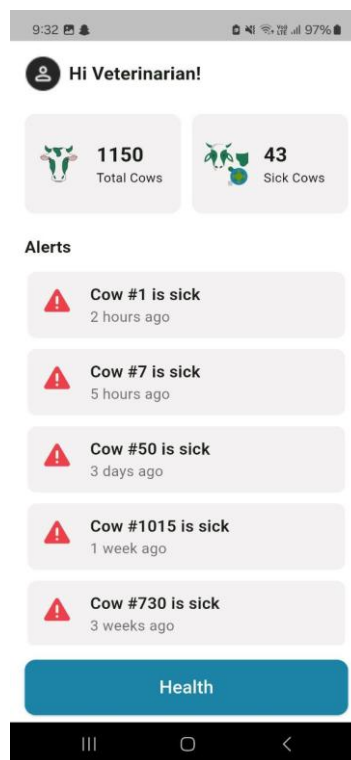


Figure 10.41: Veterinarian Home Page

10.4.3 Feed Mangement

The MooNet system includes a comprehensive nutrition management module designed to optimize livestock feeding plans through data-driven recommendations. This feature is implemented across four interconnected screens in the mobile application: Feed Inputs, Cow Nutrition Guide, Nutrition Adjustment, and Top Feed Recommender, each serving a distinct purpose in the nutritional management workflow.

The Feed Inputs screen forms the foundation of the system, allowing users to enter essential biological and production data about their cattle. The intuitive interface collects information including cow type (Dairy or Beef toggle selection), age in years, current weight in kilograms, daily milk yield for lactating animals, pregnancy status (Yes/No toggle), and breed specification. This detailed profiling enables the system to generate highly personalized feeding recommendations tailored to each animal's physiological state and production requirements. The form's clean design prioritizes usability in field conditions, with dynamic fields that adapt to the entered data - for instance, milk yield input only appears when "Dairy" is selected as the cow type.

Upon submission, the system processes these inputs and directs users to the Cow Nutrition Guide, which presents a complete feeding regimen structured into roughage and concentrate components. Each recommended feed is displayed with precise daily quantities (e.g., 6 kg/day of Alfalfa Hay) and detailed nutritional metrics including protein percentage (18.2% for Alfalfa), fiber content (30%), and energy value in Mcal (2.2). The guide goes beyond basic recommendations by incorporating practical notes on proper feed storage methods, adjustment guidelines based on the animal's changing condition, and sourcing references to local suppliers like Agricultural Cooperatives. This combination of scientific nutrition data and practical farming advice ensures both dietary adequacy and operational feasibility.

The Nutrition Adjustment screen provides ongoing dietary monitoring and optimization. It continuously evaluates the current feeding plan against the animal's needs, flagging nutritional imbalances through clear indicators like "Balanced: False"

status alerts. The system identifies specific nutrient deficiencies (such as fats or Vitamin B group) and potential excesses (notably protein levels that may require reduction), accompanied by explanatory notes linking these findings to the animal's production status. This diagnostic capability enables farmers to make evidence-based adjustments, with the system automatically suggesting modifications to feed quantities or compositions to restore nutritional balance.

Complementing these features, the Top Feed Recommender screen connects nutritional science with marketplace reality by evaluating and suggesting commercially available feed products. The system displays options like Cattle Fattening Feed (18% protein at 13.6 EGP/kg) with transparent assessments of their suitability, including "Matches Ideal Diet: False" flags when ingredient lists are incomplete or nutritional profiles diverge from recommendations. Each listing includes direct links to suppliers and price comparisons, while cautionary notes prompt users to verify unspecified ingredients. This integration of nutritional science with procurement information helps farmers make cost-effective decisions without compromising dietary quality.

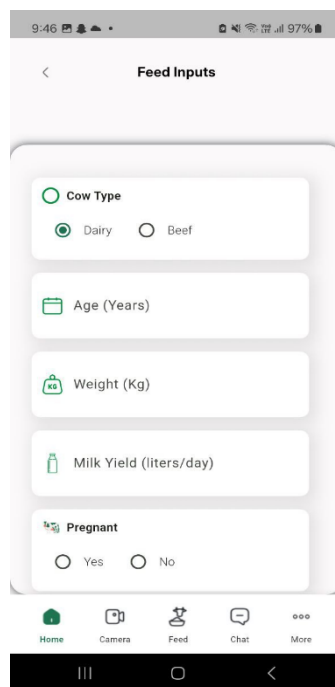


Figure 10.42: Feed Management Page

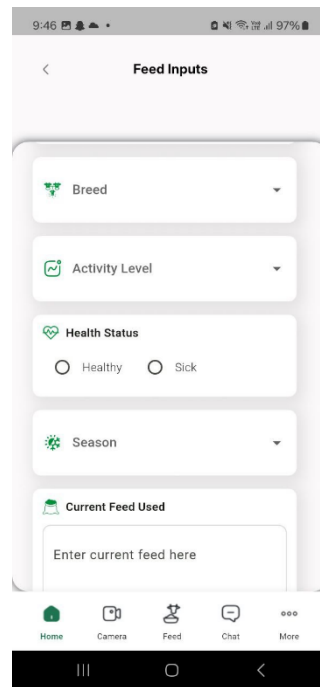


Figure 10.43: Feed Management Page-2

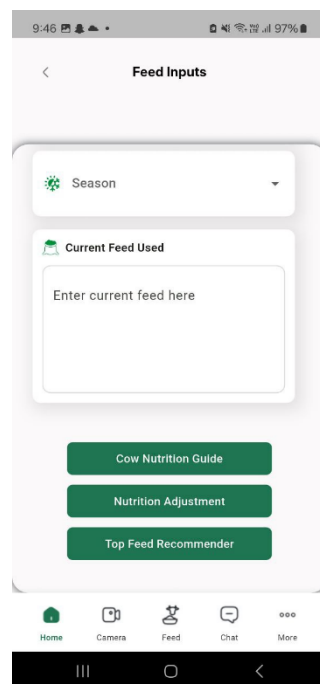


Figure 10.44: Feed Management Page

10.4.4 Chat Bot

The chat bot feature in our MooNet application plays a vital role in assisting users by providing instant responses to their inquiries. Designed with simplicity and clarity in mind, the chat interface allows users to ask one question at a time, ensuring focused

and accurate replies. A helpful instruction is displayed at the top, guiding users to phrase each question clearly for the best results. The minimal design includes a clean white background, a clearly labeled input field, and a distinct send button, all optimized for mobile usability. This streamlined experience encourages user interaction and supports seamless communication with the AI assistant. By providing a quick and accessible way to receive support, the chat bot feature improves user engagement, reduces confusion, and contributes to the overall effectiveness of the application.



Figure 10.45: Chatbot Page

10.4.5 Cow Health Monitoring

The cow monitoring feature in our MooNet application provides users with a clear and organized way to track the health status of each cow in the system. Through the Cow List screen, users can easily identify each cow by ID and view its current health condition, which is color-coded for quick recognition—green for healthy and red for sick. This visual representation allows for efficient management and prioritization of veterinary attention. The interface also includes a search bar, enabling users to quickly locate specific cows by ID.

By selecting a specific cow, users are directed to the history screen, where they can view a detailed timeline of health assessments. Each entry includes a date and time stamp, offering a transparent and traceable health record. This historical data is essential for veterinarians and farm managers to observe trends, identify recurring issues, and make informed decisions about treatment and care

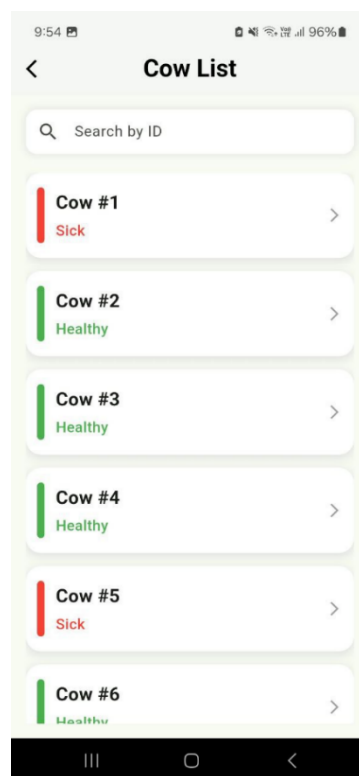


Figure 10.46: Cow List and History Pages

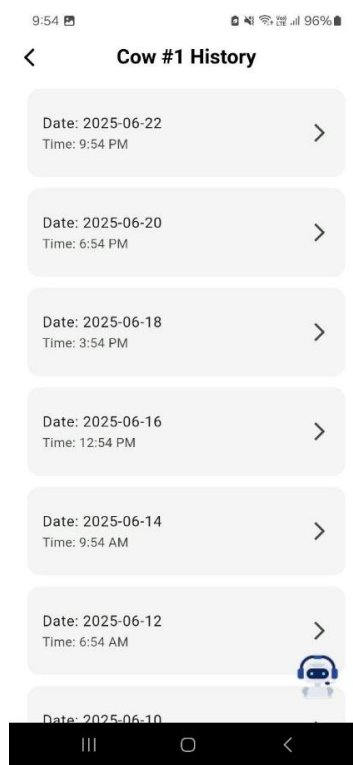


Figure 10.47: Cow List and History Pages-2

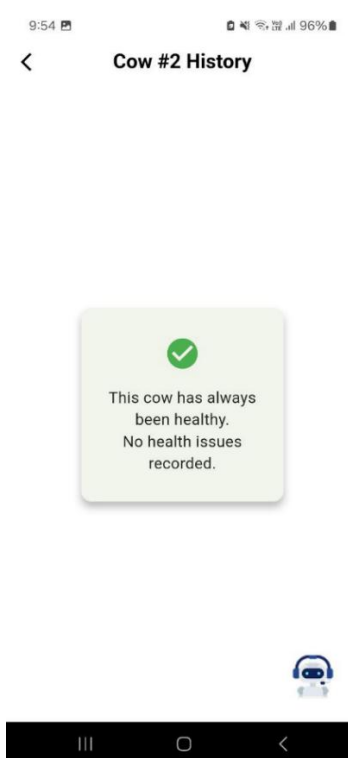


Figure 10.48: Cow List and History Pages-3

10.4.6 Flutter and Dart Code Snippets

- Login page

```
class _LoginScreenState extends State<LoginScreen> {
  int _selectedRole = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(
        children: [
          const Text('Continue as:', style: TextStyle(fontSize: 20)),
          Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Radio(
                value: 0,
                groupValue: _selectedRole,
                onChanged: (value) => setState(() => _selectedRole = value!),
              ),
              const Text('Veterinarian'),
              Radio(
                value: 1,
                groupValue: _selectedRole,
                onChanged: (value) => setState(() => _selectedRole = value!),
              ),
              const Text('Manager'),
            ],
          ),
          ElevatedButton(
            onPressed: () {
              if (_selectedRole == 0) {
                Navigator.pushReplacement(
                  context,
                  MaterialPageRoute(builder: (context) => VetHomePage()),
                );
              } else {
                Navigator.pushReplacement(
                  context,
                  MaterialPageRoute(builder: (context) => const HomeManagerScreen()),
                );
              }
            },
            child: const Text('Login'),
          ),
        ],
      ),
    );
  }
}
```

```
}
}
```

- **Veterinarian home page**

```
final List<Map<String, String>> alerts = [
  {'id': '#1', 'time': '2 hours ago'},
  {'id': '#7', 'time': '5 hours ago'},
];

Expanded(
  child: ListView.builder(
    itemCount: alerts.length,
    itemBuilder: (context, index) {
      final alert = alerts[index];
      return InkWell(
        onTap: () {
          Navigator.push(
            context,
            MaterialPageRoute(
              builder: (context) => CowProfilePage(cowId: alert['id']!),
            ),
          );
        },
      ),
    child: Container(
      margin: const EdgeInsets.symmetric(vertical: 6),
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: Color(0xFFF3F1F1),
        borderRadius: BorderRadius.circular(10),
      ),
      child: Row(
        children: [
          Image.asset('assets/alerts/sick_alter.png', width: 40, height: 40),
          const SizedBox(width: 16),
          Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text('Cow ${alert['id']} is sick',
                style: TextStyle(fontSize: 18, fontWeight: FontWeight.w600)),
              Text(alert['time']!,
                style: TextStyle(fontSize: 16, color: Colors.black54)),
            ],
          ),
        ],
      ),
    ),
  );
},
```

```
),
)
```

- Chat bot page

```
final TextEditingController _controller = TextEditingController();
final List<String> _messages = [];

void _sendMessage() {
  final text = _controller.text.trim();
  if (text.isEmpty) return;

  setState(() {
    _messages.add(text);
    _controller.clear();
  });
}

...

Expanded(
  child: ListView.builder(
    itemCount: _messages.length,
    itemBuilder: (context, index) {
      return Align(
        alignment: Alignment.centerRight,
        child: Container(
          margin: const EdgeInsets.symmetric(vertical: 4),
          padding: const EdgeInsets.all(10),
          decoration: BoxDecoration(
            color: Color(0xFFD0F0E8),
            borderRadius: BorderRadius.circular(16),
          ),
          child: Text(_messages[index], style: TextStyle(fontSize: 14)),
        ),
      );
    },
  ),
)
```

- Feed management page

Feed inputs page

```
final ageController = TextEditingController();
final weightController = TextEditingController();
final milkController = TextEditingController();
String selectedBreed = "";
bool? isPregnant;
bool? isHealthy;
```



```
Widget _buildInputCard(String hint, TextEditingController controller) => Container(
  margin: const EdgeInsets.only(bottom: 16),
  padding: const EdgeInsets.symmetric(horizontal: 12),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(10),
    boxShadow: const [BoxShadow(color: Colors.black12, blurRadius: 4)],
  ),
  child: TextField(
    controller: controller,
    decoration: InputDecoration(hintText: hint, border: InputBorder.none),
  ),
);

// Example use in build:
_buildInputCard('Age (Years)', ageController),
_buildInputCard('Weight (Kg)', weightController),
_buildInputCard('Milk Yield (liters/day)', milkController),
```

Nutrition guide page

```
Widget _buildSection(String title, String name, String amount, String protein
, String fiber, String energy, String notes, String source) {
  return [
    Text(title, style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
    const SizedBox(height: 8),
    Container(
      width: 300,
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: Colors.white,
        boxShadow: [BoxShadow(color: Colors.black12, blurRadius: 8)],
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              _buildLabelValue("Name", name),
              _buildLabelValue("Amount", amount),
            ]),
          const SizedBox(height: 12),
          Row(mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              _buildLabelValue("Protein", protein),
              _buildLabelValue("Fiber", fiber),
              _buildLabelValue("Energy", energy),
            ]),
        ],
      ),
    ],
  );
```

```

    const SizedBox(height: 12),
    _buildLabelValue("Notes", notes),
    const SizedBox(height: 8),
    _buildLabelValue("Source", source),
  ],
),
),
];
}

```

Top Feed Recommender Page

```

Widget _buildFeedCard({
  required String title,
  required String protein,
  required String price,
  required String link,
  required bool isBest,
}) {
  return Container(
    padding: const EdgeInsets.all(12),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(12),
      border: isBest ? Border(left: BorderSide(color: Color(0xFF00913C), width: 6)) : null,
      boxShadow: const [BoxShadow(color: Colors.black12, blurRadius: 8)],
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(title, style: const TextStyle(fontWeight: FontWeight.bold)),
        const SizedBox(height: 8),
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Text("Protein: $protein"),
            Text("Price: $price"),
          ],
        ),
        const SizedBox(height: 8),
        Text("Missing ingredient details to check match."),
        const SizedBox(height: 4),
        Text(link, style: const TextStyle(color: Colors.blue, decoration: TextDecoration.underline)),
      ],
    ),
  );
}

```

10.4.7 Backend Integration and API Functionality

These API integrations ensured that the app could dynamically fetch and send real-time data between users and the backend server

Below is a list of key API endpoints used in the project, along with their purpose, the screens they were connected to, the structure of the HTTP requests, and example Flutter code demonstrating how each was implemented.

- **Sign Up – POST**

```
try {
  final response = await http.post(
    Uri.parse('http://10.0.2.2:8000/signup'), // 🖱️ Emulator IP to access localhost
    headers: {
      'Content-Type': 'application/json',
      'Accept': '*/*',
    },
    body: jsonEncode({
      "name": name,
      "email": email,
      "password": password,
      "confirm_password": confirmPassword,
      "is_manager": isManager,
    }),
  );

  if (response.statusCode == 200 || response.statusCode == 201) {
    _showSnackBar('Account created successfully!', isSuccess: true);
    Future.delayed(const Duration(milliseconds: 1200), () {
      if (!mounted) return;
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(
          builder: (_) => isManager ? const HomeManagerScreen() : VetHomePage(),
        ), // MaterialPageRoute
      );
    }); // Future.delayed
  } else {
    if (response.body.isNotEmpty) {
      final data = jsonDecode(response.body);
    }
  }
}
```

- **Login – POST**

```

try {
  final response = await http.post(
    Uri.parse('http://10.0.2.2:8000/login'),
    headers: {
      'Content-Type': 'application/json',
      'Accept': '*/*',
    },
    body: jsonEncode({
      "email": email,
      "password": password,
    }),
  );

  if (response.statusCode == 200) {
    final data = jsonDecode(response.body);

    final isManager = data['is_manager'] == true;

    _showSnackBar('Login successful!', isSuccess: true);
    await Future.delayed(const Duration(milliseconds: 1000));

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(
        builder: (_) => isManager ? const HomeManagerScreen() : VetHomePage(),
      ), // MaterialPageRoute
    );
  }
}

```

- **Ideal Feed – POST**

```

const SizedBox(height: 12),
Text("Matches Ideal Diet: ${product["matches_ideal_formula"] ? 'Yes' : 'No'}", style: const TextStyle(
const SizedBox(height: 8),
Text(product["match_reason"] ?? '', style: const TextStyle(fontSize: 13)),
const SizedBox(height: 8),
Text("Ingredients: ${product["key_ingredients"]?.join(', ')}", style: const TextStyle(fontS
const SizedBox(height: 12),
Text(product["url"] ?? '', style: const TextStyle(color: Colors.blue, decoration: TextDecoration.
],

```

- Feed Comparison – POST

```
Text("Balanced: ${balanced ? 'True' : 'False'}", style:
const SizedBox(height: 8),
const Text("Missing Nutrients:"),
Padding(
  padding: const EdgeInsets.only(left: 12),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: missing.map((e) => Text("• $e")).toList(),
  ), // Column
), // Padding
const SizedBox(height: 8),
const Text("Extra Nutrients:"),
Padding(
  padding: const EdgeInsets.only(left: 12),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: surplus.map((e) => Text("• $e")).toList(),
```

- Feed Recommendations – POST

```
@override
Widget build(BuildContext context) {
  final roughage = feedData['roughage'];
  final concentrate = feedData['concentrate'];
  final supplements = List<Map<String, dynamic>>.from(feedData['supplements']);
  final water = feedData['water'];
  final notes = feedData['notes'];
  final generatedDate = feedData['generated_date'];
  final generatedTime = feedData['generated_time'];
```

- Home Page – GET

```
Future<void> fetchHomepageData() async {  
  final url = Uri.parse('http://10.0.2.2:8000/homepage');  
  final response = await http.get(url);  
  
  if (response.statusCode == 200) {  
    final data = json.decode(response.body);  
    setState(() {  
      homepageData = HomePageData.fromJson(data);  
      isLoading = false;  
    });  
  } else {  
    print('Failed to load homepage data');  
  }  
}
```

- Notification – GET

```
Future<void> fetchAlerts() async {  
  final url = Uri.parse('http://10.0.2.2:8000/notification');  
  
  try {  
    final response = await http.get(url);  
    if (response.statusCode == 200) {  
      final List<dynamic> data = json.decode(response.body);  
      setState(() {  
        alerts = data.map((alert) => {  
          'cow_id': alert['cow_id'],  
          'since': alert['since'],  
          'icon': 'assets/icons/warning_1.png',  
          'title': 'Cow #${alert['cow_id']} is sick',  
        }).toList();  
        isLoading = false;  
      });  
    } else {  
      print('Failed to fetch alerts');  
      setState(() => isLoading = false);  
    }  
  } catch (e) {  
    print('Error: $e');  
    setState(() => isLoading = false);  
  }  
}
```

- **Camera – GET**

```
Future<void> fetchLiveImage() async {
  final url = Uri.parse('http://10.0.2.2:8000/camera/latest');

  try {
    final response = await http.get(url);
    if (response.statusCode == 200) {
      final data = json.decode(response.body);
      final base64Image = data['rgb_image'] as String;

      setState(() {
        _rgbImageBytes = base64Decode(base64Image);
        _timestamp = data['timestamp'];
        _cowId = data['cow_id'];
        _isLoading = false;
      });
    }
  }
}
```

- **Cows – GET**

```
Future<void> fetchCows() async {
  final response = await http.get(Uri.parse('http://10.0.2.2:8000/cows'));

  if (response.statusCode == 200) {
    final List<dynamic> data = json.decode(response.body);
    setState(() {
      cows = data.map<Map<String, dynamic>>((cow) {
        return {
          "id": cow["cow_id"],
          "status": cow["is_sick"] == true ? "Sick" : "Healthy",
        };
      }).toList();
    });
  } else {
    print("Failed to load cows");
  }
}
```

```
@override
Widget build(BuildContext context) {
  final filteredCows = cows.where((cow) {
    return cow["id"].toString().contains(searchQuery.trim());
  }).toList();
}
```

```
MaterialPageRoute(  
  builder: (_) => isSick  
    ? CowHistorySick(cowId: int.parse(cow["id"]))  
    : CowHistoryHealthy(cowId: int.parse(cow["id"])),  
) , // MaterialPageRoute
```

- **Environment – GET**

```
// Fetch gas data  
static Future<int> getGasLevel() async {  
  try {  
    final response = await http.get(Uri.parse('${_baseUrl}/gas'));  
    if (response.statusCode == 200) {  
      final data = json.decode(response.body);  
      return data['gas']?.toInt() ?? 0;  
    }  
    throw Exception('Failed to load gas data');  
  } catch (e) {  
    throw Exception('Gas API error: $e');  
  }  
}
```

```
// Fetch temperature data  
static Future<double> getTemperature() async {  
  try {  
    final response = await http.get(Uri.parse('${_baseUrl}/temp'));  
    if (response.statusCode == 200) {  
      final data = json.decode(response.body);  
      return data['temperature']?.toDouble() ?? 0.0;  
    }  
    throw Exception('Failed to load temperature data');  
  } catch (e) {  
    throw Exception('Temperature API error: $e');  
  }  
}
```



```
// Fetch VOC data
static Future<int> getVocLevel() async {
  try {
    final response = await http.get(Uri.parse('${_vocBaseUrl}/voc'));
    if (response.statusCode == 200) {
      final data = json.decode(response.body);
      return data['voc']?.toInt() ?? 0;
    }
    throw Exception('Failed to load VOC data');
  } catch (e) {
    throw Exception('VOC API error: $e');
  }
}
```

Chapter 11

CONCLUSION AND FUTURE WORK

11.1 Introduction

This chapter presents a summary of the **MooNet** project, highlights its main outcomes, and outlines future improvements to further enhance its performance and impact.

11.2 Conclusions

This project introduced **MooNet**, an integrated smart farming system designed to transform livestock management by combining Artificial Intelligence (AI), the Internet of Things (IoT), and user-friendly mobile technologies. MooNet addresses major challenges faced by cattle farmers, including delayed disease detection, inefficient feeding strategies, lack of real-time monitoring, and inadequate environmental control.

The system integrates AI-driven disease detection models, real-time environmental sensors, feed recommendation engines, and LLM-based decision support. These components work together to provide timely, actionable insights that improve both animal welfare and farm productivity.

The main achievements of the project include:

- **Early disease detection**, particularly Mastitis and Lumpy Skin Disease, using deep learning models applied to real-time video streams.
- **Environmental monitoring and control**, with sensors and actuators managing gas levels, lighting, temperature, and water supply.
- **Feed optimization**, utilizing multi-agent collaboration (CrewAI) and intelligent APIs to recommend balanced, cost-efficient nutrition plans.

- **AI-powered chatbot**, based on large language models (LLMs) and retrieval-augmented generation (RAG), providing farmers with interactive, context-aware assistance.
- **Mobile application**, offering users real-time control, alert systems, and full access to **MooNet** features anytime, anywhere.

These outcomes confirm MooNet's value as a comprehensive solution that enhances operational efficiency, supports data-driven decision-making, and promotes sustainable farming practices.

11.3 Concluding Remarks

MooNet demonstrates the successful integration of advanced AI, sensor networks, and intuitive design into a unified system that is both practical and scalable. The solution is particularly relevant to developing regions such as Egypt and the MEA market, where precision agriculture remains underdeveloped.

Unlike fragmented solutions that focus on isolated features, **MooNet** offers a cohesive ecosystem that brings together disease monitoring, environmental control, feed optimization, and farmer support all within a single platform. Its adaptability and real-time intelligence offer a competitive edge in underserved markets.

Beyond technical feasibility, the project showcases commercial potential through a well-defined business model, alignment with market needs, and direct relevance to ongoing agricultural modernization efforts. **MooNet** supports national goals related to food security, sustainability, and digital transformation in the farming sector.

11.4 Future Work

While the current version of **MooNet** achieves its core objectives, future enhancements are proposed to expand its capabilities:

Automated Waste Management

- Integrate intelligent waste-handling systems, including slurry level sensors and compost monitoring, to improve hygiene, reduce pollution, and promote eco-friendly practices.

Camera-Based Biometric Identification

- Extend the use of YOLOv8 for visual identification of individual cows, eliminating the need for physical UWB tags and enabling smoother livestock tracking.

Health History and Predictive Analytics

- Develop per-animal medical records and introduce predictive models capable of identifying early disease trends, allowing intervention before visible symptoms occur.

These improvements will further strengthen MooNet's position as a cutting-edge solution in smart livestock farming.

BIBLIOGRAPHY

- [1]. Vu, H., Prabhune, O., Panditharatne, D., Choi, C. Y., Raskar, U., Chung, H., Kim, Y., “MMCOWS: A Multimodal Dataset for Dairy Cattle Monitoring”, Purdue University, University of Wisconsin–Madison, and Iowa State University, (2020).
- [2]. Andrew, W., Burghardt, T., Campbell, N. W., Dowsey, A., “OpenCows2020: Visual Identification of Individual Holstein Friesian Cattle via Deep Metric Learning”, University of Bristol, arXiv:2006.09205, (2020). Available: <https://arxiv.org/abs/2006.09205>
- [3]. Kaggle, “Mastitis Disease Detection Dataset”. Available: <https://www.kaggle.com/datasets/sivaprathishsiva/mastitis-disease-detection> (Accessed: 2025).
- [4]. Roboflow, Kirubel Yemane, “Cow and Mastitis Detection Dataset”, Roboflow Universe. Versions: 1, 2, 7, 11. Available: <https://universe.roboflow.com/kirubel-yemane/cow-and-mastitis-detection> (Accessed: 2025).
- [5]. Kaggle, “Lumpy Skin Images Dataset”. Available: <https://www.kaggle.com/datasets/warcoder/lumpy-skin-images-dataset> (Accessed: 2025).
- [6]. Agarwal, S., “Cow Lumpy Disease Dataset”, Kaggle. Available: <https://www.kaggle.com/datasets/shivamagarwal29/cow-lumpy-disease-dataset/data> (Accessed: 2025).
- [7]. Panwar, A., “Lumpy Cow Dataset”, Kaggle. Available: <https://www.kaggle.com/datasets/ayushpanwar058/lumpy-cow-dataset> (Accessed: 2025).

- [8]. Rimal, K., “Lumpy Skin Disease Cow Images”, Kaggle. Available: <https://www.kaggle.com/datasets/kaushalrimal619/lumpy-skin-disease-cow-images> (Accessed: 2025).
- [9]. Shahane, S., “Lumpy Skin Disease Dataset”, Kaggle. Available: <https://www.kaggle.com/datasets/saurabhshahane/lumpy-skin-disease-dataset/data> (Accessed: 2025).
- [10]. Roboflow, “Lumpy Skin Disease Detection Dataset”, Roboflow Universe. Available: <https://universe.roboflow.com/qq-mgfrz/lumpy-skin-disease-detection> (Accessed: 2025).
- [11]. Roboflow, “Cow Disease Identifier Dataset”, Roboflow Universe. Available: <https://universe.roboflow.com/project-for-grd-10/cow-disease-identifier> (Accessed: 2025).
- [12]. Roboflow, “Lumpy Disease Dataset”, Roboflow Universe. Available: <https://universe.roboflow.com/new-y4lgu/lumpy-disease> (Accessed: 2025).
- [13]. Farzad R., “Advanced Q&A and RAG Series”, GitHub Repository. Available: <https://github.com/Farzad-R/Advanced-QA-and-RAG-Series> (Accessed: 2025).
- [14]. Ultralytics, “YOLOv8 Object Detection Model”, GitHub Repository. Available: <https://github.com/ultralytics/ultralytics> (Accessed: 2025).
- [15]. Tan, M., Le, Q. V., “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks”, arXiv preprint arXiv:1905.11946, (2019).
- [16]. CrewAI, “CrewAI: Multi-Agent Orchestration Framework for Autonomous AI Agents”, GitHub Repository. Available: <https://github.com/joaomdmoura/crewAI> (Accessed: 2025).
- [17]. “MooNet Hardware Datasheets and Reference Documents”, Google Drive Folder. Available:

<https://drive.google.com/drive/folders/16z6V3xKBhjLbZSaYOpS79CIrHnxfkJkc> (Accessed: 2025).

[18]. Precedence Research, “Livestock Monitoring Market Size, Share, Growth”. Available: <https://www.precedenceresearch.com/livestock-monitoring-market> (Accessed: 2025).

[19]. Custom Market Insights, “Livestock Monitoring Market Research Report 2024 – 2032”. Available: <https://www.custommarketinsights.com/report/livestock-monitoring-market/> (Accessed: 2025).

[20]. Mordor Intelligence, “Global Livestock and Meat Market - Industry Report”. Available: <https://www.mordorintelligence.com/industry-reports/global-live-stock-and-meat-market-industry> (Accessed: 2025).

[21]. Grand View Research, “Livestock Monitoring Market Outlook – Middle East & Africa”. Available: <https://www.grandviewresearch.com/horizon/outlook/livestock-monitoring-market/mea> (Accessed: 2025).

[22]. Statista, “Livestock Production in Egypt by Species – 2023 Statistics”. Available: <https://www.statista.com/statistics/1073279/egypt-livestock-production-by-species/> (Accessed: 2025).

ملخص المشروع

تتكون هذه الوثيقة من أحد عشر باباً، يتم من خلالها توضيح كيفية تطوير نظام ذكي لإدارة الثروة الحيوانية باستخدام تقنيات حديثة مثل الذكاء الاصطناعي (AI) وإنترنت الأشياء (IoT) وفيما يلي نبذة مختصرة توضح ما يتناوله كل باب:

الباب الأول: يعرض هذا الباب الفكرة الرئيسية لمشروع MooNet ، وأهدافه، والدوافع وراء تنفيذه، بالإضافة إلى هيكل الوثيقة.

الباب الثاني: يتناول هذا الباب السياق العام للمشروع، بما في ذلك نظرة عامة على النظام، التحديات في الزراعة التقليدية، وأبرز المميزات التي يقدمها MooNet.

الباب الثالث: يقدم هذا الباب تحليلاً لسوق الثروة الحيوانية، ويحدد الفئة المستهدفة من المستخدمين، والفرص المتاحة، مع عرض تحليل SWOT.

الباب الرابع: يستعرض هذا الباب الأنظمة الموجودة في مجال إدارة الثروة الحيوانية، ويوضح المزايا التي يقدمها MooNet كمنصة موحدة تعتمد على الذكاء الاصطناعي.

الباب الخامس: يشرح هذا الباب التصميم العام للنظام، من حيث الهيكلية، وتدفق البيانات، ودور الذكاء الاصطناعي في الكشف عن الأمراض وتخطيط التغذية.

الباب السادس: يصف هذا الباب مكونات الهاردوير المستخدمة داخل النظام، مثل الحساسات، المتحكمات الدقيقة، ووحدات الاتصال.

الباب السابع: يناقش هذا الباب تنفيذ نظام الكشف عن الأمراض باستخدام نماذج الذكاء الاصطناعي مثل YOLOv8 والنماذج اللغوية الكبيرة (LLMs).

الباب الثامن: يقدم هذا الباب نظام تحسين التغذية، الذي يعتمد على تقنيات الوكلاء المتعددين (Multi-Agent) لتقديم خطط غذائية متوازنة وفعالة من حيث التكلفة.

الباب التاسع: يستعرض هذا الباب نظام المساعد الذكي (Chatbot) ، الذي يعتمد على نماذج لغوية كبيرة للإجابة على استفسارات المستخدمين والبحث في البيانات المهيكلة والإنترنت.

الباب العاشر: يعرض هذا الباب تطبيق الهاتف المحمول، وواجهة المستخدم، وآلية الاتصال مع النظام الخلفي لتوفير تحكم وتنبيهات لحظية.

الباب الحادي عشر: يتضمن هذا الباب ملخصاً لأهم نتائج المشروع، كما يناقش المقترحات المستقبلية لتطوير النظام وتحسينه.

الموجز

يشهد العالم تطورًا متسارعًا في مجالات تكنولوجيا المعلومات والاتصالات، مما جعل تبادل البيانات عبر وسائل الاتصال المختلفة ضرورة لا غنى عنها، خاصة مع اعتماد قطاعات حيوية مثل التعليم، والخدمات البنكية، والتجارة الإلكترونية، والإدارة الحكومية على شبكة الإنترنت. ومع هذا التوسع الرقمي، برزت الحاجة إلى حلول ذكية وآمنة تدعم هذه التطبيقات وتعزز كفاءتها.

نظامًا ذكيًا ومتكاملًا لإدارة الثروة الحيوانية، يعتمد على تقنيات الذكاء **MooNet** في هذا الإطار، يقدم مشروع الاصطناعي وإنترنت الأشياء، بهدف تحسين الرعاية الصحية للحيوانات، وتوفير تغذية مثلى، ورصد الظروف البيئية في الزمن الحقيقي، مما يساعد المزارعين في اتخاذ قرارات مستنيرة وفعالة.

يتضمن النظام وحدات لرصد الأبقار باستخدام نماذج كشف بصري، بينما يتم تشخيص الأمراض مثل التهاب الضرع والجلد العقدي من خلال نماذج تصنيف متقدمة تعتمد على تقنيات التعلم العميق. كما يشمل النظام وحدة لتحسين التغذية تعتمد على وكلاء ذكيين، ومساعدًا رقميًا تفاعليًا يقدم الدعم للمزارعين من خلال فهم اللغة الطبيعية، إلى جانب تطبيق مخصص للهواتف المحمولة يتيح تحكمًا لحظيًا وتنبيهات فورية.

بين تقنيات الرؤية الحاسوبية، وتحليل البيانات، والتفاعل اللغوي الطبيعي، ليقدم حلًا موحدًا MooNet يجمع وفعاليًا للزراعة الذكية. ويُعد هذا المشروع خطوة نحو تحسين رفاهية الحيوانات، وتقليل التكاليف التشغيلية، ودعم... التحول الرقمي في القطاع الزراعي، لا سيما في الدول النامية مثل مصر والمنطقة العربية.